BATTLESHIPS AI

H446 C03 Programming Project

Details

Year: 2025/2026
Candidate Number: [redacted]
Centre Number: [redacted]
Centre Name: [redacted]

[redacted]

Table of Contents

Table of Contents 1
Analysis 4
Project Overview 4
Stakeholders 4
Research of existing solutions 5
Research Overview 5
Research of Alishan Ladhani's Battleship Game 5
Research of Battleboat.js 7
Research of C. Liam Brown's Battleship 9
Core Features of this project 12
Limitations 14
Computational Methods 16
Thinking Ahead 16
Thinking Abstractly 16
Thinking Procedurally 17
Thinking Logically 18
Hardware & Software Requirements 18
Hardware Requirements for Development 18
Software Requirements for Development 19
Hardware Requirements/Recommendations for End-User 19
Software Requirements for End-User 19
Gaining insights through an interview 21
Interview on 18/06/2025 21
Review of the interview 22
Success Criteria 23
Design 25
Modules Overview 25
Top-down Decomposition Diagram 25
Details of each module from the diagram 26
Data Structures & Key Variables (before development) 27
Classes 29
Class Diagrams 29
Explanation of class diagram 30
Database Structure 31

Data Tables....31
Database Entity Relationship Diagram....33
Algorithms....34
Algorithm Flow Charts / Pseudocode....34
GUI Design....45
Explanation and Usability Features....45
Acceptance Testing Plan....46
Development....48
Development Plan....48
Initial Setup....48
Initial work on classes: Attributes and Methods....50
Creating some core endpoints....57
Implementing game statistics....78
Frontend Development....83
Initial frontend structure....84
Implementing Database & Player Logic....103
Implementing a more challenging opponent....114
Evaluation....122
Testing for evaluation....122
Usability Features....130
Navigation....130
Screen Elements....131
How well does the solution meet the success criteria?....133
User Interface / User Experience....133
Gameplay....135
Navigation....136
Misc....136
Final Gameplay Testing Video....138
Maintenance....139
Current and future maintenance....139
Limitations and resolutions to them....139
Full Code Listing....141
Root directory....141
/logic....142
/routes....155

/public 160

Analysis

Project Overview

For this project, I am planning to create a web-based battleships game that allows a player to play battleships against an AI opponent. The game UI will be accessible on a webpage created with HTML, CSS, and JavaScript, accessible through a web browser. It will communicate with a Node.js program which will handle the server-side algorithms, such as starting the game, handling moves, and handing the AI opponent. The primary focus of this project is on implementing the AI logic that can provide the player with a challenge beyond what most basic player vs computer versions of the game provide, hopefully making the game feel less repetitive as it can often feel when playing against simple computer opponents. It could potentially even provide an opportunity for the player to test their skills and even develop new strategies. There are a few additional features that I am considering implementing, depending on time constraints and feedback from stakeholders. Some of these features include: score tracking system, adjustable difficulty options, and a database system for storing results and other relevant data that we want to retain across sessions.

Stakeholders

As the concept behind the battleships game is fairly simple and easy to understand, the target audience is broader than more complex games. The primary stakeholder would be a child/young adult (aged 8-19) who would like to play battleships against an opponent that is smarter than random guessing, so it can be played against like it's another player, without the need for someone to play the game with. The option of choosing a difficulty allows younger players to have a fair chance against an opponent with a difficulty matched similar to their ability. The game will be usable on a desktop/laptop browser, and most likely also accessible on mobile devices too (and optimised for mobile if time permits), enabling younger audiences to use it too. This means that realistically anyone would be able to play the game without too much difficulty accessing or navigating it, making it suitable for any age, but still aimed towards children.

I will gather feedback of potential stakeholders throughout the development process in order to guide my decisions on the design and functionality of the project. This will ensure the solution is best suited to the stakeholders, by ensuring the user interface is intuitive and clear, and adjusting the performance of the AI in order to best suit their expectations. I may also involve stakeholders experienced in programming to review the performance and efficiency of the program.

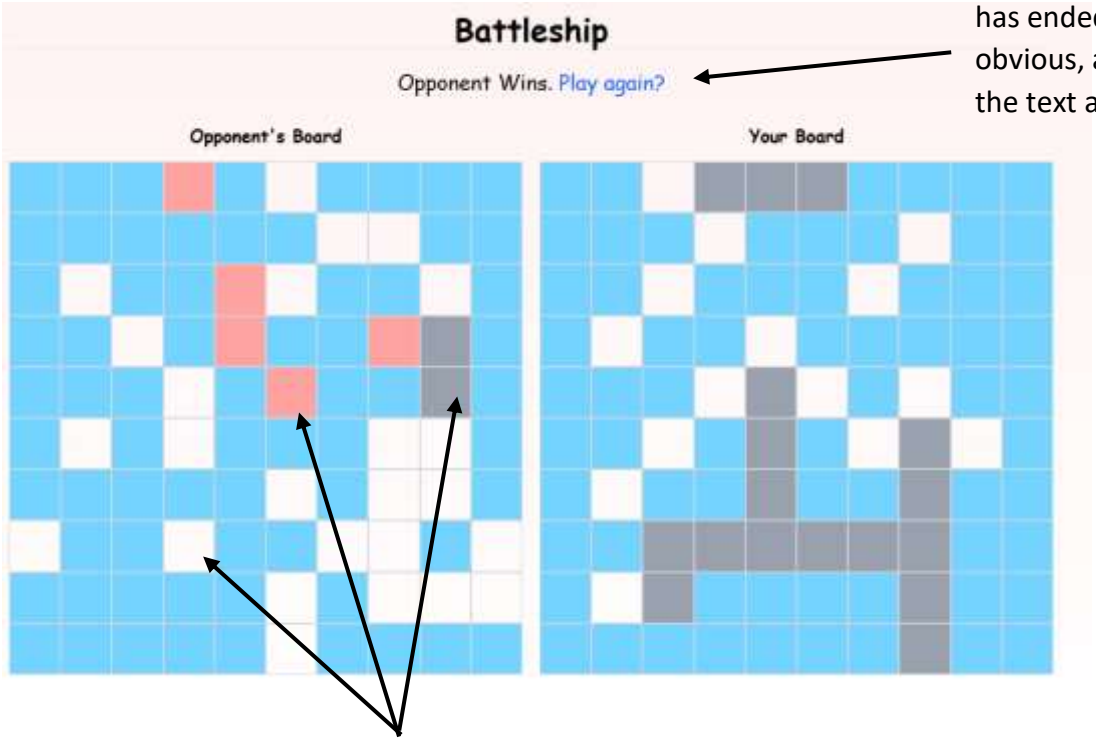
Research of existing solutions

Research Overview

I have researched and tried some existing online versions of battleships, including basic implementations containing random AI opponents, as well as more advanced projects that use probability-based algorithms or reinforcement learning. Most of the online Battleships games use a very basic AI as the opponent, or require a second player. This project should be able to provide a suitable web-based implementation of Battleships that allows the player to play against a human-like opponent, that uses more strategy than other simple AI opponents.

Research of Alishan Ladhani's Battleship Game

Battleship game is a very simplistic implementation of Battleships against an AI. The layout is extremely simple, consisting of just text and different coloured grid squares. It uses a very simple AI that will guess randomly until it gets a hit, then it targets adjacent squares. This suggests that a simple layout is important for usability. Because of this, my solution should use a large, clearly spaced grid with colour-coded grid cell states."



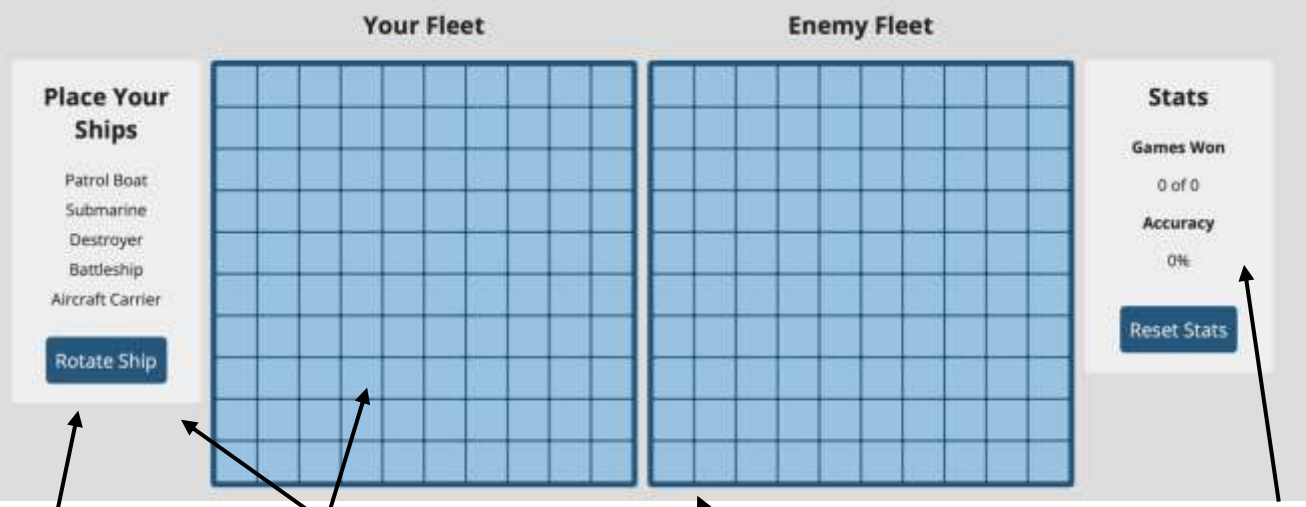
Indication that the game has ended is not very obvious, as it just changes the text at the top.

Colours change depending on if it's a hit, miss, or hit & sunk. However, the colours aren't very obvious as to their meanings. There is no kind of colour code explanation.

Overall, this implementation is a functional game, however the layout is suboptimal. The colour scheme looks very old, and as I aim to make my solution look modern, this layout may look too dated. From research of this existing solution, I have gained insights that I should use a similar grid system.

Research of Battleboat.js

Battleboat.js is a client-side implementation of Battleships against an AI player. It allows the player to play against a simple logic-based AI opponent, which makes moves that aren't too similar to how a human might play. It however has a simple, easy to understand UI, as seen below:

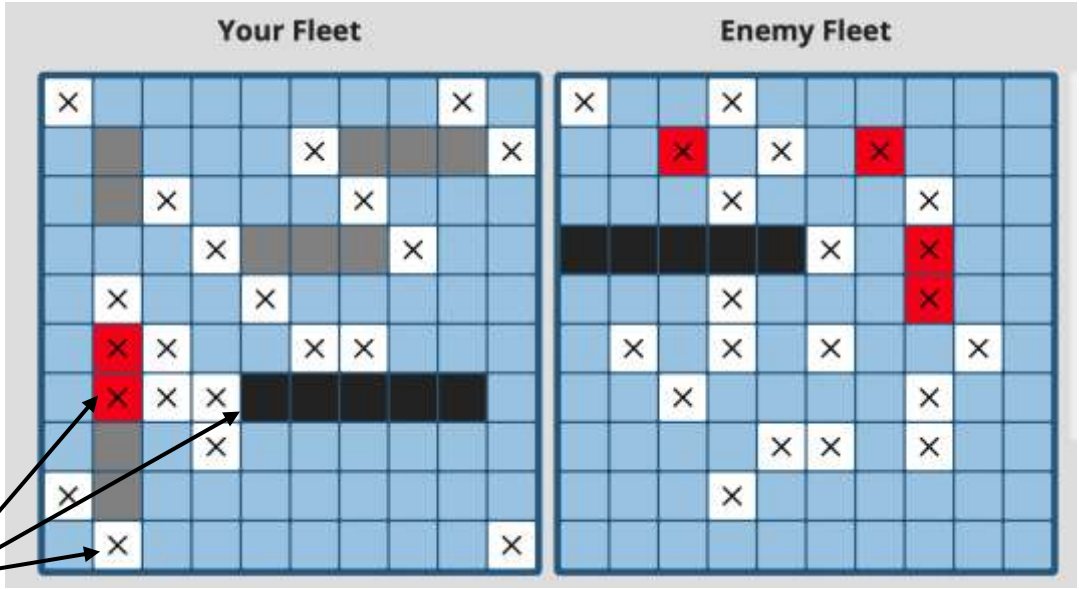


Having to click "Rotate Ship" with no keyboard key alternative is a bit limiting, making the game less intuitive

Only 2 3 colours to make the UI easily

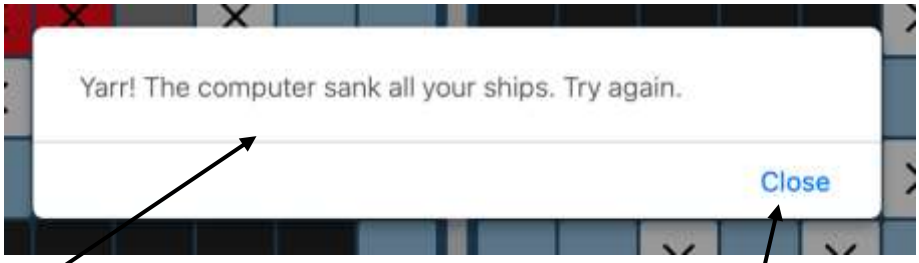
Layout makes use of the majority of screen space, allowing it to not be too crowded in one place

"Stats" section, providing statistics about previously played games



The different grid square states are represented by colours, ensuring the player easily understands if they have hit or sunk a ship.

No sound or animation plays when a move is made. This would be a nice addition to make the game more interesting



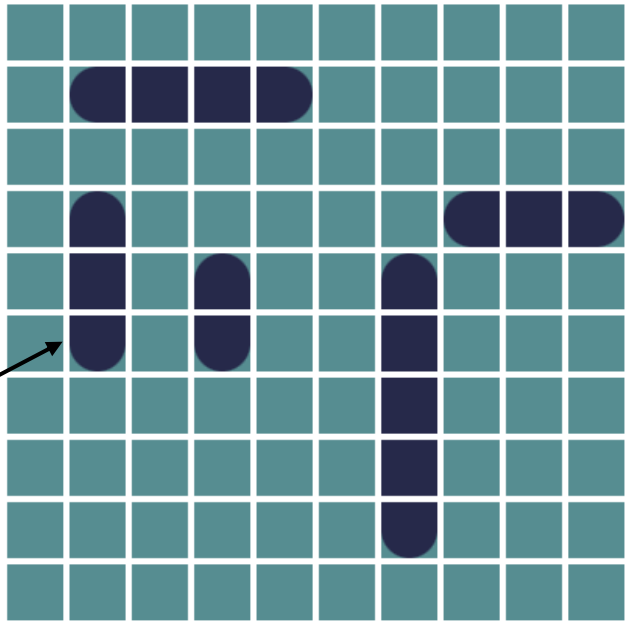
The message displayed when a game ends is a bit too simple, as it's just a simple browser alert message

More explanatory options such as "Play Again" would improve the user experience

This existing implementation provides a good insight into relevant features that I may have previously overlooked. It however does lack some features which I feel would improve the experience and usability of the game, such as a more intuitive ship rotation control, and a better post-game message. The UI is laid out well, in a way that I can take inspiration from. It takes up most of the display, allowing it to be spaced out, making it easier to click where intended. Therefore, my solution should use a similar UI which is easy to navigate, as this existing solution is very good for usability.

Research of C. Liam Brown's Battleship

C. Liam Brown's implementation is the closest to what I want my solution to be. It has a clearly laid out user interface, with hints to help navigate and interact with the UI. The game itself allows you to play against various AI opponent: random, probability based, and machine learning. This is a very good feature as it allows for some form of difficulty control, albeit in big steps of opponent difficulty. Similarly, my solution should implement similar difficulty levels, as for this existing solution, it helps to provide more variety to the game.

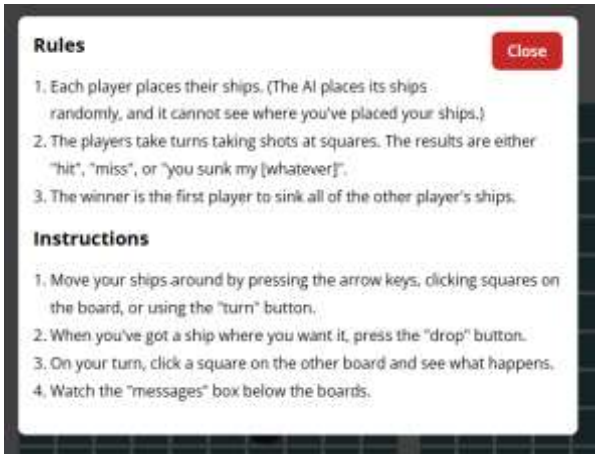


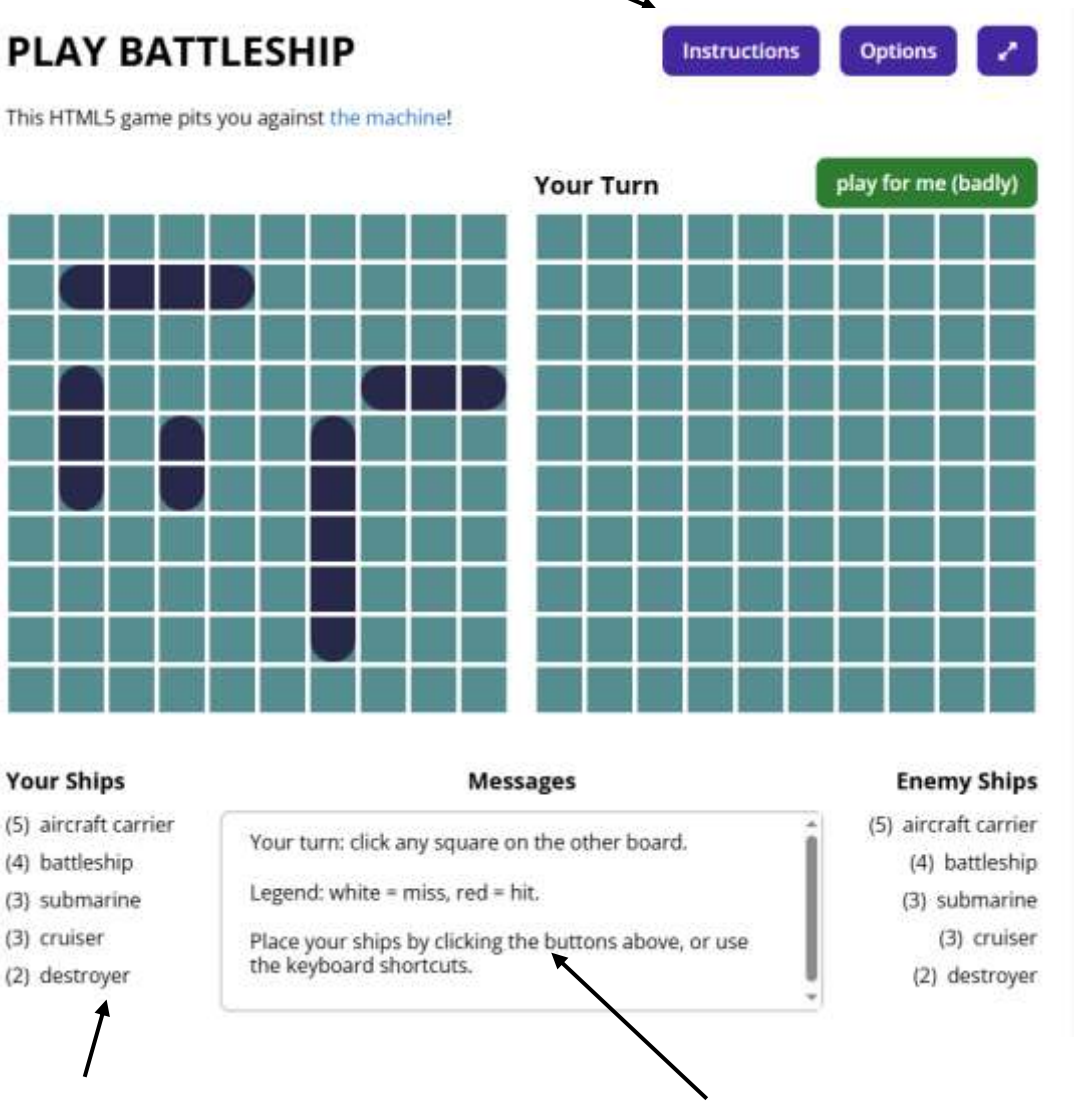
Ships have a nice rounded shape, making the design look more modern



This implementation provides clearly labelled buttons, and even provides keyboard key alternatives, which makes the game less effort to use.

Clicking the instructions button will show the user a box containing the rules and instructions, allowing new players to learn how to play.

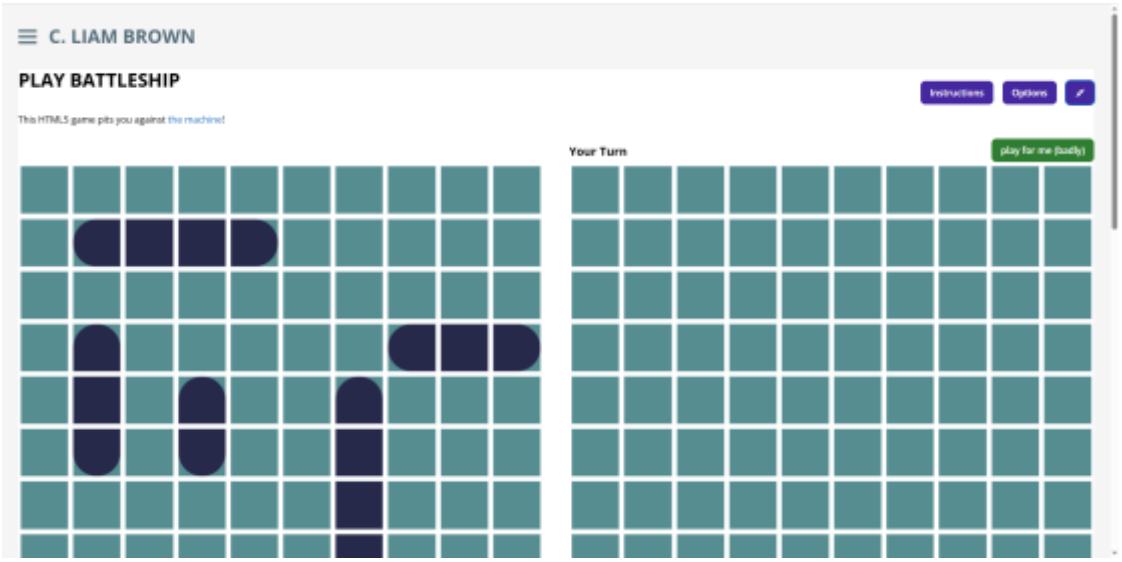




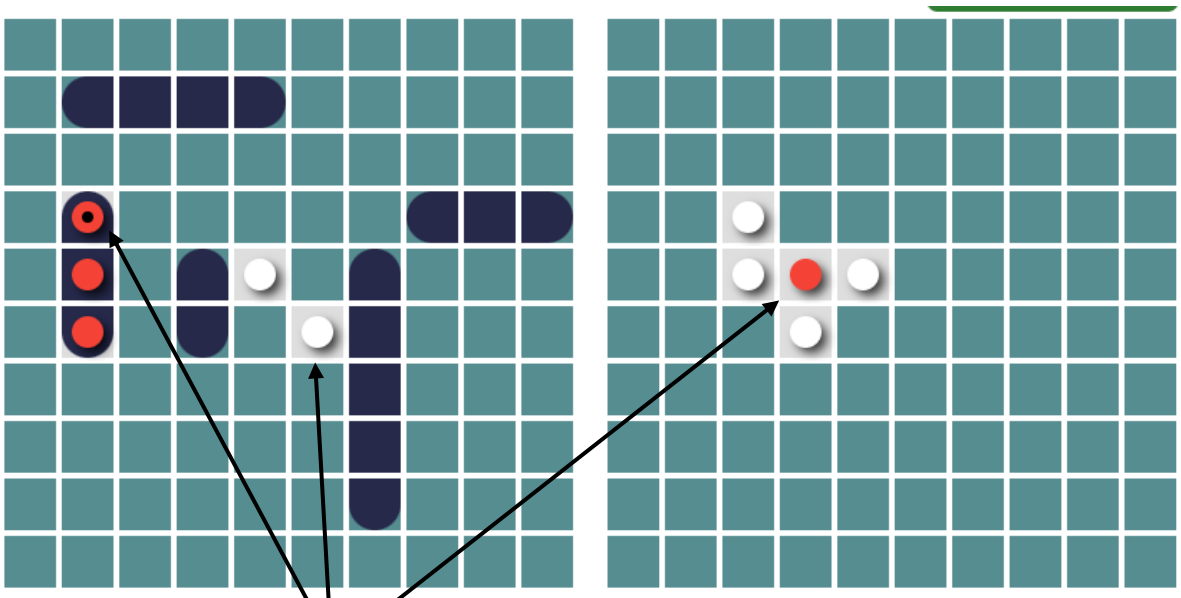
The list of ships gives the player more information on what ships they have on the board and which have been sunk.

The message box provides useful hints on how to play, the colour code, and also outputs messages after each move, such as “You hit my cruiser”. This helps to reduce the learning curve, helping new players to play, without needing to figure out how to play without guidance.

Clicking the full screen button in the top right of the game UI resizes the game to take up the full screen, hiding the menu and other irrelevant UI elements. It however isn't optimised very well, as scrolling is required to see the full board. The screenshot below demonstrates this.



Board is cut off, requiring the user to scroll down, therefore losing some UI elements at the top instead



This implementation has the best colour scheme of all the implementations I have researched.

Its easy to understand what each colour means, even without seeing the legend.

Core Features of this project

This table contains the core features of my project that need to be completed in order to provide an effective solution.

Feature	Explanation	Estimated complexity
Visual game UI with grid and other UI elements	This is critical because it allows the player to easily interact with the game in a clear, simple visual UI.	Low
Server program	This is necessary to make the game functional. While client-side processing is still needed, this will enable the game run procedures that need to be processed server-side.	High
AI opponent (at least random targeting)	An opponent is needed to allow the player to play Battleships without needing another player. The AI opponent needs to be at least functional, if not a more advanced AI system. As there is no PvP mode being implemented, this is entirely necessary to implement an opponent algorithm.	Moderate-High
Game turn logic for giving the player and AI moves in turn	This is a basic part of the original Battleships game. The game needs to let each party take their turn before moving onto the other party. Without this, the game won't function correctly, so this is necessary to implement.	Moderate
Ship placement system	The player needs to be able to choose where to place their ships, in order for the game to be fully interactive. The server program needs to handle whether or not the ship placements are allowed within the game rules. Without the ship placement system, the game is not in a usable state for players to use.	Moderate
Mouse control	From evaluating other implementations, clicking boxes in the game grid seems to be the most user-friendly way of accepting input from the user in terms of choosing grid spaces to attack. This helps with usability and means no keyboard controls are needed.	Low
Game over screen	This will inform the player if they have won or lost, and how long the game lasted for. This is necessary for informing the user when the game ends.	Low

If those core features are completed with a suitable amount of time remaining, and I am able to allocate time to this project amongst other commitments, then I will ideally try to implement some of these following features.

Feature	Explanation	Estimated Complexity
Difficulty options	This would be a useful feature to make the game more tailored to the player, but it may require multiple different AI versions/models which may pose to be a challenge in terms of time constraints.	Moderate
Score saving	Saving previous scores in a SQL database would help to improve the user experience.	Moderate
Move history and statistics	Statistics about the current game such as which grid squares are being hit could be interesting for more advanced players, but is likely less important than other features	Moderate
Simple Visual/Sound effects	This would make the game more entertaining and immersive. However, it isn't an important feature	Low
Time limit for moves	This would be an optional feature with potentially different time limit settings. This could make the game a lot more fast-paced and allows it to be more unique and less boring.	Moderate

Limitations

The following table contains some requirements which will not be possible due to time constraints and gaps in my skills. These features could be implemented further down the line if I continued to maintain and improve this solution after completion.

Requirement	Justification
Online multiplayer (two players over the internet)	This would allow two players to play against each other over the internet, which would potentially make the game more engaging

In order to achieve this requirement, I would need to implement a full client-server architecture with networking. Each client would make requests to the server. The underlying architecture for the client-server system will already be implemented, but it would need to be expanded and modified in order to handle two clients and the communication between them. Due to the complexity and time constraints of this project, this feature will not be implemented.

Requirement	Justification
AI learns from previous games	This would make the AI feel more human-like by adapting to use previous games to improve and adjust its strategy against the player

In order to achieve this requirement, I would need to implement a reinforcement learning algorithm. This would allow the AI to store and use past moves and their outcomes in order to improve the AI model. This however requires some complex algorithms and data handling, which is not really feasible with the scope and timeframe of this project. This is however a feature I would look to implement as a later feature after completion of the project.

Requirement	Justification
Advanced graphical effects and animations	This would improve the overall visual experience for the user, making it more entertaining

In order to achieve this requirement, I would need to create sprite sheets for each animation and effect, and update the grid to display these effects in an animated form. The sound effects would also need to be synchronised with the animations. While this is possible, it would require a large amount of development time which could otherwise be put towards more important/core features of the solution.

Requirement	Justification
Full mobile compatibility	This would ensure the game is fully optimised for mobile, without any major UX issues or bugs

In order to achieve this requirement, I would need extensively modify the style to make the UI scale to any mobile device screen, as well as ensuring all UI elements are scaled correctly with no clipping issues or overlapping UI elements. Game instructions would also need to be adapted to show mobile controls. While the project will likely work on mobile, it will not be specifically optimised with full mobile compatibility as this would require a significant amount of time that could be put towards more important optimisations.

Requirement	Justification
Replay system	This would allow users to watch past games which would allow them to improve their strategy, and overall make the system more interesting

In order to achieve this requirement, I would need to save every move of the game and save it in a structured format. It would then need to be read and played through retrospectively. This would require extra file storage and many new Classes and procedures. While this is possible, it is not necessary to meet the requirements of the project, as it is not vital for the stakeholders.

Computational Methods

This project can be developed using computational methods. This is because the project will need to be abstracted and decomposed in order to make it a viable project, as each section can be focused on, piece by piece. The project can be worked on in a logical way, working through each component.

Thinking Ahead

Thinking ahead and having an idea of how I would like to work on this project will allow me to have a rough plan on what I need to do and the steps involved for each part of the process.

- I will be using Node.js for the backend processing of the system, as this is a robust programming language/framework that I am familiar with.
- I will use a web-based UI written in HTML and JavaScript as this is a straightforward option and it integrates well with the Node.js backend server through web requests.
- The game will likely use the keyboard and mouse to control ship placement and choosing where to attack.
- The outputs may contain sound effects if this proves to be a feature desired by stakeholders in feedback questionnaires.

Thinking Abstractly

For this project, I will need to review what components are necessary and need to be prioritised, and which components I can categorise as additions that may be beneficial if I have time to implement them.

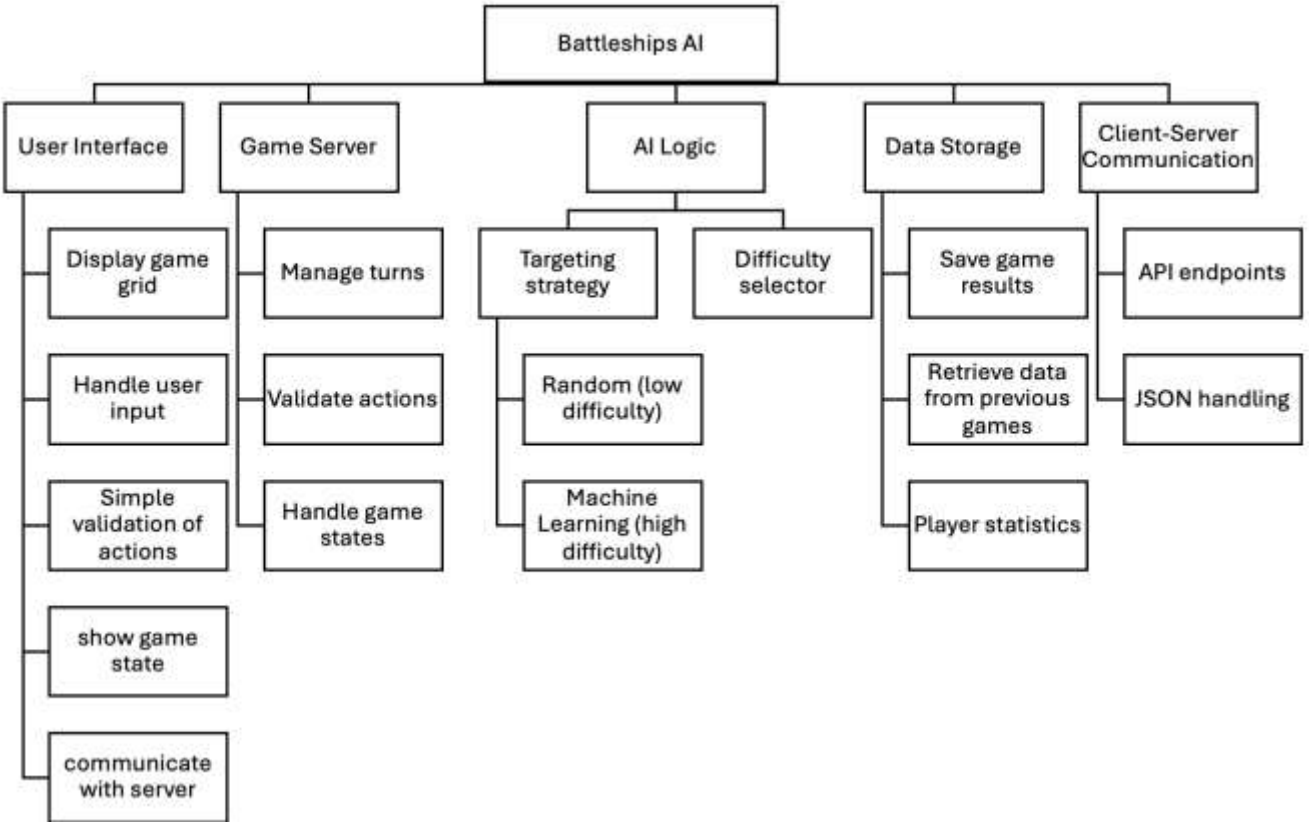
A few key points are as follows:

- Don’t create a 3D or otherwise unnecessary UI as this will take up time which could be used on more necessary components.
- Ensure all the key elements exist such as a menu and basic game layout.
- Remove unnecessary visual/sound effects which do not add any important addition to the game.

Thinking Procedurally

This helps to structure the problem, which can in turn make it easier to identify individual components of the problem, helping to make the development process more efficient.

The top-down diagram below demonstrates how the overall problem can be broken down into smaller components.



The project has been broken down into 5 main components. These components are distinct components that can function independently, and do not necessarily need to be developed in a specific order.

I broke down the solution into these elements because it will aid with designing the solution, helping me to decide which algorithms need to be reusable and independent, and which parts are dependent on other parts of the solution.

User Interface:

- This allows the user to interact with the game, by displaying a visual grid and interface that can be easily and intuitively used by a player with any level of experience. It will also contain simple action validation to reduce the amount of unnecessary requests being made to the server. This part of the solution is necessary because the user needs to be able to visually interact with the game. The user interface will contain buttons and text, as well as the grids containing ships. It will perform simple validation, handle user input, and communicate with the server through HTTP requests.

Game Server:

- This is vital to the functionality of the game, as it handles all the communication and functionality of the game itself. Without it, the game cannot function as the main game handling will not be handled client-side.

The game server listens for requests from clients, and respond with appropriate responses. It will handle the state of the game and validate actions.

AI Logic:

- This is important as the player needs an opponent for the game. This will involve developing and training the AI if I decide a ML AI opponent is viable. If it proves to be too time consuming, it may instead involve creating a simple rule based or random opponent. The opponent will choose grid cells to attack, and these will be reflected to the frontend and updates the current game grids. As shown in the top-down diagram, there will be a difficulty selector to choose between a low difficulty mode and a high difficulty mode.

Data Storage:

- If I have time to implement data storage features, then this can be broken down into the separate components of saving game results, retrieving data, and potentially retrieving statistics about the player such as their win rate. The data storage will store game results as well as statistics for players.

Client-Server Communication:

- This is necessary because it enables the server program to communicate with the client JavaScript, allowing the game to function as planned. This would involve creating and utilising endpoints, and handling JSON data used in requests and responses. This part of the solution will contain API endpoints on the backend which will be interacted with from the frontend.

Thinking Logically

Thinking logically helps me to understand the desired process and how it can be developed.

For example, when the user selects a grid space to target, the client-side JavaScript should make a GET request to the server, which will validate if the move is acceptable, such as a valid grid space, and that it's the player's turn. It will then check if a ship was present in that space, returning a status. This result will then be returned back as a response to the initial request. The client can then process this data to display a response to the user depending on what data was returned.

Hardware & Software Requirements

Hardware Requirements for Development

Hardware	Explanation
Desktop PC / Laptop	Required for coding and running the Node.js server. Also required for testing from the browser. A mobile device would not be sufficient for development.
Monitor	Required for viewing the UI, and designing the layout. A laptop screen also is sufficient, but some form of display is necessary.

Mouse & Keyboard	Required for development and for game input in the testing process. This is necessary because otherwise the solution cannot be interacted with for testing.

Software Requirements for Development

Software	Explanation
Node.js	Node.js must be installed for running the server logic. I am using Node.js because I am most comfortable with it as a backend, and it is quite simple to set up and use.
Code editor (e.g. Visual Studio Code)	Recommended over standard text editing software due to its additional capabilities to aid in the development process when writing code for the project. This is necessary to speed up development times by aiding with debugging.
Database software (e.g. MySQL / MongoDB)	Required for storing and retrieving game results. Only necessary if this feature is implemented.
Browser (e.g. Chrome / Edge)	Required for testing the client-side interface and interaction during development. A browser is being used because a web UI is the easiest approach for the frontend, which requires a browser.

Hardware Requirements/Recommendations for End-User

Hardware	Explanation
Desktop PC / Laptop / Mobile Device	Required for accessing the web-based UI. As the solution uses a web-based UI, any modern device should work.
Keyboard & Mouse / Touchscreen	Needed for interacting with the game, such as placing ships and choosing grid spaces to attack. This is necessary because all interactions in the solution require either a keyboard & mouse, or touchscreen touches.
Speaker / Headphones	Optionally needed for audible sound effects. This is necessary because otherwise the sound effects won't be audible.
20MB+ free disk space	Ensures all assets required by the battleships game can be downloaded to the client device. 20MB is a rough estimate of the total size of the assets. This is important because otherwise the solution will fail to load on the end-user's device.
4GB+ RAM (recommended)	An average web browser requires a few hundred MBs to run smoothly. Modern devices tend to have at least 4GB of RAM so this shouldn't be an issue. This is important because otherwise the solution may fail to load or have poor performance.

Software Requirements for End-User

Software	Explanation

Browser compatible with HTML5 (e.g. Chrome, Safari)	The game will run in a browser using HTML, CSS, and JavaScript. A modern browser is required as outdated browsers such as Internet Explorer may not perform as expected.
Internet access	Required for connecting to the webpage and server if not running locally, which is the case for the end-user. This is important because the solution is accessed through the internet, as it is not an offline game.

Gaining insights through an interview

Interview on 18/06/2025

In order to gain more insights into what features are the most important, I asked another student who is interested in the game to answer some questions so that I have some initial feedback from potential stakeholders on what may be required in the solution, so I can prioritise features that are actually desired by potential users of the solution.

General Usage

- Have you played battleships before?
  Yes
- Would you be interested in playing a version of Battleships against a computer?
  Yes
- In what situation would you want to play this game?
  "Probably when I'm bored at home"
- Do you prefer playing games alone or with other people?
  "With other people but if I have no one to play with then NPCs are almost the same"
- What do you think makes a good Battleships game fun/engaging?
  "When it works well and when I have someone to play against or if it's against the computer"
- How important is it for you to have different difficulty levels for an AI opponent?
  "Probably fairly important. I'd still play it with only one option but multiple difficulties would be useful"
- Would you want to see your score or a win/loss history of previous games?
  "Yes that would be good because then I can see how I've done and if I'm improving"
- Would you prefer a fast-paced game or one with no time limits for each move?
  "Both would be cool but if it can only be one of those then probably no time limit would be better"

Opponent Behaviour

- Would you prefer the AI opponent to behave randomly or more strategic/challenging?
  "Definitely more strategic. Otherwise it would get boring very quickly if it was random"
- Should the AI learn from your moves and adjust it's strategy, or just follow a fixed strategy?
  "Learning from my moves would be cool because it would make it more like a human, but it could also make it too difficult potentially"

Interface & Accessibility

- Would you want to use the game on a computer or a mobile device?
  "Probably a bit of both. I'd use it on my computer when I'm at home but maybe on my phone in other places"

- Do you have a preferred browser that you use most often?
  "Yes Chrome is the only one I use on my computer and on my phone I use Safari"

- Would you want the game to have visual/sound effect?
  "I guess that would add something interesting to the game but I wouldn't be disappointed if there weren't any visual or sound effects"

- Do you need any kind of accessible layout, such as colourblind-friendly buttons or a simple layout
  "No. But I guess a simple layout always looks good"

Data & Progress Tracking

- Is saving and remembering your name a feature you'd like to see?
  "I don't really mind but it could make it more personalised"

- Would you like the option to reset your data, such as your statistics?
  "I wouldn't really ever need that but other people might want that"

Review of the interview

Through that interview, I can identify some key insights.

- Will likely be played on both computer and sometimes mobile.
- Difficulty options may be well received by players.
- Both a time-limited and unlimited time mode would be appreciated by players.
- Players would like a strategic AI opponent.
- An opponent that learns from the player's moves would be desired by players.
- Visual/Sound effects would be seen as a beneficial feature but isn't necessary for players to enjoy the game.
- A simple layout is desired.
- Saving/tracking data is not very important to players, so should not be high priority.

Success Criteria

Criteria Number	Requirement	Explanation	Reference	Category
1	Easily navigable UI	This ensures stakeholders can play the game without too much confusion, or needing instructions. This is quite critical.	A feature in existing solutions such as C. Liam Brown's Battleship; Interview on 18/06/2025	Usability
2	Client and server communicate through HTTPS requests	This is an effective way for the client to interact with the backend server in a way that is compatible with as many devices as possible	Project plan	Functionality Robustness
3	Different difficulty levels	Allows players of all levels of experience to play without a disadvantage, keeping it engaging. This isn't a critical feature but from the interview, it appears to be a desired feature.	Interview on 18/06/2025	Functionality
4	Win/loss history page (optional)	Allows players to see how they have improved across multiple games	Interview on 18/06/2025; Project plan	Usability Functionality
5	Easy opponent	An easy opponent should be present for inexperienced players	Project plan; Other solutions	Functionality
6	Strategic AI opponent	The AI should use a proper strategy to play, rather than randomly targeting grid squares. This is semi-important but could be skipped in worst case scenario.	Interview on 18/06/2025; Project plan	Functionality
7	Mobile and desktop compatibility	The game should function correctly on both desktop and mobile without major issues, and should fit on the screen without overlapping elements. However, mobile usability depends on time constraints as it is low priority.	Interview on 18/06/2025; Project plan	Usability
8	Simple layout	The layout should be clean and simple with not too many unnecessary visual elements.	Interview on 18/06/2025	Usability
9	Clear visual responses to actions	Players should be able to see the result of their actions, with a visual change and maybe sound effects too.	Interview on 18/06/2025	Functionality Usability

10	Clear instructions for playing the game	Allows the player to understand what they need to do without prior knowledge of the game	Project plan; Other solutions	Usability
11	Stable performance	The game should be able to run smoothly without glitching or causing frequent errors. It should also load within 5 seconds. This is very important for user experience.	Project plan; Other solutions	Robustness
12	Game result screen after game ends	This shows the player that the game is over and shows them who won, potentially with additional statistics such as total time, etc.	Project plan	Functionality
13	Using the mouse to select grid squares	This is an easy and intuitive way of interacting with the game	Project plan; Hardware requirements	Usability Functionality
14	Time limit for games stored in memory	This ensures there are no memory leaks, ensuring fast and stable performance. This is less critical though so could be skipped if needed.	Project plan	Robustness
15	Stable and consistent ship placement system	This makes the game less confusing as the rules don't change unexpectedly	Project plan; Other solutions	Robustness Usability
16	Accessible UI	This makes the UI easier to understand and navigate for all users of different abilities	Project plan; Other solutions	Usability
17	Sound effects for different actions	This provides an audible indication of what the player's action has done.	Project plan	Functionality Usability

Design

Modules Overview

Top-down Decomposition Diagram



By producing a top-down diagram, I am able to break down the project into clear tasks to complete individually. This makes the project more manageable and structured. I can therefore use algorithmic thinking to better decide on how to produce these modules as I am aware of which inputs and outputs will be required.

Details of each module from the diagram

Main game page

- Start Game – Initialises the game
    - Grid – Displays the game grid. Each square should be initialised ready to store information of its current status. I chose to use a grid system because this is the least complex way of representing the data necessary.
    - Visual effects – Add and play visual effects for different events. For example, a smoke effect when a ship is hit. This is not necessarily required but could improve user experience, helping it to be better suited to the stakeholders.
- Turn System
    - Validate each turn – verify to make sure the player's moves are valid. For example, the player shouldn't be able to choose a grid square that has already been used. This is important to prevent misuse.
    - Mark grid squares - It should also display this status visually with a colour. This is needed to make the solution usable and accessible by making it less confusing.
    - Poll for AI turn – Repeatedly make requests to the server to retrieve the AI's move, then update the board to show the response visually. This assumes that the AI's moves take a while to process.
- Place Ships
    - Validate ship placement – Make sure the ships fit in the grid and don't overlap or touch. The placement must be in line with the rules, possibly validated client-side and also server-side.
    - Rotate ships – Allow the user to use their keyboard or mouse to rotate the ships before placing them. This helps with usability.
- Difficulty – Allow the player to select a difficulty level. The level they choose changes the opponent (random moves, or ML)
- End of game summary
    - Display who won – When the player or the AI opponent sinks all the opponents' ships, display the winner.
    - Display total game time – Record game time, and after the game ends, display the time the game lasted.
- Game controls Display the game controls and other instructions on how to navigate the game interface. This helps with making the solution easily accessible to all users.

Stats page

- Display win/loss history shows a record of all previous games, stored in the backend server. This means the user can see if they have improved or not. May show additional statistics such as percentage win rate, etc.
    - Retrieve data from server – Sends a request to the backend server which will retrieve the win/loss history and other statistics from the database and return the data in the response.

Data Structures & Key Variables (before development)

The following table contains the methods and key variables I expect to use in the project. Some methods or variables shown here may be different once I start developing the solution as different requirements may be needed depending on the course of the development. I created this table because it gives a rough guide to aid my development, as I will know roughly what data structures and variables should be implemented.

Method / Variable Name	Data Type	Explanation	Justification
gameState	String	Stores which state the player is currently in (e.g. “menu”, “placingShips”, “playing”, “end")	Allows the program to keep track of what state the player is in, to know what procedures are needed next.
grid	2d array	Represents the 2 boards: the player’s board and the AI opponent’s board. Each item in the 2d array stores data about if the grid square is empty, has a ship, or has been hit/sunk.	This is a core part of the game, and serves as an efficient way of reading various important values about the game. This is more efficient than other data types.
ship	Class	Stores information as attributes, such as the size of the ship, the coordinates, and which parts have been hit	Using objects allows for easy creation of new ships as certain attributes can be predefined, and it also allows for methods to be used. This also provides scope for encapsulation.
ships	Array of ship objects	Stores each of the active ship objects	Ensures all the ship objects are accessible in one place, which can potentially be iterated over if needed.
isPlayerTurn	Boolean	True if it’s the player’s turn, and false if it’s the AI’s turn.	This is a simple way to determine who’s turn it is, and alternate between them. A simple Boolean flag is easier to modify and access than alternative data types.
move	Class	Stores information about a move such as the coordinates, the hit/miss status, etc.	Using a class is an easy way to store moves that all have the same attributes and methods
moveHistory	Array of move objects	Stores each of the moves made in the current game	Acts as a single variable to access all the moves and iterate through them for statistical data creation. It may also be useful for debugging purposes.
difficulty	Integer	Stores the difficulty as an integer to determine each difficulty level	Allows for easy management of which AI opponent to use. An integer allows more than 2 difficulties to be present.

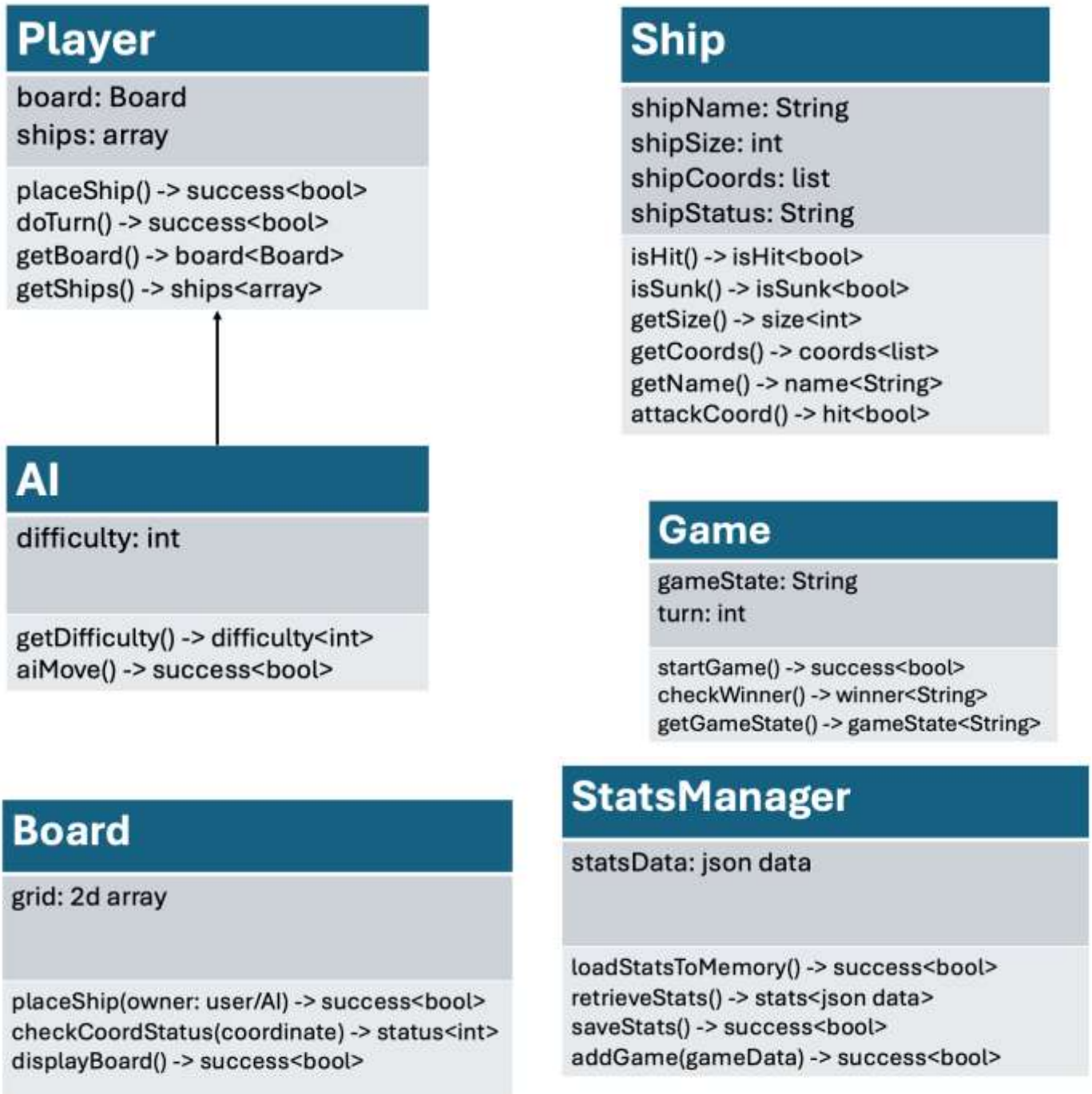
aiMove()	Procedure	Data about the game state is passed in, and the AI's move is returned	This keeps the AI logic contained in one place so it can be called easily during the game.
processPlayerMove()	Procedure	Data about the game state and requested player move is passed in, and a result is returned	This keeps the player move logic contained in one place to be called elsewhere to validate and finalise a player's move
validateShipPlacement()	Procedure	Ensures ships do not overlap and are in spaces that are allowed. This procedure will likely exist on both clientside and serverside (in different forms) to prevent cheating.	Prevents ships from being placed in invalid positions. This ensures the rules of the game are followed.
validateTurn()	Procedure	Checks that a move is allowed (e.g. isn't in a grid square that has already been chosen before)	This may prevent potential errors trying to perform an action twice, and it also improves the user experience. Using a procedure allows it to be reused in various places within the solution.
stats	JSON object (from SQL DB)	Stores the win/loss count and other stats such as average game time.	This allows data to persist between game sessions and allows the user to see their statistics. The database data will be converted to a JSON object to allow the frontend to access the data.
updateStats()	Procedure	Updates the user's statistics after a game ends using SQL to modify the DB.	Allows a permanent record of statistics to be stored for the user to see their performance such as win rate. Using a procedure allows it to be reused in various places within the solution.
generateShips()	Procedure	Initialises instances of each type of ship	Allows for other algorithms to easily initialise all the ships for the game by simply calling this reusable procedure. Using a procedure allows it to be reused in various places within the solution.

Classes

Class Diagrams

Below are diagrams of the classes I plan to implement. Arrows indicate inheritance between the classes. These class diagrams contain the class name, class attributes, and class methods. Methods will be used to read/write attributes where necessary, and the attributes themselves will be private, as encapsulation is best practice for classes.

The attributes and methods may differ to the final versions once created due to unforeseen requirements or changes that need to be made. A finalised version of the class diagrams will be provided during development.



Explanation of class diagram

The Game class controls the state of the game. This is important because it acts as a central class for all the core parts of the game, which can be implemented as methods. I could instead have chosen a procedural approach, but since a game is something that needs to repeatedly be initialised, a class works better for this.

Both the player and the AI opponent will have a Player class. This holds their board and ships. It provides methods for interacting with the game such as placing ships and taking their turns in the game.

The AI opponent will use the AI class which inherits attributes and methods from the Player class. This means that we can use the same methods for the opponent's moves as the player's moves, but the AI class provides us with additional attributes and methods, such as getDifficulty() which isn't required for the human player.

Each player object contains a board attribute which is an object of the Board class.

The Board class contains grid – a 2d array of the player/opponent's board. We can use the methods in Board such as placeShip() and checkCoordStatus() in order to modify the board. For example, checkCoordStatus() will return an integer, serving as the coordinate status. This will be either 0 for not attacked, 1 for attacked but no hit, or 2 for attacked and hit a ship. This is vital for ensuring the game rules are followed by not allowing the player to choose a grid square that they have already attacked.

Each player object also contains the ships array. This is an array of Ship objects, allowing us to keep track of which ships belong to each player.

The Ship class contains attributes such as the name, size, coordinates, and status of the ship. We can query these attributes using the methods, for example getCoords() will return all the coordinates that the ship occupies. The method attackCoord() will be used by iterating through each ship and attacking the coordinate. It will return a hit value which determines if that coordinate was occupied by the ship.

The StatsManager class will be used to manage game statistics, such as win/loss history. We can use the methods to query or save these statistics. We will first load the stats from the database to memory with loadStatsToMemory(). We can then retrieveStats() to return the stats stored in memory. We can addGame() to update the stats in memory, and then saveStats() to save from memory to the database.

Database Structure

Data Tables

Player Table

Field	Key	Data Type	Validation	Default	Notes
PlayerID	Primary	Integer	Not Null	AutoIncrement	Unique identifier for the player
DisplayName		String	3-50 characters	"User"	Name chosen by the player

Game Table

Field	Key	Data Type	Validation	Default	Notes
GameID	Primary	Integer	-	AutoIncrement	Unique identifier for the game
PlayerID		Integer	Not Null	-	Unique identifier for the player
Winner		String	-	"Unknown"	Stores the winner – "AI" or "Player"
Duration		Float	> 0 Not Null	-	Duration is stored in seconds, so float is a suitable data type
Timestamp		Integer	> 0 Not Null	-	Start date and time of the game, stored as an epoch integer timestamp

Stats Table

Field	Key	Data Type	Validation	Notes
PlayerID	Primary	Integer	Not Null	Unique identifier for the player
GamesPlayed		Integer	>= 0 Default 0	Running total of the games played
Wins		Integer	>= 0 Default 0	Running total of games won
Losses		Integer	>= 0 Default 0	Running total of games lost. While this isn't necessary, it can indicate any issues with data storage if GamesPlayed - Wins does not equal Losses

Important Additional Notes

- The Player table will use AutoIncrement for the PlayerID field. This means we do not need to manually create a PlayerID and risk creating a PlayerID that has already been used. AutoIncrement will automatically fill the field with an incremented integer for the PlayerID. This may not be best practice for a largescale solution, but for a small scale / proof-of-concept solution, it will be suitable.

- The default value for DisplayName in the Player table will be "User". This means that in an unlikely situation where the DisplayName value is not stored correctly, it will use the default value. This will prevent logic errors later down the line when accessing this value as a string, as if it was null it could cause problems.

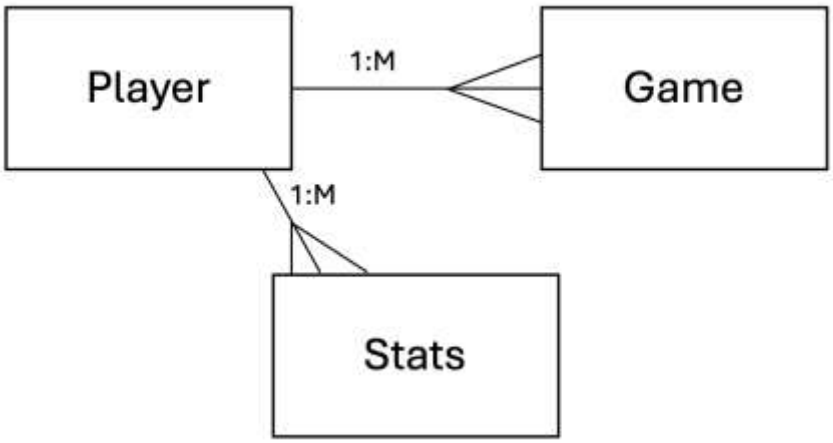
- The GameID field in Game table will also use AutoIncrement. This allows it to automatically create a unique ID without any additional logic in the backend program.

- The Result field in the Game table will have a default value of "Unknown". This means we can handle cases where the winner for some reason ends up as null. While these situations will be minimised through iterative testing, there is always potential for unusual effects.

- The Losses field of the Stats table is not necessarily needed. This is because we could just calculate the difference between total games and won games. However, by including this field, we can potentially spot issues with the data automatically and automatically resolve the discrepancies if needed. This may not be added for the first version of it, but if I decided to continue work on it in the future, I could add handling of incorrectly stored data.

- I have considered an AverageTime field for the Stats table to store the average time taken for each game. However, this is unnecessary as the average time can be calculated when requested by using the time taken for each game. This also allows for more control over what data we include, such as filtering out games with unrealistic times.

Database Entity Relationship Diagram



The database will be designed to store persistent game data such as player statistics and previous game results and time taken. Each player will have one stats record which will contain some basic data about their overall statistics. They will also have multiple game records, which stores data about each game. This allows the statistics page to display information about previous games that the player has played. I could have instead chosen a many:many relation, which would simplify the setup process for the database and reduce the complexity, but it could cause data consistency problems at a later stage. Because of this, I feel it would be better to use good practice of not having any many:many relations.

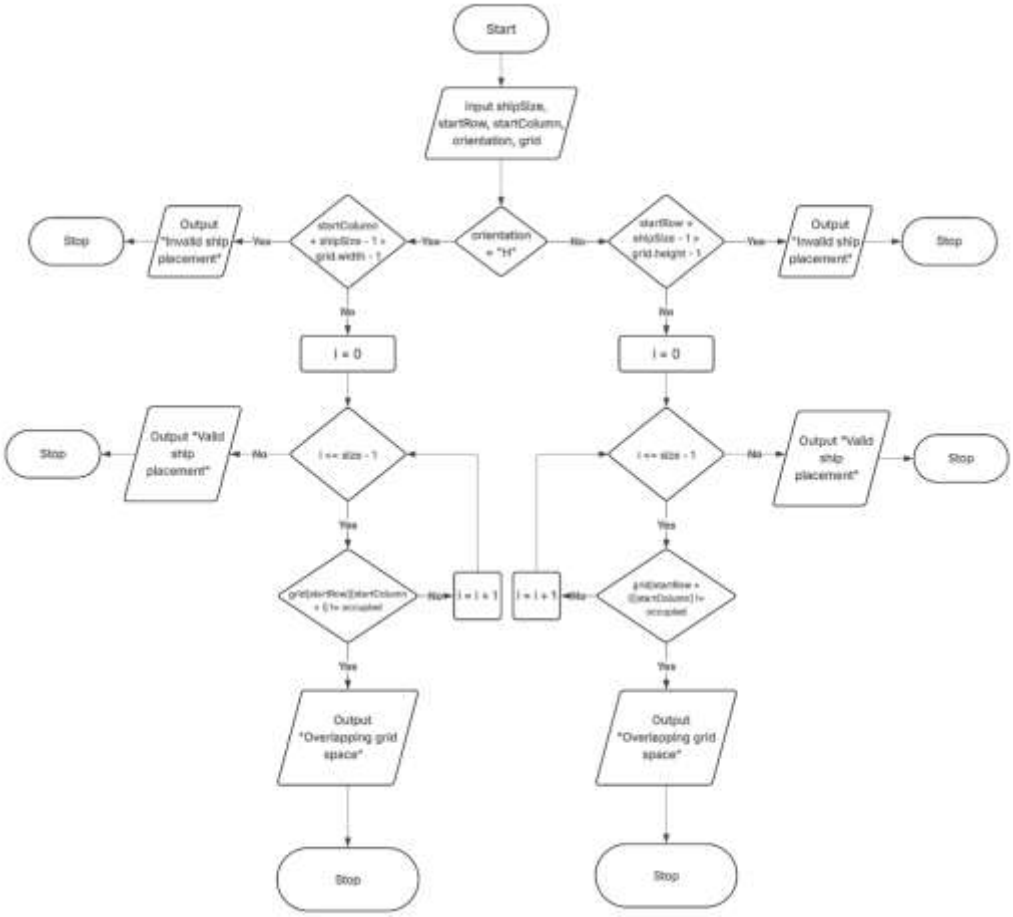
Algorithms

Algorithm Flow Charts / Pseudocode

Below are flow charts / pseudocode of the algorithms I plan to implement.

These algorithms may differ to the final versions once created due to unforeseen requirements or changes that need to be made. A finalised version of each algorithm will be provided during development.

Ship Placement Algorithm (Player)



This algorithm takes parameters of the ship size, the starting row, starting column, orientation of the ship, and the grid object to modify. Starting row and starting column may be later merged into one input of a 2d array, such as [3][2]. Some of these parameters will be calculated, while others will be inputted to the endpoint. I chose to use a 2d array because its an efficient and simple way of storing and handling the data. It can also be referenced in a logical way like a coordinates system.

The algorithm will then use selection to split into two parts, one for a horizontal ship, and one for a vertical ship.

In both branches, the algorithm will check if the ship will be able to fit in the grid using boundary checks. If the ship cannot fit, it will return “Invalid ship placement”. Otherwise, it will move onto a loop. An iterative variable, i, is initialised and used as our loop counter. For each grid square the ship will occupy, the algorithm checks if the grid

square is already occupied. If it is, the algorithm returns “Overlapping grid space”. Once all relevant grid squares have been checked, it returns “Valid ship placement”. The validation in this algorithm is necessary to prevent invalid ship placements, which would otherwise negatively impact user experience.

This is a simplified representation of the algorithm. When developing the actual solution, I will likely develop the algorithm to return a boolean value rather than a string. This means the solution can more easily handle different outcomes from the algorithm. I chose to return a string in this algorithm plan for simplicity and readability reasons. Furthermore, the “occupied” variable appears in the above flow chart. This is an abstract variable that will need to be defined in the finalised algorithm. At the moment, it is difficult to plan where this variable will be defined, as it may prove to be suitable to retrieve it from a different algorithm or module.

Testing Plan

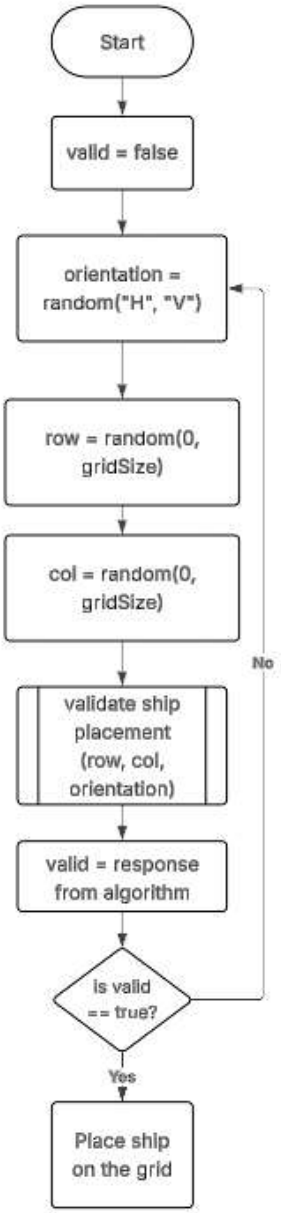
Test Number	What will be tested	Input	Type	Expected Outcome	Justification
1	Missing required values	gameId: a valid ID nothing else	Invalid	Returns an error	Handles misuse where required values aren’t provided to the algorithm
2	Correct request	gameId: a valid ID shipType: “Battleship” startRow: 0 startCol: 0 orientation: “H”	Valid	Returns success Success: true Message: “”	Needs to be checked to ensure the algorithm functions correctly
3	Passing a co-ordinate out of range	gameId: a valid ID shipType: “Battleship” startRow: 100 startCol: 0 orientation: “H”	Invalid	Returns an error	Handles misuse where an invalid ship placement shouldn’t be allowed
4	Passing an invalid orientation	gameId: a valid ID shipType: “Battleship” startRow: 0 startCol: 0 orientation: “test”	Invalid	Returns an error	Handles misuse where somehow an invalid orientation is provided and shouldn’t cause crashes
5	Choosing co-ordinates that are already occupied	gameId: a valid ID shipType: “Battleship” startRow: a co-ordinate already used startCol: a co-ordinate already used orientation: “H”	Invalid	Returns an error Message: “Invalid ship placement”	Ensures ships cant overlap
6	Passing invalid game ID	gameId: “12345” shipType: “Battleship” startRow: 0 startCol: 0 orientation: “H”	Invalid	Returns an error Error: “Game not found”	Ensures no crashes or errors occur when a move is made for an invalid game
7	Passing invalid ship type	gameId: a valid ID shipType: “test”	Invalid	Returns an error	Prevents misuse by ensuring invalid

		startRow: 0 startCol: 0 orientation: “H”		Message: “Invalid ship type	ship types don’t cause errors

The tests in the above testing plan will be performed both during development and after development, as they all aim to identify problems with the functionality, robustness, and usability of the algorithm, and all need to pass to ensure the algorithm meets the success criteria. As well as the above tests, this last test will be performed after development to ensure the algorithm works correctly consistently.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Algorithm functions correctly when used as part of the entire solution	gameId: a valid ID shipType: “Battleship” startRow: 0 startCol: 0 orientation: “H”	Algorithm functions correctly with no bugs	Ensures the algorithm meets the success criteria of stable performance

Ship Placement Algorithm (AI)



This algorithm does not take parameters, as it is used to randomly place a ship on the grid. It sets the variable "valid" to false, then repeatedly chooses a random orientation, row, and column, and uses a validate algorithm to validate whether or not the ship placement is valid. If it isn't, it continues the iteration. If it is valid, it will break the loop and place the ship on the grid. The ship placement validation is necessary to prevent the AI opponent from making invalid ship placements. This keeps user experience positive if all moves follow the rules of the game. I chose to use this algorithm because it's a simple way of choosing a random co-ordinate and then using the existing validate ship placement algorithm.

Testing Plan

Test Number	What will be tested	Input	Type	Expected Outcome	Justification
1	Algorithm consistency	N/A	Valid	Algorithm never returns errors	Ensures robustness of the algorithm

Validate Player Turn Algorithm

```
procedure validatePlayerTurn(row, col, grid)
    if row < 0 || row >= GRID_HEIGHT || col < 0 || col >= GRID_WIDTH then
        return {valid: false, msg: "Invalid move (out of grid)"}
    endif
    if grid[row][col] = "hit" || grid[row][col] = "miss" then
        return {valid: false, msg: "Invalid move (grid square already attacked"}
    endif
    return {valid: true, msg: "Valid move"}
endprocedure
```

This algorithm will check whether a chosen coordinate is in a valid grid square and that it hasn't been attacked before in a previous move.

This algorithm will prevent errors and duplicate attacks, which will in turn improve reliability and user experience.

For the algorithm design, I decided to return JSON objects, because this allows it to return multiple values. For example, this algorithm returns both a Boolean value for is/is not valid, and a message which we can use for debugging purposes by outputting it in the console, etc. The constants GRID_HEIGHT and GRID_WIDTH will also need to be defined, most likely as global variables, as these can then be accessed from any algorithm. I decided to use this algorithm design because it is simple yet still successfully validates a player turn by validating it.

Testing Plan

Test Number	What will be tested	Input	Type	Expected Outcome	Justification
1	Accepts valid data	row: 0 col: 0 grid: a valid grid	Valid	Valid: true Msg: "Valid move"	Ensures the algorithm functions correctly with valid data
2	Prevents out of range data	row: 99 col: 0 grid: a valid grid	Invalid	Valid: false Msg: "Invalid move (out of grid)"	Ensures the algorithm refuses invalid ship placement, ensuring robustness
3	Prevents already attacked cells	row: 0 col: 0 grid: a valid grid where grid cell 0,0 has been hit	Invalid	Valid: false Msg: "Invalid move (grid cell already attacked)"	Ensures the algorithm prevents cells being attacked more than once

The tests in the above testing plan will be performed both during development and after development, as they all aim to identify problems with the functionality, robustness, and usability of the algorithm, and all need to pass to

ensure the algorithm meets the success criteria. As well as the above tests, this last test will be performed after development to ensure the algorithm works correctly consistently.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Algorithm functions correctly when used as part of the entire solution	row: 0 col: 0 grid: a valid grid	Valid: true Msg: “Valid move”	Ensures the algorithm meets the success criteria of stable performance

Player Turn Algorithm

```
procedure playerTurn(row, col, grid)
        validateResponse = validatePlayerTurn(row, col, grid)
        if validateResponse.valid == true then
                if grid[row][col] == 1 then
                        grid[row][col] = “hit”
                else
                        grid[row][col] = “miss”
                endif
                return true
        endif
        return false
endprocedure
```

This algorithm will take user inputted row and column as parameters, as well as the grid being attacked.

The algorithm will first use the validatePlayerTurn algorithm to validate the turn to prevent errors and duplicate attacks, improving user experience and robustness.

The algorithm then sets the grid cell data based on whether or not there is a ship present (1 or 0).

For the algorithm design, I decided not to return any value. This assumes that no error occurs. In the final solution, I will likely need to return a value so that the parent function knows whether it succeeded or not, for effective error handling. I chose to go with this algorithm design because it’s simple and makes use of the validate player turn algorithm.

Testing Plan

Test Number	What will be tested	Input	Type	Expected Outcome	Justification
1	Accepts valid data	row: 0 col: 0 grid: a valid grid	Valid	Returns true	Ensures the algorithm functions correctly with valid data
2	Prevents out of range data	row: 99 col: 0 grid: a valid grid	Invalid	Returns false	Ensures the algorithm stops after validation fails
3	Prevents already attacked cells	row: 0 col: 0 grid: a valid grid where grid cell 0,0 has been hit	Invalid	Return false	Ensures the algorithm stops after validation fails

The tests in the above testing plan will be performed both during development and after development, as they all aim to identify problems with the functionality, robustness, and usability of the algorithm, and all need to pass to ensure the algorithm meets the success criteria. As well as the above tests, this last test will be performed after development to ensure the algorithm works correctly consistently.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Algorithm functions correctly when used as part of the entire solution	row: 0 col: 0 grid: a valid grid	Returns true	Ensures the algorithm meets the success criteria of stable performance

Simple (random) AI Moves

```
procedure AIMoveRandom(grid)
        repeat
                row = random(0, GRID_HEIGHT - 1)
                col = random(0, GRID_WIDTH - 1)
        until grid[row][col] != "hit" && grid[row][col] != "miss"
        return (row, col)
endprocedure
```

This algorithm will be used for the most basic AI algorithm which just uses random targeting. This will be the first AI algorithm to be developed, as a baseline AI algorithm, as it is the most simple to create and can be used for temporary testing of core components of the game, such as handling both user and AI moves and displaying them correctly. This algorithm may or may not be used in the final solution as a difficulty option for the player to choose.

The algorithm itself chooses random coordinates until it finds a set of coordinates that haven't yet been attacked before, so as to prevent duplicate moves.

Testing Plan

The endpoint will contain some logic before passing on the data to the AIMoveRandom algorithm. For example, the endpoint will accept a game ID, and the player's grid for that game will be retrieved and passed into the algorithm.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Correct Request on a game with player ships	gameId: a valid ID	Returns a row and column	Ensures the algorithm functions correctly when valid data is used
2	AI attacks only new co-ordinates	gameId: a valid ID	Every response shows a new co-ordinate	Ensures the algorithm's robustness and functionality by functioning as expected

The tests in the above testing plan will be performed both during development and after development, as they all aim to identify problems with the functionality, robustness, and usability of the algorithm, and all need to pass to ensure the algorithm meets the success criteria. As well as the above tests, this last test will be performed after development to ensure the algorithm works correctly consistently.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Algorithm functions correctly when used as part of the entire solution	gameId: a valid ID	Algorithm returns co-ordinates with no bugs or inconsistencies	Ensures the algorithm meets the success criteria of stable performance

Check for game over algorithm

```
procedure checkGameOver(ships)
        for ship in ships
                if ship.isSunk == false then
                        return false
                endif
        endloop
        return true
endprocedure
```

This algorithm iterates through all ships to see if any are still unsunk using a simple for loop. If all the ships are sunk, the algorithm returns true, which will then be sued to initiate the end of game algorithms.

This algorithm will be run after every move. As the ships array is quite small due to no unnecessary data being stored in it, the algorithm can be run frequently without any performance implications. I chose to use this algorithm because it's extremely simple as it just iterates through all the ships to identify if they are all sunk or not.

Testing Plan

The endpoint will contain some logic before passing on the data to the checkGameOver algorithm. For example, the endpoint will accept a game ID, and the ships for that game will be retrieved.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Correct Request on a game with remaining player ships	gameId: a valid ID	Game not yet over	Ensures the algorithm checks correctly that the game hasn't yet ended
2	Correct request on a game with no remaining player ships	gameId: a valid ID	Game over	Ensures the algorithm checks correctly that the game has already ended

The tests in the above testing plan will be performed both during development and after development, as they all aim to identify problems with the functionality, robustness, and usability of the algorithm, and all need to pass to ensure the algorithm meets the success criteria. As well as the above tests, this last test will be performed after development to ensure the algorithm works correctly consistently.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Algorithm functions correctly when used as part of the entire solution	gameId: a valid ID	Game not yet over / game over Depending on game state	Ensures the algorithm meets the success criteria of stable performance

Update player statistics

```
procedure addGame(winner, gameDuration)
        statsData.gamesPlayed  = statsData.gamesPlayed + 1
        if winner == “player” then
                stats.wins = stats.wins + 1
        else
                stats.losses = stats.losses + 1
        endif
        stats.totalTime = stats.TotalTime + gameDuration
        saveStats()
endprocedure
```

This algorithm updates the user stats in memory and then saves the new values to the database using saveStats(). This ensures all the statistics remain consistent and stored in one central place, reducing the likelihood of data corruption. I chose to use this algorithm design because it’s a logical approach to updating the data.

Testing Plan

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Data is updated correctly	Winner: “player” or “AI” gameDuration: a valid integer	Data is calculated and saved with saveStats()	Ensures the robustness of the algorithm

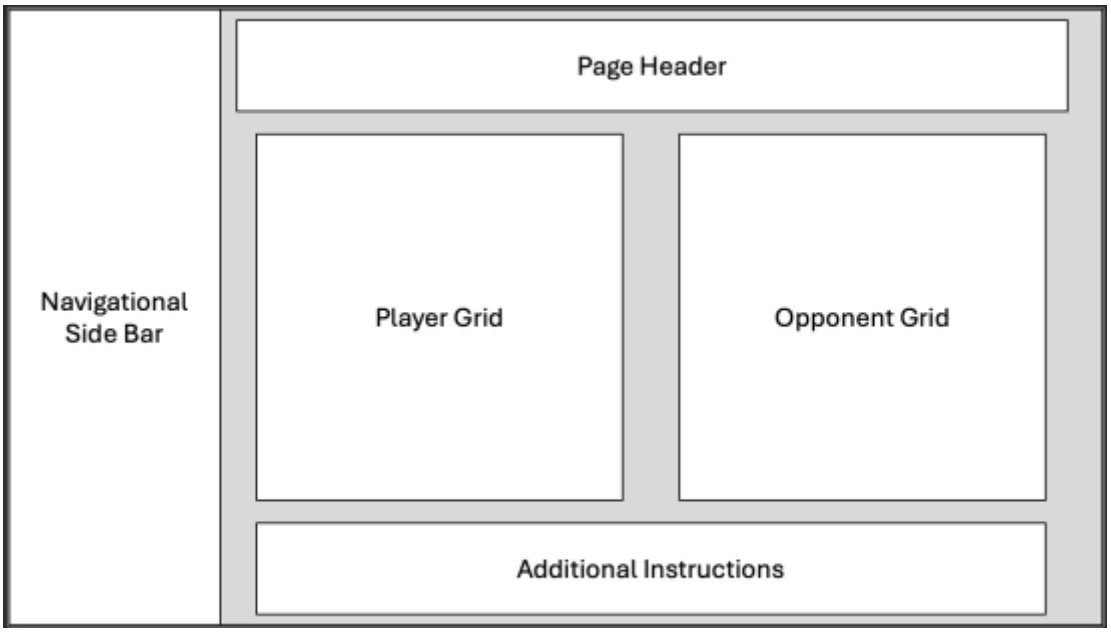
The tests in the above testing plan will be performed both during development and after development, as they all aim to identify problems with the functionality, robustness, and usability of the algorithm, and all need to pass to ensure the algorithm meets the success criteria. As well as the above tests, this last test will be performed after development to ensure the algorithm works correctly consistently.

Test Number	What will be tested	Input	Expected Outcome	Justification
1	Algorithm functions correctly when used as part of the entire solution	Winner: “player” or “AI” gameDuration: a valid integer	Data is consistently calculated and saved with saveStats()	Ensures the algorithm meets the success criteria of stable performance

Important information about algorithms

While all core algorithms, which together would create a working solution, have been shown above, I am conscious that some other coded elements may be brought to light during development which will then be implemented. These elements cannot be effectively planned for as during the development stage, I will gain more context of how the planned components are implemented, and what smaller coded elements need to be implemented to improve the solution in ways such as accessibility or performance. However, all finalised algorithms will be shown during and after development.

GUI Design



Explanation and Usability Features

The above diagram is the planned layout for the main game page. This layout will be the aim of the final design; however certain element positions may prove to be better suited to different locations if it improves visibility or accessibility. The simplicity of the layout needs to be prioritised in order to ensure it is suitable for all the stakeholders. Other pages of the solution will be based on the core layout shown above, but adapted to suit the content of the page.

I hoped to incorporate a vertical grid layout, with each grid expanding/enlarging when clicked. However, this is likely too complex for the timeframe, so instead a horizontal grid layout was chosen for the GUI design. The more complex grid layout may be implemented if time permits.

The navigational side bar will contain links to different pages, such as the main game page and the game statistics page. I chose to use a navigational sidebar because this side bar improves the visibility of other page links, making the entire solution easier to navigate, helping to meet the success criteria of an easily navigable UI.

The additional instructions section will provide info or hints on how to use the game effectively, such as keyboard shortcuts, etc. I may make this section collapsible if it proves to not negatively impact the aspect ratio of other UI elements. I chose to add the instructions section because this helps to inform users on what to do, improving usability.

The GUI will use a simplistic layout and colour scheme, consisting of greys, whites, etc. This gives it a more modern and clean look – making it easier to navigate (therefore meeting the success criteria of a simple layout and easily navigable UI), while also making it more accessible to colour blind users. While the exact font faces have not yet been decided, they will be sans-serif font faces, as they look far more modern and simplistic than serif font faces. A small variety of different fonts will be used for different HTML elements, such as the header and smaller text elsewhere in the UI.

Acceptance Testing Plan

Below is a table of some requirements that will be tested during and after development to ensure the solution functions as correctly without any logic or runtime errors, and is capable of handling misuse. If the expected outcome is not met, changes will need to be made to the solution in order to meet the expected outcomes.

Requirement	Input	Type	Expected Outcome	Priority
The player cannot place a ship that overlaps another ship	Invalid ship placement (overlapping)	Invalid	Ship does not get placed, and the user is informed why	High
The player can place all their ships (in valid placements) on the grid	User selects the ship and the starting cell, and chooses the orientation of the ship	Normal	The ships appear correctly in the desired coordinates without any invalid placement	High
The player cannot place ships outside of the grid boundaries	Invalid ship placement (out of bounds)	Invalid	Ship does not get placed, and the user is informed why	High
The player can choose a coordinate to attack	Selecting a grid cell	Normal	Cell status changes to hit or miss, and feedback is provided visually	High
The AI opponent can take a valid shot	Player turn ends	Normal	AI makes an attack on a coordinate that has not yet been attacked	High
Hits and misses get tracked correctly	Player or AI attacks a grid cell	Normal	Visual representation of hits and misses, and does not discard any statuses	High
The game detects when all the ships of the player or AI are sunk	Player or AI attacks and sinks the final ship of the opponent	Normal	Game ends and end-of-game results are shown to the user.	High
Statistics update after game ends	Game ends	Normal	Winner and other statistics are stored in the DB	Moderate
Navigation menu functions correctly	User clicks a navigation option	Normal	User is redirected to the correct page	Moderate
Player cannot attack invalid grid cells	User clicks a grid cell that has already been attacked	Invalid	Grid cell is not attacked again, and instead visual feedback is provided to the user. Game does not error.	High
Player cannot take two turns consecutively	User clicks a grid cell twice in quick succession	Invalid	Only the first click is used for the turn. The following click(s) are discarded	Moderate-High

Game does not run forever	Player closes the game tab	-	Game times out after a specific period of time with no user requests	Low-Moderate

Beyond just the tests in the above table, the solution will be continually tested for syntax errors as these are the most visible and most important. Internal data such as variables will be outputted to the console/frontend temporarily for the purpose of testing the solution after each iteration of development, as this can reveal important issues with data handling.

Development

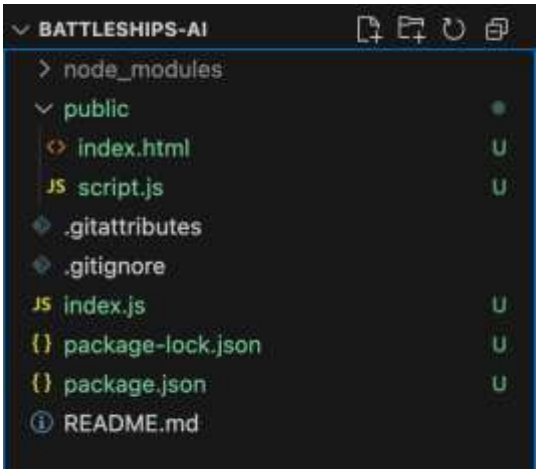
Development Plan

The most suitable way to approach the problem of development is by deciding on what parts need to be completed first. From the project breakdown in the analysis, I identified that the game server is a large section that needs to be created. As the backend node.js webserver is the main core part of the project (with the client JS relying on it), it will be best to approach that first, creating and testing the endpoints with raw data, before then moving onto creating the client-side part of the solution. This backend server will work towards success criteria 2 with focus on success criteria 11 and 15. This section of development is one of the most important parts of the solution.

Initial Setup

As this project will use a Node.js webserver and the client interacts with HTML and JavaScript locally, I need to set up the project directory with relevant starting files.

I have created the following files in the solution's root directory:

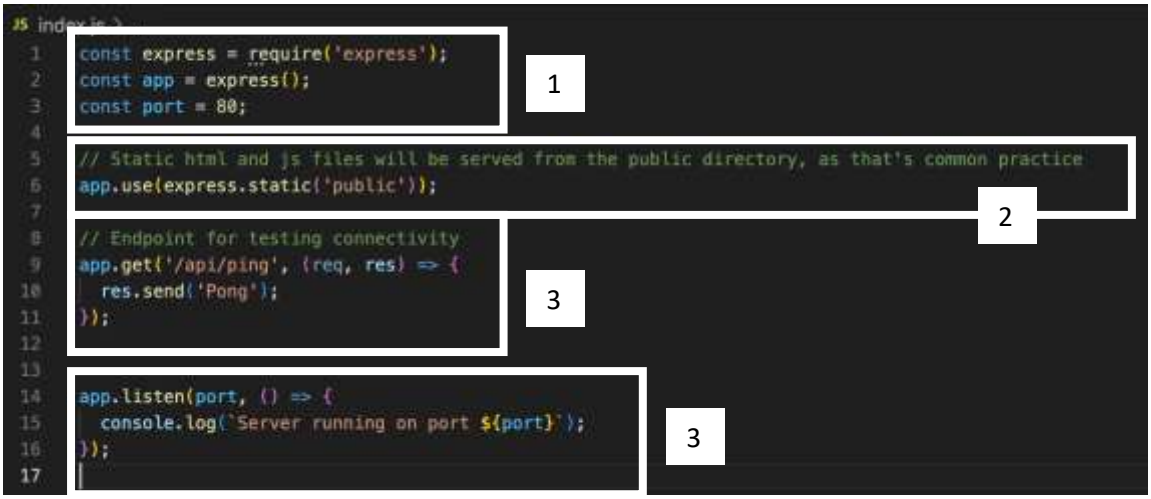


The package.json and package-lock.json files are required by Node.js to provide data about the program. The node_modules directory contains modules required by the project. Currently this just includes requirements for Express, as that is my preferred Node.js webserver module.

The README.md, .gitignore, and .gitattributes are files for a Git repository, which are not relevant to the development of this project, but rather for keeping track of changes in case I need to revert changes or view older versions of the project.

Index.js will be the main file controlling the Express webserver. The client will be served files from the public directory, such as index.html and script.js, both of which are currently empty at this stage in development.

The index.js file currently contains the following content:



The code above has been broken down into sections as follows:

1. These three lines are important. The first line imports the express module, as this will be required for the webserver functionality. The second line initialises it. The third line defines what port the program will run on.
2. The app.use() statement means we can serve static files to the user. In this case, the public directory is used, so navigating to "/" will serve the file at public/index.html.
3. This is a testing endpoint that allows me to easily identify if the webserver is configured correctly and responding to requests.
4. The app.listen() part is the most important, as this makes the express app start listening for requests on the port we chose - 80.

The reason I chose port 80 for this program is because port 80 is the default port for HTTP. This means that no port has to be specified when connecting to the program, as it defaults to port 80. This solution is unlikely to use HTTPS, as it's not necessary for functionality, and would require extra setup. If the user self hosts the backend, it definitely won't be necessary. If I decide to host the backend externally, then I would likely choose a different port and use a reverse proxy for it with SSL. This however is not very important, especially not at such an early stage of development.

Initial work on classes: Attributes and Methods

The first class to work on should be the Game class, as this is the most core part of the game.




```
export class Game {
  constructor(difficulty = "easy") {
    this.difficulty = difficulty;
    this.playerBoard = new Board(); // Player's board
    this.aiBoard = new Board(); // AI's board
    this.isPlayerTurn = true;
    this.gameOver = false;
    this.winner = null;

    this.placeAIShips();
  }

  // Randomly place AI ships
  placeAIShips() {
    const shipsToPlace = generateShips(); // Creates a list of Ship objects
    shipsToPlace.forEach(ship => {
      let valid = false;
      while (!valid) {
        const orientation = Math.random() < 0.5 ? 'H' : 'V'; // Choose a random orientation
        const row = Math.floor(Math.random() * this.gridSize);
        const col = Math.floor(Math.random() * this.gridSize);
        valid = validateShipPlacement(ship.size, row, col, orientation, this.aiBoard.grid);
        if (valid) { // Ship placement is valid (not out of bounds and not overlapping)
          this.placeShipOnGrid(ship.size, row, col, orientation, this.aiBoard.grid); // Place each ship on the AI's board
          this.aiBoard.ships.push({ ...ship, row, col, orientation, hits: 0});
          break;
        }
      }
    });
  }

  // Function for placing ships on the grid
  placeShipOnGrid(size, startRow, startCol, orientation, grid) {
    for (let i = 0; i < size; i++) { // For each grid cell the ship occupies
      if (orientation === "H") grid[startRow][startCol + i] = 1; // Set to 1 to indicate there is a ship
      else grid[startRow + i][startCol] = 1;
    }
  }

  // Function for player placing a ship
  placePlayerShip(ship, startRow, startCol, orientation) {
    const valid = validateShipPlacement(ship.size, startRow, startCol, orientation, this.playerBoard.grid); // Validate the ship placement
    if (!valid) return{ success: false, message: "Invalid ship placement"};   // Return both a boolean flag and a message

    this.placeShipOnGrid(ship.size, startRow, startCol, orientation, this.playerBoard.grid);
    this.playerBoard.ships.push({ ...ship, row: startRow, col: startCol, orientation, hits: 0});
    return{ success: true, message: ""};   // A message isnt really relevant if its successful
  }
}
```

The above code is the first bit of work on the Game class.

The Game class contains the following attributes:

- difficulty
- playerBoard
- aiBoard
- isPlayerTurn
- gameOver
- winner

The attributes currently are not encapsulated, which in production should be changed. For now though I chose to not use encapsulation because direct modification of the attributes by other parts of the codebase is fine as it is in a development stage and the codebase is quite small.

The Game class can be split into these core sections:

1. The constructor for the Game class accepts a difficulty parameter to determine the game difficulty. The default difficulty is “easy” if no difficulty is provided. The attributes are then set and the placeAIShips method is run.

2. The placeAIShips method first gets a list of ships from the generateShips function (shown further on in the documentation). For each ship, it chooses a random co-ordinate and orientation and checks if the placement is valid. It loops until it finds a valid placement. Once found, it places the ship on the grid with the placeShipOnGrid method and adds the data to the aiBoards ships attribute.

3. The placeShipOnGrid method takes ship data and the grid to place the ship on as parameters. It then iterates through every cell the ship should occupy and sets the cell value to 1 to indicate a ship is present.

4. The placePlayerShip method will be used by a player when they place a ship. It takes ship data as parameters. It uses the validateShipPlacement function (shown further on in the documentation) to check if the placement is valid. If it’s not valid, it returns an error message, otherwise it places the ship on the grid, adds the ship to the player’s grid’s ships attribute, and returns a success message.

The playerBoard and aiBoard attributes are Board objects.

The Board class currently contains just the following code:

```
export class Board {
  constructor() {
    this.grid = this.createEmptyGrid();
    this.ships = [];
  }

  // Create a 10x10 grid for the game board
  createEmptyGrid() {
    return Array.from({ length: 10 }, () => Array(10).fill(0)); // No point setting a height attribute because grid is square
  }
}
```

Each Board object contains a grid, and an array which will contain the ships that are on the board. The grid is initialised using the createEmptyGrid function, which makes a simple 10x10 grid using a 2d array.

The Game class contains methods for basic game functionality, such as placing the AI's ships and placing a player ship. To ensure the robustness of the program, all ship placements are validated using the validateShipPlacement() method.

```
export function validateShipPlacement(size, startRow, startCol, orientation, grid) {
  const gridSize = grid.length;

  if (orientation === "H" && startCol + size > gridSize) return false;
  if (orientation === "V" && startRow + size > gridSize) return false;

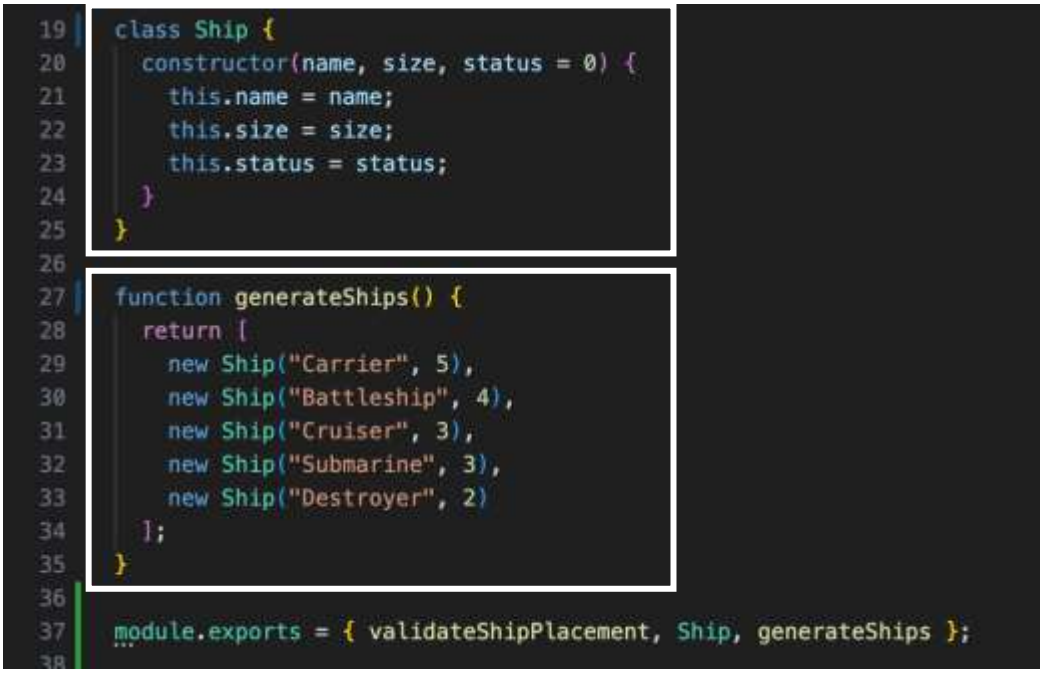
  // At this point, the ship is within the bounds of the grid but may overlap other ships still

  for (let i = 0; i < size; i++) { // Check each grid square to see if it's already occupied by a ship
    if (orientation === "H" && grid[startRow][startCol + i] !== 0) return false;
    if (orientation === "V" && grid[startRow + i][startCol] !== 0) return false;
  }
  // Ship placement valid
  return true;
}
```

1
2
3

1. The method is defined, allowing ship data and the grid object to be passed in as parameters. It finds the grid size from the grid.
2. It will then check if the ship would go beyond the bounds of the grid by checking if the last cell the ship would occupy is a higher index than the grid size.
3. If it passes that check, it will then check each grid square it would occupy to determine if it would overlap an existing ship by checking that each square's status 0, which indicates there is nothing there. If it doesn't overlap, it returns true, indicating that the ship placement is valid.

Each ship is an object that contains the attributes name, size, and status. This is close to the original plan in the class diagrams.



```
class Ship {
  constructor(name, size, status = 0) {
    this.name = name;
    this.size = size;
    this.status = status;
  }
}

function generateShips() {
  return [
    new Ship("Carrier", 5),
    new Ship("Battleship", 4),
    new Ship("Cruiser", 3),
    new Ship("Submarine", 3),
    new Ship("Destroyer", 2)
  ];
}

module.exports = { validateShipPlacement, Ship, generateShips };
```

The generateShips() function will return a list of all 5 ships by creating new Ship instances. As the status isn't passed in, it uses the default status of 0, which is what it should be, as 0 indicates it hasn't been hit.

This function is called from placeAIShips(), and will be called elsewhere when the player interaction logic is introduced.

The module.exports line at the bottom allows the functions to be used in other files in the solution.

Progress Review

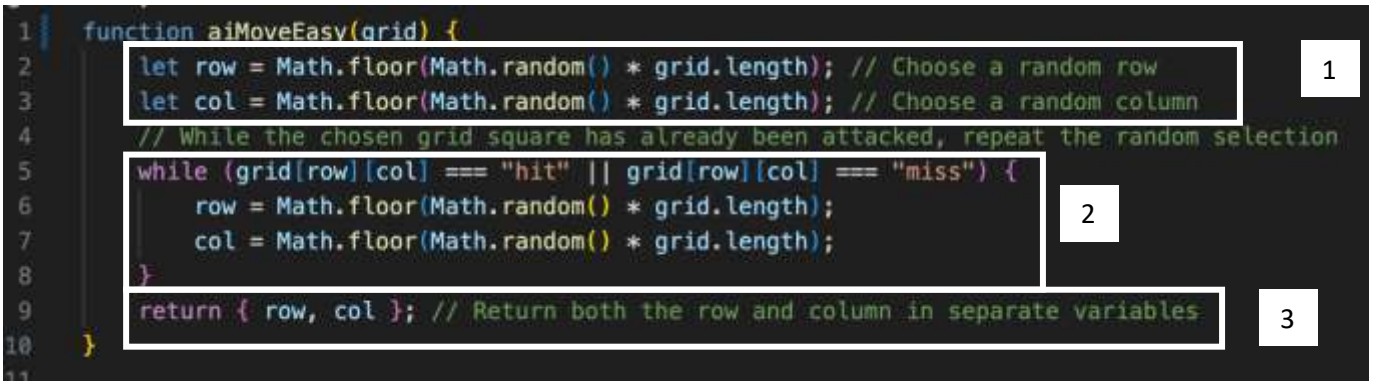
At this stage in development, most of the classes have been implemented and many of the core algorithms are present. The code so far has been created in a modular way so that I can easily implement additional parts later on in development.

The solution however is **not ready for testing yet,** as some of the algorithms rely on others, and without all the core algorithms implemented, it would be challenging to test, leading to wasted time.

The next task to do is to implement the remaining core algorithms, focusing on testing each part to ensure success criteria 11.

Simple AI attacks

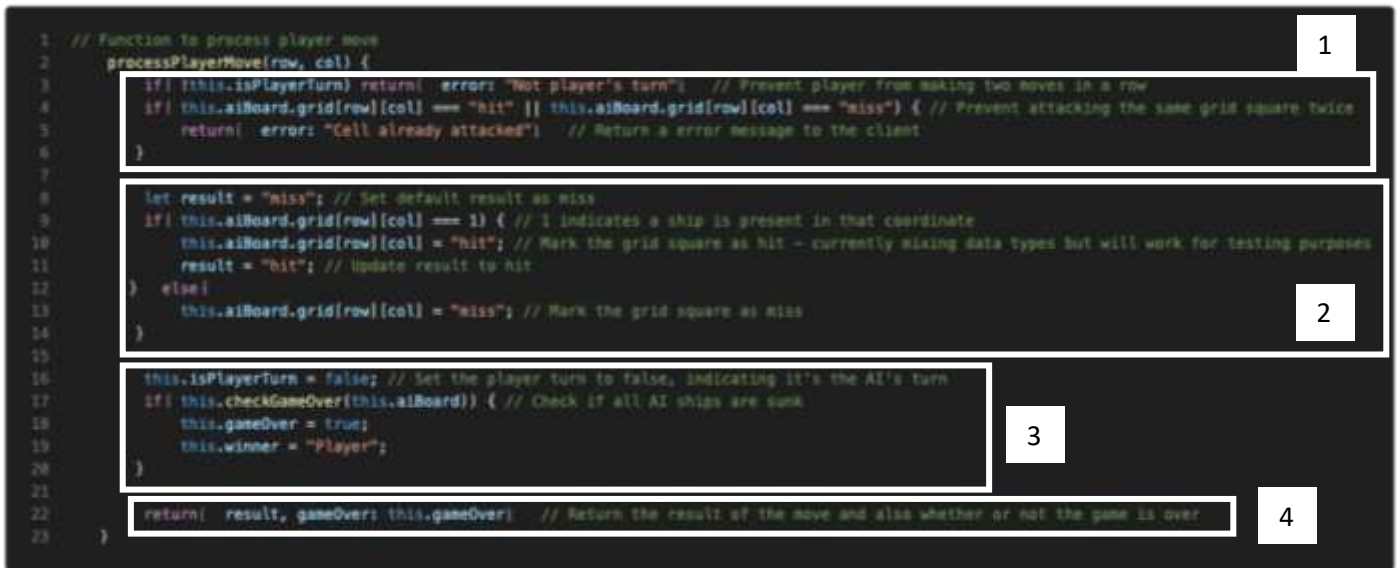
Before implementing a complex targeting algorithm, a simple algorithm is first needed to ensure that the core parts of the game function correctly. The project breakdown contains both a random targeting strategy, so that's what I will start with now. This part of the development works towards success criteria 5 by creating an easy opponent for inexperienced players.



The function aiMoveEasy has 3 parts.

1. A random row and column is chosen by using Math.random() multiplied by the length of the grid. This gives a random float, so Math.floor() is used to floor the float to get an integer, as a float would cause an error. Math.floor() must be used rather than Math.round() because otherwise it could lead to logic errors if the grid square is rounded up to an invalid grid square.
2. The coordinates on the grid are checked to see if they have already been attacked, as their status would either be "hit" or "miss", indicating that they have been attacked. If this condition is true, i.e. it's been attacked already, it will generate a new row and column to avoid attacking a previously attacked grid square.
3. Once the chosen coordinate hasn't already been attacked, the while loop breaks and the function returns both the row and column so a different function can then manage what should happen next.

More core methods




```
// Function to process player move
    processPlayerMove(row, col) {
        if (this.isPlayerTurn) return { error: "Not player's turn" }; // Prevent player from making two moves in a row
        if (this.aiBoard.grid[row][col] === "hit" || this.aiBoard.grid[row][col] === "miss") { // Prevent attacking the same grid square twice
            return { error: "Cell already attacked" }; // Return a error message to the client
        }

        let result = "miss"; // Set default result as miss
        if (this.aiBoard.grid[row][col] === 1) { // 1 indicates a ship is present in that coordinate
            this.aiBoard.grid[row][col] = "hit"; // Mark the grid square as hit - currently mixing data types but will work for testing purposes
            result = "hit"; // Update result to hit
        } else {
            this.aiBoard.grid[row][col] = "miss"; // Mark the grid square as miss
        }

        this.isPlayerTurn = false; // Set the player turn to false, indicating it's the AI's turn
        if (this.checkGameOver(this.aiBoard)) { // Check if all AI ships are sunk
            this.gameOver = true;
            this.winner = "Player";
        }

        return { result, gameOver: this.gameOver }; // Return the result of the move and also whether or not the game is over
    }
```

The processPlayerMove function will be called when a player attempts to make a move.

1. This first part is just validation to make sure the move is valid. First it checks if it's the player's turn. If its not, we return an error to prevent the player making 2 turns in a row.
   Then it checks if the grid cell has already been attacked. This check will also happen on the client to reduce unnecessary network requests, but it should also happen server-side to prevent misuse.

2. Next the method will check the grid square. If its status is 1 then that means there is currently a ship in that grid square, so the method sets its status as "hit" and sets the result flag as "hit" too. Otherwise, the status is set to "miss".

3. The method then checks if the game should end by calling the method checkGameOver. If this method returns true, then the gameOver attribute is set to true and the winner is set.

4. The method finally returns the result of the move, and the gameOver status.

```
112     // Check if all ships are sunk
113     checkGameOver(board) {
114         for (let row = 0; row < this.gridSize; row++) {
115             for (let col = 0; col < this.gridSize; col++) {
116                 if (board[row][col] === 1){
117                     return false; // 1 indicates theres a ship that hasnt been hit yet
118                 }
119             }
120         }
121         return true;
122     }
```

The checkGameOver method is relatively simple. It uses nested iteration, which I had to research on StackOverflow in order to implement. For each row in the grid, it iterates over each column for that row. This essentially allows us to iterate through each coordinate. If the status of a grid cell is 1, then the method returns false, ending the iteration there. If the iteration finishes without returning, then true is returned, indicating that the game should now end.

```
1   // Function to process AI moves
2       aiMove() {
3           if( this.gameOver) return{  result: "Game already over"}   // Make sure AI doesn't move if game is over
4           const move = aiMoveEasy(this.playerBoard); // Returns a row and column
5
6           const{  row, col}  = move;
7           let result = "miss";
8           if( this.playerBoard[row][col] === 1) { // Indicates a ship is present
9               this.playerBoard[row][col] = "hit"; // Mark the grid square as hit
10              result = "hit";
11          }  else{
12              this.playerBoard[row][col] = "miss";
13          }
14
15          this.isPlayerTurn = true;
16          if( this.checkGameOver(this.playerBoard)) { // Check if all player ships are sunk
17              this.gameOver = true;
18              this.winner = "AI";
19          }
20
21          return{  row, col, result, gameOver: this.gameOver}   // Return the AI's move to the client
22      }
```

1
2
3
4

The aiMove method is similar to the processPlayerMove method, but is a bit simpler.

1. The method first checks if the game is already over to ensure the game doesn't continue by letting the AI make moves.
   The aiMoveEasy method is then called to choose a random grid square. This method already has validation within it, so we don't need to check again if the grid cell is valid.
2. The row and col values are retrieved from the function. The coordinate is then checked to see if a ship exists. If the status is 1, indicating there is an unhit ship, it will set it's status to hit, otherwise it sets it to miss.
3. The isPlayerTurn attribute is then set to true to allow processPlayerMove() to run. The method then checks if the game should end with checkGameOver().
4. Finally, the method returns the row, column, result of the move, and gameOver status up the call chain.

Progress Review

At this stage of development, all core algorithms are now implemented. While I could implement temporary functions for testing these algorithms, it makes more sense to work on endpoints instead, as this means I can test more easily, and means that I save time as the endpoints will be used in the final solution.

Some Considerations

Currently, the grid cell status mixes integers and strings which is bad practice. I will likely change this at a later stage to use just integers, such as 1 for ship present, 2 for miss, 3 for hit, or just strings. Mixing data types however works fine for testing purposes, as JavaScript is lenient with data types.

Furthermore, the methods I have implemented so far do not contain much validation. While some validation may also occur on the client-side, it is still important that I implement more validation on the server-side to prevent misuse. This however is not important in the initial development, but I may implement more validation later on, closer to completion, as it would be necessary in a production environment.

Creating some core endpoints

As defined in the project breakdown, the client will communicate with the server through API endpoints. Below is a screenshot of a .js file I created in the routes directory of the root project, broken down into each part.

routes > JS game.js > ...

```
1  const express = require('express');
2
3  const {Game} = require('../logic/game.js');
4  const app = express.Router();
5  let games = {}; // Store active games in memory
6
```

This first part of the program imports the modules and initialises the express router. I chose to use express because I am more experienced with express than other similar libraries.
Express is imported, and later initialised with express.Router(). It also imports the Game class from logic/game.js.

An empty array is created which will be used to store active games.

This Express system will work towards success criteria 2 by allowing clients to communicate with the backend server.

Start Game Endpoint:



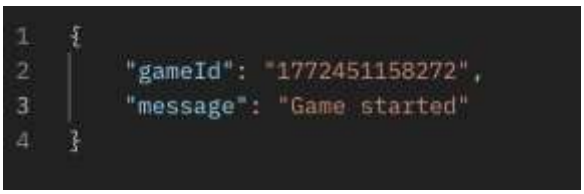
This endpoint will be used for initialising a new game.

1. The difficulty of the game is set as either a difficulty string passed by the client in the request body, or falling back to easy if none is specified. This fallback prevents logic errors if the value is undefined. It will then use the validateDifficultyString() function to validate whether or not the difficulty is valid. This prevents the user from sending invalid data, which would otherwise cause logic errors. It will instead send a graceful error message if an invalid difficulty level is sent in the request body.
2. A game id is created from the current timestamp string. This would not be suitable in production incase 2 games were started at the exact same time, causing a clash. However, for testing purposes for a small scale system, it would work fine as two games getting assigned the same gameId is incredibly unlikely.
3. A new Game instance is initialised, passing in the difficulty for the difficulty attribute. This Game instance is added to the games array with the key being the gameID.

Start-game Endpoint Testing

Test Number	What is being tested?	Request	Input	Type	Expected Outcome	Actual Outcome	Passes Test
SG-1	Correct request	POST request to /start-game	difficulty: “easy”	Valid	Returns 200 Message: “Game started”	Returns 200 Message: “Game started”	Yes
SG-2	Correct request	POST request to /start-game	No difficulty passed	Valid	Returns 200 Message: “Game started”	Returns 200 Message: “Game started”	Yes
SG-3	Passing an invalid difficulty	POST request to /start-game	difficulty: “test”	Invalid	Returns 400 Message: “Invalid difficulty level”	Returns 200 Message: “Game started”	No
SG-4	Passing invalid json syntax	POST request to /start-game	difficulty: easy	Invalid	Returns 400 Message: “Invalid difficulty level”	Returns 400 Throws syntax error: Unexpected token 'e', ..."ficulty": easy } " is not valid JSON	No
SG-5	Request using wrong method	GET request to /start-game	difficulty: “easy”	--	Refuses request due to incorrect method	Returns 404	Yes

Tests SG-1, SG-2, SG-3:



Test SG-4:



Test SG-5:



The tests reveal that the endpoint partially works, but isn't very robust to misuse. For example, passing invalid syntax to the endpoint throws a syntax error.

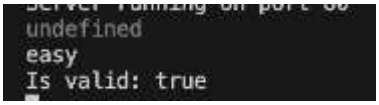
Furthermore, passing an invalid difficulty value returns 200. This suggests that the passed value isn't actually being used, or the validateDifficultyString() function isn't working properly. This will require testing in order to fix.

To diagnose the issues of the endpoint returning unexpected results, I will add lines to output some relevant variables to the console.



This should help to identify where the issue originates from.

When making a request to the endpoint, the console logs reveal that req.body.difficulty is undefined, and it then proceeds to use the fallback of 'easy'.



After diagnosing the issue, it turns out I was passing difficulty as a parameter rather than in the request body. This would mean the code would need to be adapted to req.params.difficulty. After putting the difficulty value in the request body, sending a request with "difficulty: "test" returns a 400 error, as expected. I chose to do this instead of modifying the code to read from the parameters because the logic was already implemented to read it from the request body. It would be unnecessary to modify the code to accept a parameter instead, since the request body works fine.

The syntax error when passing invalid JSON syntax is caused by the Express router using the "app.use(express.json());" middleware. The error however does not cause the program to terminate, so this issue can be ignored for now. I could instead focus on fixing this error, but it's not high priority as it doesn't crash the program and doesn't happen when the endpoints are used correctly. I may address this issue later on if time permits.

To reduce the likelihood of other issues or errors causing the program to exit, I added a try catch statement to the /start-game endpoint. I will also implement this for all other endpoints that could potentially error for unexpected reasons.

```
app.post('/start-game', (req, res) => {
    try {
        console.log(req.body.difficulty)
        const difficulty = req.body.difficulty || 'easy'; // Use easy as a default difficulty incase none is specified
        console.log(difficulty)
        console.log(`Is valid: ${validateDifficultyString(difficulty)}`)
        if (!validateDifficultyString(difficulty)) {
            return res.status(400).json({ error: "Invalid difficulty level" });
        }
        const gameId = Date.now().toString(); // Unique ID based on current timestamp. Not suitable long-term, but will work for now

        games[gameId] = new Game(difficulty);
        return res.json({ gameId, message: 'Game started' });
    } catch (error) {
        console.error('Error starting game:', error);
        return res.status(500).json({ error: 'Internal server error' });
    }
});
```

This try catch statement means that if any of the code within the try{} clause throws an error, the catch{} clause will run, which will log the error to the console, and return error 500 to the client.

Progress Review

At this stage, there is a working prototype of the solution which simply starts a game. Currently there is no way to interact with the game, such as placing ships or attacking. I chose to work on this start-game logic first as this is the entry point to the game for a player. This means it makes more sense to start with this than starting with another part of the solution such as the opponent attacks.

Testing after making changes:

Test Number	What is being tested?	Request	Input	Type	Expected Outcome	Actual Outcome	Passes Test
SG-1	Correct request	POST request to /start-game	difficulty: “easy”	Valid	Returns 200 Message: “Game started”	Returns 200 Message: “Game started”	Yes
SG-2	Correct request	POST request to /start-game	No difficulty passed	Valid	Returns 200 Message: “Game started”	Returns 200 Message: “Game started”	Yes
SG-3	Passing an invalid difficulty	POST request to /start-game	difficulty: “test”	Invalid	Returns 400 Message: “Invalid difficulty level”	Returns 200 Message: “Invalid difficulty level”	Yes
SG-4	Passing invalid json syntax	POST request to /start-game	difficulty: easy	Invalid	Returns 400 Message: “Invalid	Returns 400 Throws syntax error:	No

					difficulty level”	Unexpected token 'e', ..."ficulty": easy } " is not valid JSON	
SG-5	Request using wrong method	GET request to /start-game	difficulty: “easy”	--	Refuses request due to incorrect method	Returns 404	Yes

Testing video name: “start-game testing after changes.mov”

0:00 – SG-1

0:04 – SG-2

0:10 – SG-3

0:17 – SG-4

0:27 – SG-5

All the tests now pass except for test SG-4, but this is expected. The current prototype of the solution is working and (seems to be) bug-free, allowing a game to be created.

Place Ship Endpoint

Currently, the only game related endpoint implemented so far is start-game. The program runs correctly without any errors, but has very limited functionality currently.

By using algorithmic thinking and problem decomposition, I believe the ship placement endpoint may be a suitable to implement next. This endpoint will need to be developed and tested extensively in order to meet success criteria 15.



This endpoint will be used for the player to place ships on the board. I have broken it down into parts:

1. The algorithm first initialises some constants from the request body. This can be done in one line to make the code more readable and maintainable.
2. The provided gameId is then looked up in the games list. If there is no game matching that gameId, then a 404 response code is returned, with the error “Game not found”. This will be handled by the frontend.
3. The placeShip method of the game instance is then run. This will actually place the ship on the board for the player.
4. If the result returned by the placeShip method doesn’t have the success value set to true, then it will return a 400 error to indicate that something in the request is wrong.
5. The response from the placeShip method (formatted as: {“success”: bool, “message”: string}) will be returned as a response to the endpoint request. This means it will always return 200, so additional checks need to be in place on the client side to check the success value. It will then also log to the console the data stored in each board. This will be removed at some later stage in development, but will be used for now to help with debugging by providing more information about different data in the game.

The place-ship endpoint will need to be extensively tested to ensure it functions as expected. As seen in the project breakdown, the game server should validate actions, so this needs to be tested.

Place-Ship Endpoint Testing

Test Number	What's being tested	Request	Input	Type	Expected Outcome	Actual Outcome	Passes Test
PS-1	Missing required values	POST request to /place-ship	gameId: a valid ID returned by /start-game	Invalid	Returns 400	Returns 500 *TypeError: Cannot read properties of undefined (reading 'toLowerCase')*	No

PS-2	Correct request	POST request to /place-ship	gameId: a valid ID returned by /start-game
			shipType: "Battleship"
			startRow: 0
			startCol: 0
			orientation: "H"
				Valid	Returns 200
					Success: true
					Message: ""
						Returns 200
						
						{
						"success": true,
						"message": ""
						}
							Yes

PS-3	Passing a co-ordinate out of range	POST request to /place-ship	gameId: a valid ID returned by /start-game shipType: "Battleship" startRow: 100 startCol: 0 orientation: "H"	Invalid	Returns 400	Returns 500 TypeError: Cannot read properties of undefined (reading '0')	No

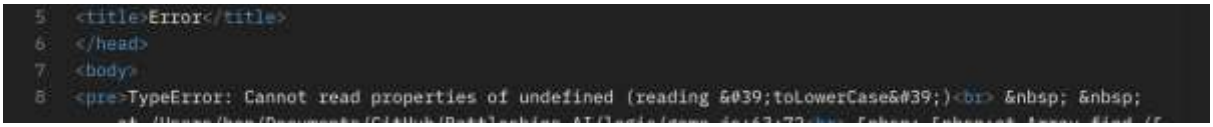
PS-4	Passing an invalid orientation	POST request to /place-ship	gameId: a valid ID returned by /start-game
			shipType: "Battleship"
			startRow: 0
			startCol: 0
			orientation: "test"
				Invalid	Returns 400	Returns 200
						
						{
						"success": true,
						"message": ""
						}
							No

PS-5	Choosing co-ordinates that are already occupied	POST request to /place-ship	gameId: a valid ID returned by /start-game
			shipType: "Battleship"
			startRow: a co-ordinate already used
			startCol: a co-ordinate already used
			orientation: "H"
				Invalid	Returns 400
					Message: "Invalid ship placement"
						Returns 200
						
						{
						"success": true,
						"message": ""
						}
							No

PS-6	Passing invalid game ID	POST request to	gameId: "12345" shipType: "Battleship"	Invalid	Returns 404 Error: "Game not found"	Returns 404 {	Yes

		/place-ship	shipType: “Battleship” startRow: 0 startCol: 0 orientation: “H”			"error": "Game not found" }	
PS-7	Passing invalid ship type	POST request to / place-ship	gameId: a valid ID returned by /start-game shipType: “test” shipType: “Battleship” startRow: 0 startCol: 0 orientation: “H”	Invalid	Returns 400 Message: “Invalid ship type”	Returns 400 { "message": "Invalid ship type" }	Yes
PS-8	Using wrong method to make the request	GET request to /place-ship	gameId: a valid ID returned by /start-game shipType: “Battleship” shipType: “Battleship” startRow: 0 startCol: 0 orientation: “H”	--	Refuses request due to incorrect method	Returns 404	Yes

PS-1:



PS-2, PS-4, PS-5:



PS-3:



PS-6:

```
{
    "error": "Game not found"
}
```

PS-7:

```
{
    "success": false,
    "message": "Invalid ship type"
}
```

PS-8:

```
<body>
    <pre>Cannot GET /api/game/place-ship</pre>
</body>
```

This stage of iterative testing reveals some problems with the existing validation logic, mostly with validation of inputted data, as invalid data can currently either cause errors or have an unexpected result.

To solve tests 1, 3, and 4, I made the following changes to the placeShip algorithm:

```
// Function for player placing a ship
placeShip(shipType, startRow, startCol, orientation) {
    const ship = ships.find(s => s.name.toLowerCase() === shipType?.toLowerCase()); // 1
    if (!ship) return { success: false, message: "Invalid ship type" };

    if(orientation !== "H" && orientation !== "V") {
        return { success: false, message: "Invalid orientation" };
    }
    if(startRow < 0 || startRow >= this.gridSize || startCol < 0 || startCol >= this.gridSize) {
        return { success: false, message: "Invalid co-ordinate" };
    }

    const valid = validateShipPlacement(ship.size, startRow, startCol, orientation, this.playerBoard.grid); // Validate the ship
    if (!valid) return { success: false, message: "Invalid ship placement" }; // Return both a boolean flag and a message

    this.placeShipOnGrid(ship.size, startRow, startCol, orientation, this.playerBoard.grid);
    this.playerBoard.ships.push({ ...ship, row: startRow, col: startCol, orientation, hits: 0 });
    return { success: true, message: "" }; // A message isnt really relevant if its successful
}
```

1. Test 1 is fixed by replacing “shipType.toLowerCase()” with “shipType?.toLowerCase()”. This is because adding the question mark makes it conditional, so if shipType is undefined, it won’t throw an error but instead just be null, leading to the condition on the following line entering and returning the “Invalid ship type” error. I chose to fix it this way because otherwise additional validation would be required. Using “?.” is the simplest way of resolving it without unnecessary code.

2. Test 4 is fixed by adding validation to check if orientation is either ‘H’ or ‘V’, or it will return an error. I chose to do this because validation is needed to prevent misuse. While the frontend should always provide an orientation, if the solution was released to the public, some users may try to misuse it, so this needs to be validated.

3. Test 3 is fixed by adding range validation to check if the startRow and startCol are within the range of 0 <= value < gridSize. Essentially, if either value is less than 0 or greater than or equal to the grid size, it will return an error. This should prevent invalid ship placement.

Post change testing

Test Number	What's being tested	Request	Input	Type	Expected Outcome	Actual Outcome	Passes Test

PS-1	Missing required values	POST request to /place-ship	gameId: a valid ID returned by /start-game	Invalid	Returns 400	Returns 400
						{
						"error": "Invalid ship type"
						}
							Yes

PS-3	Passing co-ordinate out of range	POST request to /place-ship	gameId: a valid ID returned by /start-game
			shipType: "Battleship"
			startRow: 100
			startCol: 0
			orientation: "H"
				Invalid	Returns 400	Returns 400
						{
						"error": "Invalid co-ordinate"
						}
							Yes

PS-4	Passing invalid orientation	POST request to /place-ship	gameId: a valid ID returned by /start-game
			shipType: "Battleship"
			startRow: 0
			startCol: 0
			orientation: "test"
				Invalid	Returns 400	Returns 400
						{
						"error": "Invalid orientation"
						}
							Yes

PS-5	Choosing co-ordinates that are already occupied	POST request to /place-ship	gameId: a valid ID returned by /start-game
			shipType: "Battleship"
			startRow: a co-ordinate already used
			startCol: a co-ordinate already used
			orientation: "H"
				Invalid	Returns 400
					Message: "Invalid ship placement"
						Returns 400
						{
						"error": "Invalid ship placement"
						}
							Yes

Testing video name: “Place-ship testing.mov”

0:00 – PS-1

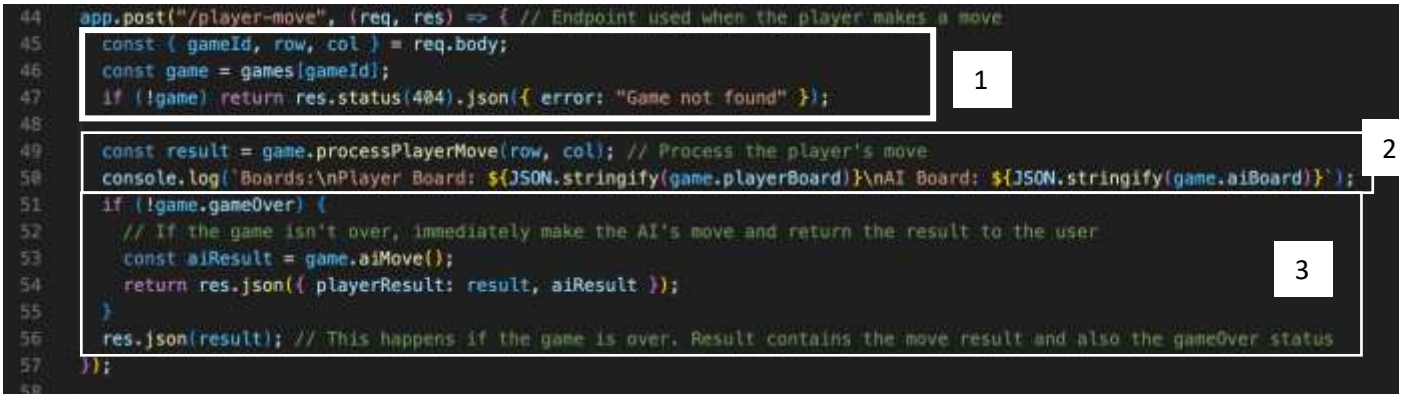
0:04 – PS-3

0:10 – PS-4

After making these changes, the tests pass correctly, showing that the place-ship endpoint is function as of now. It will likely still need further changes to make it more robust, but for the first stage of iterative testing, it should work suitably.

Player Move Endpoint

This endpoint will be used to allow the user to attack the opponent's grid.




```
app.post("/player-move", (req, res) => { // Endpoint used when the player makes a move
  const { gameId, row, col } = req.body;
  const game = games[gameId];
  if (!game) return res.status(404).json({ error: "Game not found" });

  const result = game.processPlayerMove(row, col); // Process the player's move
  console.log(`Boards:\nPlayer Board: ${JSON.stringify(game.playerBoard)}\nAI Board: ${JSON.stringify(game.aiBoard)}`);
  if (!game.gameOver) {
    // If the game isn't over, immediately make the AI's move and return the result to the user
    const aiResult = game.aiMove();
    return res.json({ playerResult: result, aiResult });
  }
  res.json(result); // This happens if the game is over. Result contains the move result and also the gameOver status
});
```

The player-move endpoint has been broken down into core sections based on their logic:

1. The gameId, row, and col value are retrieved from the request body. The gameId is then checked to validate that it's a valid game ID. If it's not, it returns a 404 error.
2. The processPlayerMove method of the game class is called, which handles the logic for processing the move. It will return a result containing the result of the move and the gameOver status. A console.log statement is also present to aid in debugging this endpoint by outputting relevant data.
3. The gameOver attribute of the game is checked to see if the game has ended due to the player's last move. If the game is not over, the aiMove method of the game is called and the results of both the player's move and the AI's move is returned to the user. If the game is over, it returns just the result of the player's move, which already contains the gameOver status.

Player-Move Endpoint Testing

Test Number	What's being tested	Request	Input	Type	Expected Outcome	Actual Outcome	Passes Test
PM-1	Correct Request	POST request to /player-move	gameId: a valid ID row: 0 col: 0	Valid	Returns 200 {playerResult: …., aiResult: …..}	Returns 200 {playerResult: …., aiResult: …..}	Yes

PM-2	Invalid game ID	POST request to /player-move	gameId: "12345"
			row: 0
			col: 0
				Invalid	Returns 404	Returns 404
						{
						"error": "Game not found"
						}
							Yes

PM-3	Invalid data type	POST request to /player-move	gameId: a valid ID row: -1 col: 0	Invalid	Returns 400	Returns 500 TypeError: Cannot read properties of undefined (reading '0')	No

PM-4	Out of range data	POST request to /player-move	gameId: a valid ID row: 15 col: 0	Invalid	Returns 400	Returns 500 TypeError: Cannot read properties of undefined (reading '0')	No
PM-5	Boundary grid range	POST request to /player-move	gameId: a valid ID row: 9 col: 9	Boundary	Returns 200	Returns 200 {playerResult: …., aiResult: …..}	Yes

Testing video name: “player-move testing.mov”

0:00 PM-1

0:03 – PM-2

0:06 PM-3

0:20 – PM-4

0:27 PM-5

The testing results reveal that test number 3 and 4 failed, returning the same type error. This indicates that I’ve made a mistake somewhere in the code which both of those tests reached and failed at.

After diagnosing the code and the inputs throughout the function, I diagnosed the cause of the issue and created a solution, as it was missing important validation of the coordinates.

```
// Function to process player move
processPlayerMove(row, col) {
    if (!this.isPlayerTurn) return { error: "Not player's turn" }; // Prevent player from making two moves in a row
    console.log(`Player attacking row ${row}, col ${col}`);

    // Check if row or column is out of range
    if (row < 0 || row >= this.gridSize || col < 0 || col >= this.gridSize) {
        return { error: "Invalid coordinates" };
    }

    if (this.aiBoard.grid[row][col] === "hit" || this.aiBoard.grid[row][col] === "miss") { // Prevent attacking the sa
        return { error: "Cell already attacked" }; // Return a error message to the client
    }
```

The new section of code checks if the row and column are within a valid range. This prevents the algorithm trying to access a coordinate that doesn’t exist. I made this change because it’s necessary to fix the problem.

After making this change, the 2 tests that previously failed now pass:

Test Number	What’s being tested	Request	Input	Type	Expected Outcome	Actual Outcome	Passes Test
PM-3	Invalid data type	POST request to /player-move	gameId: a valid ID row: -1 col: 0	Invalid	Returns 400	Returns 400 {"error": "Invalid coordinates"}	Yes

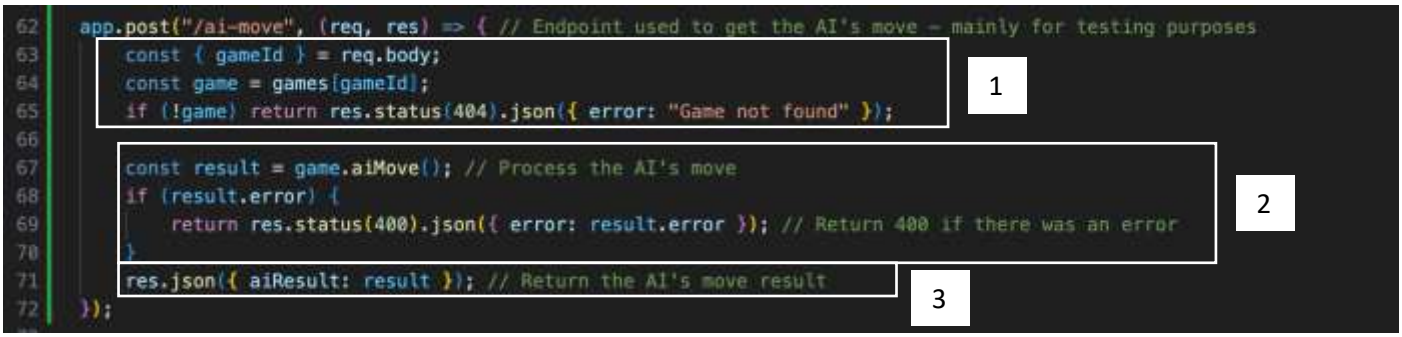
PM-4	Out of range data	POST request to /player-move	gameId: a valid ID row: 15 col: 0	Invalid	Returns 400	Returns 400 {"error": "Invalid coordinates"}	Yes

PM-3 and PM-4 after changes:



AI Move Endpoint

This endpoint will be used primarily for testing and debugging purposes, to ensure the functionality is as expected. It may also be useful later on in the development, although for now it won't be needed for other uses apart from testing.



1. The endpoint first retrieves the game from the request body, and checks it against the games array to decide if it's a valid game ID or not. If it isn't, it returns a 404 error.
2. The algorithm then runs the aiMove method of the game instance. If it returns an error, the algorithm returns a 400 error. This could also instead be a 5xx error, but as this endpoint is only for testing purposes, I chose to use 400 to distinguish it from an unhandled error.
3. The algorithm then returns the result of the AI move

As this endpoint will only be used for testing purposes, it does not require as much input validation as end-users wouldn't be interacting with it.

Because of this, the testing does not need to cover misuse of inputs, as only I will be using this endpoint. This means the testing will be less extensive, as more extensive testing is unnecessary.

AI-Move Endpoint Testing

Test Number	What's being tested	Request	Input	Expected Outcome	Actual Outcome	Passes Test
AIM-1	Correct Request on a game with player ships	POST request to /ai-move	gameId: a valid ID	Returns 200 {aiResult: .....} Game not yet over	Returns 200 {"aiResult": {"row": 3, "col": 8, "result": "miss", "gameOver": false } }	Yes
AIM-2	Correct request on a game with no player ships	POST request to /ai-move	gameId: a valid ID	Returns 200 {aiResult: .....} Game over	Returns 200 {"aiResult": { "row": 0, "col": 6, "result": "miss", "gameOver": true } }	Yes
AIM-3	AI attacks only new	POST request to /ai-move	gameId: a valid ID	Every response	Every response shows a new co-ordinate attacked	Yes

	co-ordinates			shows a new co-ordinate attacked		
AIM-4	AI correctly attacks ships	POST request to /ai-move	gameId: a valid ID	Returns a “hit result” when an occupied co-ordinate is hit	Returns 200 { "aiResult": { …., "result": "hit", …. } }	Yes
AIM-5	Game ends when AI wins	POST request to /ai-move	gameId: a valid ID	Returns gameOver: true when AI wins	Returns 200 {"aiResult": { ….., gameOver": true } }	Yes
AIM-6	AI won’t attack after game has ended	POST request to /ai-move	gameId: a valid ID which has already ended	Returns “Game already over"	Returns 200 {"aiResult": { “result": "Game already over” } }	Yes
AIM-7	AI wont attempt to attack in an invalid game	POST request to /ai-move	gameId: an invalid ID	Returns “Game not found”		Yes

Example responses for AIM-2, AIM-4, AIM-5, AIM-6:



Testing video name: “ai-move endpoint testing.mov”

The video demonstrates that the opponent doesn’t attack the same cell twice, correctly reflects the cell state with hit or miss, and doesn’t attempt to attack for an invalid game ID.

Check game over Endpoint

The check-game-over endpoint may be used later on in development to query whether or not the game has ended. This may or may not be used, depending on if the player-move response is sufficient or not. For now, it can be used for testing purposes, but later may be necessary to meet success criteria 12.

```
74  app.get("/check-game-over", (req, res) => { // Endpoint to check if the game is over
75      const { gameId } = req.query;
76      const game = games[gameId];
77      if (!game) return res.status(404).json({ error: "Game not found" });
78
79      if(game.gameOver) { // If the game is already over, return the winner
80          return res.json({ gameOver: true, winner: game.winner });
81      }
82
83      if (game.checkGameOver(game.aiBoard.grid)) { // Check if all AI ships are sunk
84          this.gameOver = true;
85          this.winner = "Player";
86      } else if (game.checkGameOver(game.playerBoard.grid)) { // Check if all player ships are sunk
87          this.gameOver = true;
88          this.winner = "AI";
89      }
90
91      res.json({ gameOver: game.gameOver, winner: game.winner });
92  });
```

1
2
3

1. The algorithm first checks if the provided game ID is valid. If not, it returns 404.
2. The algorithm then checks if the game is already over. If it is, it will return the existing game attributes.
3. If the game is not already over, then the checkGameOver method will be called for both the player board and the AI board, which will check the grids to see if the game should end due to no more active ships. If the game should end, it will set the attributes and return the new data.

Check-Game-Over Endpoint Testing

Test Number	What's being tested	Request	Input	Expected Outcome	Actual Outcome	Passes Test

CGO-1	Correct Request on a game with remaining player ships	GET request to /check-game-over	gameId: a valid ID	Returns 200 {aiResult: .....} Game not yet over	Returns 200
					{
					"gameOver": false,
					"winner": null
					}
						Yes

CGO-2	Correct request on a game with no remaining	GET request to /check-game-over	gameId: a valid ID	Returns 200 {aiResult: .....} Game over	Returns 200
					{
					"gameOver": true,
					"winner": "AI"
					}
						Yes

	player ships					
CGO-3	Invalid game ID	GET request to /check-game-over	gameId: “12345”	Returns 404	Returns 404 {"error": "Game not found"}	Yes

Testing video name: “check-game-over testing.mov”

0:00 – CGO-1

0:04 – CGO-2

0:08 – CGO-3

As all tests now pass, I can move onto other parts of the backend. A suitable endpoint to implement at this stage of development is the get-game-state endpoint. I did not implement this earlier as without the other endpoints, the get-game-state endpoint would not be useful. This means that now is a suitable time to implement it.

Get Game State Endpoint

The get-game-state endpoint will be used for retrieving the current state of a requested game, in order to determine what should happen on the client frontend. This may alternatively be used to meet success criteria 12 instead of the /check-game-over endpoint, however this will be determined later in development.

```
98  app.get("/get-game-state", (req, res) => { // Endpoint to retrieve the current game state
99      /* This endpoint will return:
100         masked AI grid
101         full player grid
102         correct turn
103         correct gameOver flag
104     */
105     const { gameId } = req.query;
106     const game = games[gameId];
107     if (!game) return res.status(404).json({ error: "Game not found" });
108
109     // Mask the AI grid to hide remaining ships
110     const maskedAIBoard = game.aiBoard.grid.map(row =>
111         row.map(cell => (cell === 1 ? 0 : cell)) // Replaces existing ships with empty spaces (0)
112     );
113
114     res.json({
115         playerBoard: game.playerBoard.grid,
116         aiBoard: maskedAIBoard,
117         isPlayerTurn: game.isPlayerTurn,
118         gameOver: game.gameOver,
119         winner: game.winner
120     });
121 });
```

1
2
3

1. The algorithm first retrieves the gameId from the request query. It then checks if it's a valid running game. If it's not, it returns a 404 error.

2. The algorithm then uses a map function to mask the AI's board. This replaces all existing ships with empty spaces, preventing the player from cheating to keep the game fair.

3. The algorithm then finally returns JSON data containing the player's board, the AI's masked board, the isPlayerTurn flag, the gameOver flag, and the winner of the game (only needed if the game is over)

Get-game-state Endpoint Testing

Test Number	What's being tested	Request	Input	Expected Outcome	Actual Outcome	Passes Test
GGS-1	Correct request with a valid game ID	GET request to /get-game-state	gameId: a valid ID	Returns 200 with JSON data	Returns 200 with correct JSON data	Yes

GGS-2	Incorrect request, as game ID is invalid	GET request to /get-game-state	gameId: an invalid ID	Returns 404 Does not error	Returns 404 Does not error	Yes
GGS-3	Incorrect request, as no game ID provided	GET request to /get-game-state	-	Returns 404 Does not error	Returns 404 Does not error	Yes
GGS-4	Correct request on an already ended game	GET request to /get-game-state	gameId: a valid ID	Returns 200 with JSON data (containing the correct winner)	Returns 200 with correct JSON data (contains the correct winner)	Yes

Testing video name: “get-game-state testing.mov”

0:00 GGS-1

0:10 – GGS-2

0:20 – GGS-3

0:27 – GGS-4

Progress Review

At this stage of development, all main endpoints have been implemented and thoroughly tested to ensure they are suitable and robust with validation to prevent misuse. This is a big checkpoint as the core logic behind the solution is mostly complete.

The final part of the original plan which still needs to be implemented into the backend is the game statistics. This should be completed next, before then moving onto the frontend development as this is a logical order to complete the tasks.

Implementing game statistics

A game statistics system should be implemented as it will work towards success criteria 4 of a statistics page. While it's marked as optional, it would be beneficial to the user to implement this. To implement a game statistics system, the requirements need to be identified. From the analysis and design of this project, I decided on the statistics to include. One of these statistics will be the average game duration. In order to implement this, I need to add a new attribute to the constructor for the game class.

```
17  class Game {
18      constructor(difficulty = "easy") {
19          this.gridSize = 10;
20          this.difficulty = difficulty;
21          this.playerBoard = new Board(); // Player's board
22          this.aiBoard = new Board(); // AI's board
23          this.isPlayerTurn = true;
24          this.gameOver = false;
25          this.winner = null;
26          this.startedAt = new Date();
27
28          this.placeAIShips();
29      }
```

To implement the functionality for game statistics, I will need some additional helper functions: getStats() and updateStats().

In order to add persistence to the data, I will need to implement some functionality to write to secondary storage in some way. The initial plan was to implement an SQL database, however this may not be possible due to time constraints. For now, a JSON system will be implemented, and may be switched to a SQL database at a later stage if time permits. However, for now, a JSON file will be sufficient, as not much data needs to be stored.

logic > JS stats.js > ...

```
1   function getStats() { // Retrieve stats from JSON file
2       const data = fs.readFileSync('stats.json', 'utf-8');
3       return JSON.parse(data);
4   }
5
6   function updateStats(game) { // Update the statistics after a game concludes
7       const stats = getStats();
8       stats.gamesPlayed += 1;
9       if (game.winner === "Player") {
10          stats.wins += 1;
11      } else if (game.winner === "AI") {
12          stats.losses += 1;
13      }
14      const timeTaken = (new Date() - game.startedAt) / 1000; // Time in seconds
15      stats.totalTime += timeTaken;
16      stats.averageTime = stats.totalTime / stats.gamesPlayed;
17
18      fs.writeFileSync('stats.json', JSON.stringify(stats, null, 2)); // Save to file for persistence
19  }
20
21
22  module.exports = { getStats, updateStats };
```

1
2
3
4

1. The getStats function will be used to read data from a JSON file, and return the data.
2. The updateStats function will take a game instance as an attribute. It will then call the getStats function to retrieve the current stats from the JSON file.

3. The gamesPlayed value is incremented in memory, and the winner of the game determines if the wins should increment or if losses should increment.
4. The duration of the game is calculated by subtracting the startedAt value from the current timestamp, then divided by 1000 to get the time in seconds. The totalTime stat is increased, and the average game time is recalculated. The stats are then written back to the stats file using the writeFileSync method of fs. An async approach could be used, but the performance difference would not be noticeable, as its only writing a small amount of data.

In other algorithms, when the gameOver status is checked, it will now also update the statistics by calling the updateStats() function, passing in the game instance.

```
83      if (game.checkGameOver(game.aiBoard.grid)) { // Check if all AI ships are sunk
84          this.gameOver = true;
85          this.winner = "Player";
86          updateStats(game); // Update stats if game is over
87      } else if (game.checkGameOver(game.playerBoard.grid)) { // Check if all player ships are sunk
88          this.gameOver = true;
89          this.winner = "AI";
90          updateStats(game); // Update stats if game is over
91      }
```

This current implementation of statistic tracking will not allow for multiple different users having separate statistics data. Later in development, this will be modified to distinguish different users.

Testing

Test Number	What's being tested	Expected Outcome	Actual Outcome	Passes Test
GS-1	Correct handling of missing stats file	No error occurs if stats file is missing	Error: ENOENT: no such file or directory, open 'stats.json'	No
GS-2	Stats file is created if it's not present already	File is created	-	-
GS-3	Games played counter is incremented correctly	Games played counter increments by 1	-	-
GS-4	Total time increments correctly	Total time increments by the game duration	-	-
GS-5	Average game time increments correctly	Average game time recalculates to the correct time	-	-
GS-6	Statistics increment correctly when no data is present	No error occurs if a statistic is undefined	-	-

Test 1 result:





Test 1 failed, as the file didn't exist. This means none of the other tests can be carried out, as the algorithm errors immediately, causing them all to fail with the same error.

To resolve this, I implemented validation to contain a fallback.



Instead of erroring if the file can't be read, it will instead move to the catch clause which returns an empty JSON object.

I chose to add this validation because otherwise it can create errors when the file does not exist, which would be problematic when deploying to a new system.

Test Number	What's being tested	Expected Outcome	Actual Outcome	Passes Test
GS-1	Correct handling of missing stats file	No error occurs if stats file is missing	No error occurs	Yes
GS-2	Stats file is created if it's not present already	File is created	File is created	Yes

GS-3	Games played counter is incremented correctly	Games played counter increments by 1	Increments by 1
			{
			"gamesPlayed": 2,
			....
			}
				Yes

GS-4	Total time increments correctly	Total time increments by the game duration	Increments by game time
			{
			...
			"totalTime": 7.129,
			...
			}
				Yes

GS-5	Average game time increments correctly	Average game time recalculates to the correct time	Recalculates correctly
			{
			....
			"averageTime": 3.5645
			....
			}
				Yes

GS-6	Statistics increment correctly when no data is present	No error occurs if a statistic is undefined and data is written correctly	File is populated with null data
			{
			"gamesPlayed": null,
			"losses": null,
			"totalTime": null,
			"averageTime": null
			}
				No

GS-7	Statistics increment correctly when data is present	No error occurs and data is written correctly	Correct data is written
			{
			"gamesPlayed": 1,
			"losses": 1,
			"totalTime": 3.528,
			"averageTime": 3.528
			}
				Yes

Sample response:



Test 6 failed as the populated data was not correct. This could be debugged to identify the root cause, but due to time constraints, I will instead just make a small change to prevent this issue from occurring.

```
function getStats() { // Retrieve stats from JSON file
    try {
        const data = fs.readFileSync('stats.json', 'utf-8');
        return JSON.parse(data);
    } catch (err) {
        const data = {
            "gamesPlayed": 0,
            "losses": 0,
            "totalTime": 0,
            "averageTime": 0
        };
        return data;
    }
}
```

By returning the correct fields instead of just {}, this issue no longer occurs. The root cause may be debugged later on in development, but for now the issue is resolved.

6	Statistics increment correctly when no data is present	No error occurs if a statistic is undefined and data is written correctly	File is populated with null data
			{
			"gamesPlayed": 1,
			"losses": 1,
			"totalTime": 4.239,
			"averageTime": 4.239
			}
				Yes

To test persistence, the following test will be carried out.

What's being tested	Expected Outcome	Actual Outcome	Passes Test

GET /stats returns the correct data after a server restart	The data is returned, and no error occurs	Data is returned correctly:
		{
		"gamesPlayed": 1,
		"losses": 1,
		"totalTime": 4.239,
		"averageTime": 4.239
		}
			Yes

As the test passes, the statistics system seems to be functional and robust for use. This means I can now work on developing other parts of the solution.

Frontend Development

At this stage, all core/important parts of the backend have now been implemented and tested to ensure they are suitable for use. While success criteria 14 has not yet been met, this is something that can be implemented at a later stage, as while this is not available for public use, the memory leak isn't a big problem.

I can now move onto developing a simple frontend to interact with the Node.js backend, in order to visually represent the data for the game in the backend. This will contain all the parts of "User Interface" in the project breakdown. In particular, the get-game-state endpoint will be a basis for visualising all the key information about the game and its current state in a user-friendly way, as it will be a web-based GUI.

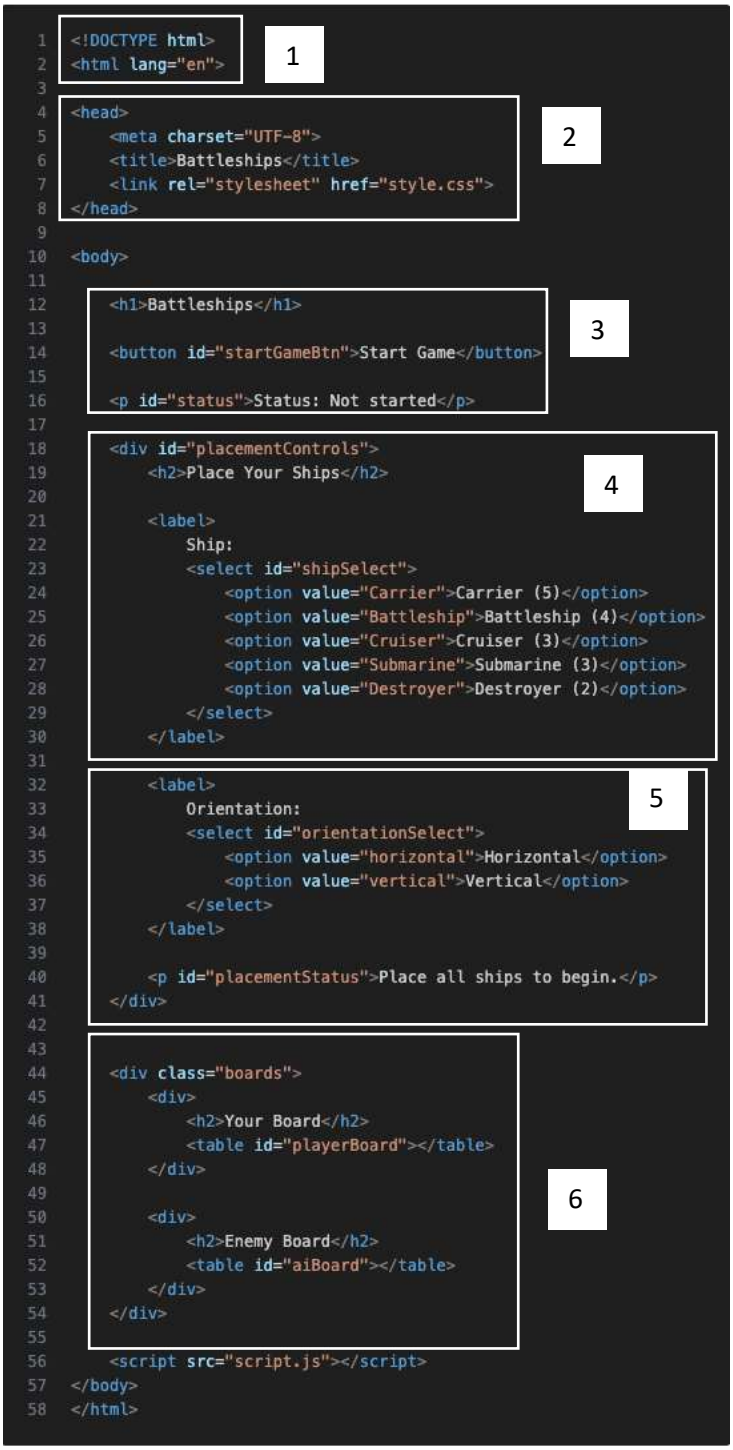
The reason I chose to delay the development of the frontend until now is because without a fully functional backend, the frontend will be able to function as expected, making it less useful. It can also create time pressure for developing the more important algorithms, leading to less thorough testing, making the user experience worse.

The frontend will be created as a web page in HTML and JS. I could have instead chosen to create a desktop app, but a webpage is more accessible to more devices and doesn't require installation.

This part of development will focus on success criteria 1, 2, 4, 7, 8, 9, 10, 12, and 13.

Initial frontend structure

I created a simple HTML page, with basic CSS styling in order to create a simple UI for testing the backend endpoints.



```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>Battleships</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <h1>Battleships</h1>

    <button id="startGameBtn">Start Game</button>

    <p id="status">Status: Not started</p>

    <div id="placementControls">
        <h2>Place Your Ships</h2>

        <label>
            Ship:
            <select id="shipSelect">
                <option value="Carrier">Carrier (5)</option>
                <option value="Battleship">Battleship (4)</option>
                <option value="Cruiser">Cruiser (3)</option>
                <option value="Submarine">Submarine (3)</option>
                <option value="Destroyer">Destroyer (2)</option>
            </select>
        </label>

        <label>
            Orientation:
            <select id="orientationSelect">
                <option value="horizontal">Horizontal</option>
                <option value="vertical">Vertical</option>
            </select>
        </label>

        <p id="placementStatus">Place all ships to begin.</p>
    </div>

    <div class="boards">
        <div>
            <h2>Your Board</h2>
            <table id="playerBoard"></table>
        </div>

        <div>
            <h2>Enemy Board</h2>
            <table id="aiBoard"></table>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

1. The doctype and language are defined
2. The charset, page title, and css file link is set
3. The header and buttons are created
4. The ship placement controls allow the user to choose a ship

5. The orientation selector allows the user to choose whether the ship should be placed horizontally or vertically
6. The boards div contains the player's board and the AI's grid, represented with tables

Relevant style data is contained in style.css.

I chose to use simplistic HTML and CSS in order to make the layout simple and clear. This works towards meeting success criteria 8. This may also incidentally work towards success criteria 7.

Now that there is a frontend UI, it needs JavaScript code to make it interactive.

I created a simple JavaScript script to make the page interactive. The script is attached into the HTML using the <script> with a src of "script.js".

```
let gameId = null;
const gridSize = 10;

const playerBoardEl = document.getElementById("playerBoard");
const aiBoardEl = document.getElementById("aiBoard");
const statusEl = document.getElementById("status"); // Status element for messages about game state etc

document.getElementById("startGameBtn").addEventListener("click", startGame);

async function startGame() {
    const res = await fetch("/api/game/start-game", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ difficulty: "easy" })
    });

    const data = await res.json();
    gameId = data.gameId;
    statusEl.textContent = "Game started";
    await updateGameState();
}

async function updateGameState() {
    const res = await fetch(`/api/game/get-game-state?gameId=${gameId}`);
    const data = await res.json();

    renderBoard(playerBoardEl, data.playerBoard, false);
    renderBoard(aiBoardEl, data.aiBoard, true);

    if  data.gameOver) {
      ( statusEl.textContent = `Game Over. Winner: ${data.winner}`;
    }
}
```

1. The gameID is initially set to null, and the gridSize is set to 10 to ensure predictable logic. The player board div, the ai's board div, and the status element are all defined using the getElementById function.
2. An event listener is added to the start game button, which triggers the startGame function.
3. The startGame function uses fetch() to make a POST request to the start-game endpoint. It provides the JSON body of difficulty: "easy", although this currently is the only option anyway.
4. The updateGameState function will make a request to the get-game-state endpoint, passing in the game ID, and retrieving game state data. This is then used to update the boards and tell the user that the game has ended (if applicable). This works towards success criteria 12. While it doesn't fully meet it as it isn't a full results screen, it is a start which can be improved later in development if the time constraints allow it.

```
function renderBoard(table, grid, isEnemy) {
    table.innerHTML = "";

    for  let r = 0; r < gridSize; r++) {
       (const row = document.createElement("tr"); // tr = table row

        for  let c = 0; c < gridSize; c++) {
           (const cell = document.createElement("td"); // td = table data cell
            const value = grid[r][c];
```

5

```
            if  value === "hit") cell.className = "hit";
            else if  value === "miss") cell.className = "miss";
            else if( value === 1 && !isEnemy) cell.className = "ship";
            else cell.className = "water";

            // Ship placement
            if  !isEnemy && placementStage) {
              ( // Attach an event listener to each cell for placing ships
                cell.addEventListener("click", () => placeShip(r, c));
            }

            // Attacking AI (only after placement)
            if  isEnemy && !placementStage && value === null) {
              ( // Attach an event listener to each cell for attacking
                cell.addEventListener("click", () => playerMove(r, c));
            }

            row.appendChild(cell);
        }

        table.appendChild(row);
    }
}
```

6

5. The renderBoard function is used to display a boards to the user. The function takes parameters of the html element of a board, the grid data, and if the grid is the opponents grid or not. The algorithm first clears the innerHTML of the board. It then iterates through every grid cell, creating a grid table for each row, then a grid data cell for each column in that row, in order to create a full table.
6. The algorithm then checks the status of the grid cell from the returned data and applies the relevant className, which determines the style of the cell in order to visibly indicate the status to the user. The algorithm then attaches an event listener to each grid cell which triggers the relevant function based on who's board it is, and the current game stage. This event listener allows the project to meet success criteria 13 as well as 15.

```
async function playerMove(row, col) {
    const res = await fetch("/api/game/player-move", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ gameId, row, col })
    });

    const data = await res.json();

    if  data.error) {
      ( statusEl.textContent = data.error;
        return;
    }

    await updateGameState();
}
```

7

7. The playerMove function makes a POST request to the player-move endpoint, passing in the game ID, row, and column. The data is then retrieved. Currently this data is not used to determine game updates, but instead just to determine if the move failed due to things such as invalid attacked cell. The updateGameState function is then called (if the move succeeded to avoid excess API calls) to update the UI.



```
let placementStage = true;

const ships = {
    Carrier: 5,
    Battleship: 4,
    Cruiser: 3,
    Submarine: 3,
    Destroyer: 2
};

const placedShips = new Set();

async function placeShip(row, col) {
    const shipName = document.getElementById("shipSelect").value;

    if (placedShips.has(shipName)) {
        statusEl.textContent = "Ship already placed";
        return;
    }

    let orientationVal = document.getElementById("orientationSelect").value;
    const orientation = orientationVal === "horizontal" ? "H" : "V";

    const res = await fetch("/api/game/place-ship", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            gameId,
            shipType: shipName,
            startRow: row,
            startCol: col,
            orientation
        })
    });

    const data = await res.json();

    if (!data.success) {
        statusEl.textContent = data.message;
        return;
    }

    placedShips.add(shipName);
    statusEl.textContent = `${shipName} placed`;

    await updateGameState();

    if (placedShips.size === Object.keys(ships).length) {
        placementStage = false;
        document.getElementById("placementStatus").textContent = "All ships placed.";
    }
}
```

8
9
10

8. A placementStage variable is set to true to indicate that the user shouldn't be able to attack grid cells yet. The ships are defined, and a new Set is initialised to hold data about the currently placed ships.
9. The placeShip function retrieves the value of the selected ship. It then checks if that ship already exists in placedShips, returning a simple message to the user. The orientation is then checked, and a POST request is made to the place-ship endpoint. The game ID, ship type, start row, start column, and orientation are passed in.
10. If the placement was unsuccessful, it shows a message to the user and the algorithm ends. The ship is then added to the placedShips set and a message is shown to the user. The updateGameState function is called and the algorithm checks if all ships have been placed. The message helps to work towards success criteria 10.

Frontend testing

Test Number	What's being tested	Expected Outcome	Actual Outcome	Passes Test
FE-1	Player can place ships at the start of the game	No errors occur and all ships can be placed	No errors occur and all ships can be placed	Yes
FE-2	Player can't attack opponent before ships are placed	Player cannot attack opponent before they place all ships	Player cannot attack opponent before they place all ships. No useful message provided to the user though.	Yes
FE-3	Relevant ship placement messages are shown to the user	Messages are shown depending on the user action	Messages such as "All ships placed" are shown after relevant actions.	Yes
FE-4	Player can attack opponent after all ships are placed	Player attacks are processed	Nothing happens	No

Testing video name: "frontend testing 1.mov"

0:00 – FE-1

0:05 – FE-2

0:13 – FE-3

0:28 FE-4

Progress Review

This frontend runs without any outputted errors, although it does contain some problems as it is not yet fully functional. For example, the attacking logic doesn't seem to work as determined by test 4. This may be tricky to debug due to it not being an outputted error. Furthermore, the creation of event listeners at each game state refresh may cause a memory leak, but this can be addressed at a later stage as it isn't critical.

Furthermore, the UI is not as finalised as I hope it to be eventually. The UI doesn't currently meet the planned wireframe diagram, and there is limited functionality. For example, once the game starts, if the user refreshes the page, there is no way to continue the game.

Stakeholder Feedback

- What feedback can you provide on the functionality of the game?
  "I can start the game easily and place ships. It makes sense how to do it, but then I can't attack after I finish placing the ships which is a bit confusing"

To understand if the UI is suitable for users, in order to meet success criteria 1 and 8 (easily navigable UI and simple layout), I asked a potential stakeholder.

- Is the UI easy to navigate around?
  "Yes I can see what to do"

- Is there too much clutter in the UI?
  "No its very clear and simple"

To fix the bug in test 4, I moved the updateGameState to after the the placementStage is set to false, otherwise the event listeners are never attached.

```
131
132     placedShips.add(shipName);
133     statusEl.textContent = `${shipName} placed`;
134
135     if (placedShips.size === Object.keys(ships).length) {
136         placementStage = false;
137         document.getElementById("placementStatus").textContent = "All ships placed.";
138     }
139
140     await updateGameState();
141 }
```

```
56              // Attacking AI (only after placement)
57              if (isEnemy && !placementStage && value === 0) {
58                  // Attach an event listener to each cell for attacking
59                  cell.addEventListener("click", () => playerMove(r, c));
60              }
```

I also had to change "value === null" to "value === 0" as 0 represents an unattacked grid cell.

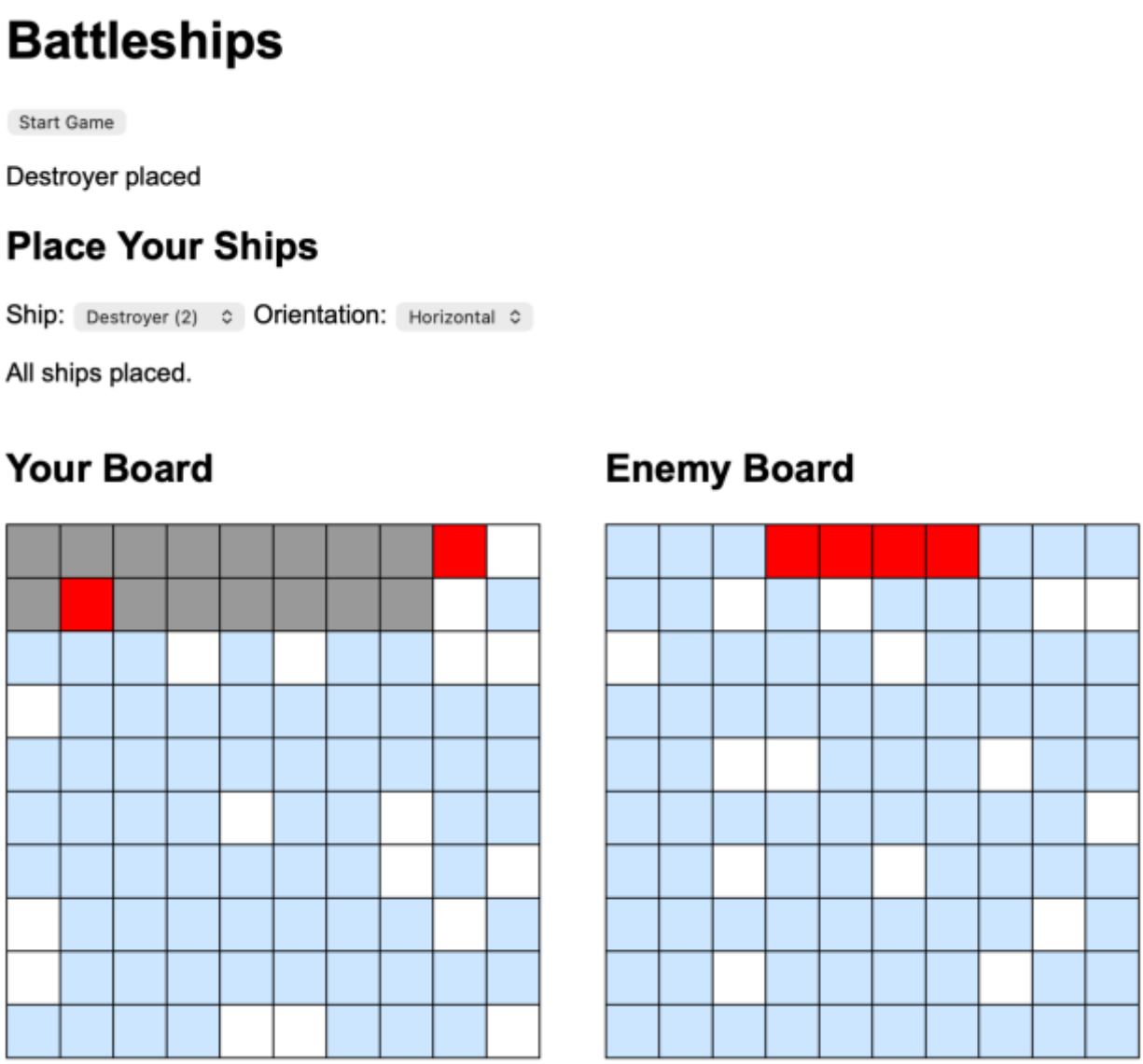
I did this because otherwise only cells with a state of null could be attacked, but there is never a situation where the cell state is null, meaning that no cells could be attacked.

After making those changes, the game now works as expected.

This is demonstrated in this video:

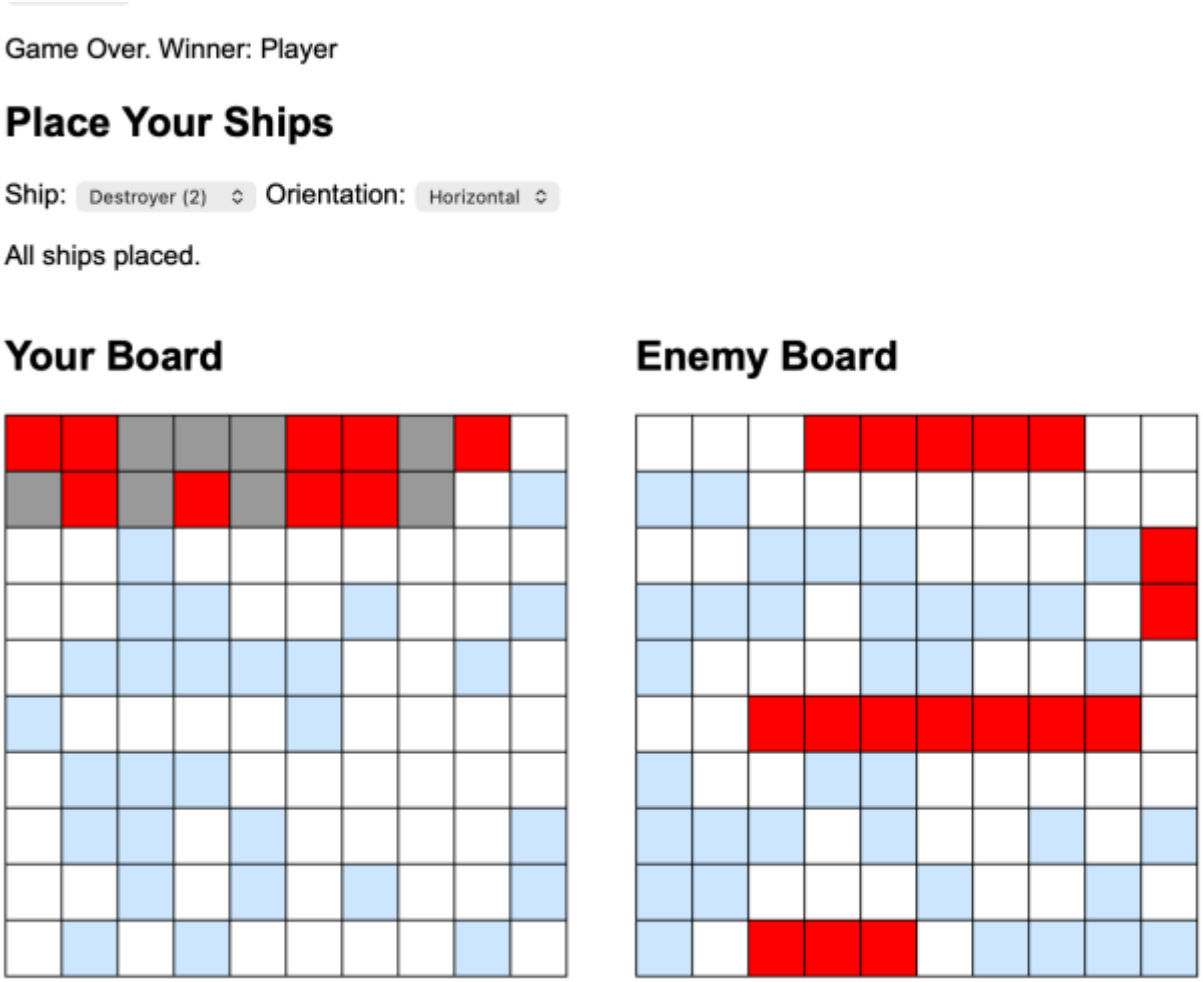
Gameplay demonstration video name: "game state change test.mov"

As seen in the screenshot below, the user can now attack cells and the opponent then attacks their grid. Grid cells change colour to indicate different states, meeting success criteria 9, as the responses to actions are clear to the user.



Test Number	What's being tested	Expected Outcome	Actual Outcome	Passes Test
FE-4	Player can attack opponent after all ships are placed	Player attacks are processed	Nothing happens	Yes
FE-5	Player can't attack the same grid cell more than once	Repeated attacks are ignored	Repeated attacks are ignored client-side, avoiding excess requests to the backend process.	Yes
FE-6	Sunk ships are distinct from hit ships	Sinking a ship changes the entire ship's colour	Ship colour remains the same – the red colour indicating hit	No

FE-7	Game ends when the last ship is hit	The game ends and the winner is announced	The game ends and the winner is announced	Yes



Test 7 reveals that a change needs to be made to the frontend and potentially backend code to implement a feature to reveal the entire ship as sunk once all cells the ship occupies are hit. This is not yet implemented, and should be easy to implement.

Stakeholder Feedback

- Can you provide feedback on the functionality of the game at this stage?
  "The game starts and I can place the ships. This time it lets me attack the opponent which it didn't before so that's good"

Progress Review

The solution is now in a prototype state where it's functionally working to a basic level, but there are still a few changes that should be made.

1. After the game ends, don't even attempt to request player-move endpoint when user clicks grid cells to avoid unnecessary API calls.

2. Ships shouldn't be able to be placed within 1 cell of each other.
3. Ships should change colour once they have been sunk.

Both changes should be easy to implement due to the problems being able to be broken down into small tasks that can be solved independently in frontend and backend.

There are still a few other enhancements that should be made to the frontend, but these can be implemented at a later stage.

Making the 3 identified changes

The first change I will tackle is preventing ships from being placed within 1 cell of each other. Currently success criteria 15 is not met due to this problem. Due to the modularity of the solution, I can identify that the function to modify is the validateShipPlacement algorithm.

The problem originated from the original algorithm plan I created. I had forgotten that validation to prevent ships touching needed to be present.

I produced an updated algorithm to now meet this criteria

```
function validateShipPlacement(size, startRow, startCol, orientation, grid) {
  const gridSize = grid.length;

  if (orientation === "H" && startCol + size > gridSize) return false;
  if (orientation === "V" && startRow + size > gridSize) return false;

  // Check the area including the ship and its adjacent cells to prevent touching
  let minRow, maxRow, minCol, maxCol;
  if (orientation === "H") {
    minRow = Math.max(0, startRow - 1);
    maxRow = Math.min(gridSize - 1, startRow + 1);
    minCol = Math.max(0, startCol - 1);
    maxCol = Math.min(gridSize - 1, startCol + size);
  } else { // Vertical
    minRow = Math.max(0, startRow - 1);
    maxRow = Math.min(gridSize - 1, startRow + size);
    minCol = Math.max(0, startCol - 1);
    maxCol = Math.min(gridSize - 1, startCol + 1);
  }

  for (let r = minRow; r <= maxRow; r++) {
    for (let c = minCol; c <= maxCol; c++) {
      if (grid[r][c] !== 0) return false;
    }
  }

  // Ship placement valid
  return true;
}
```

This updated algorithm now check adjacent cells as well as the ship-occupied cells to prevent ships from being placed too close to each other.

After testing this algorithm, I can confirm that it works, as ships can no longer be placed next to each other. This means success criteria 15 has been met.

As seen in the testing video, ships can only be placed when there is at least a 1 cell gap between them and another ship.

Testing video name: “frontend - clash post edit test.mov”

The second change to make is preventing unnecessary requests being made to the backend server once the game ends.

To make this change, I can simply implement a new flag to restrict what functions can run when. I created a new variable gameOver to act as this flag. By default, it is set to false. Once the game ends, the flag is set to true to indicate that the game has ended.

```
57  let gameOver = false;
58
59  async function updateGameState() {
60      const res = await fetch(`/api/game/get-game-state?gameId=${gameId}`);
61      const data = await res.json();
62
63      renderBoard(playerBoardEl, data.playerBoard, false);
64      renderBoard(aiBoardEl, data.aiBoard, true);
65
66      if (data.gameOver) {
67          statusEl.textContent = `Game Over. Winner: ${data.winner}`;
68          gameOver = true;
69      }
70  }
```

This flag can then be used to check if the game is over, preventing unnecessary calls to the backend.

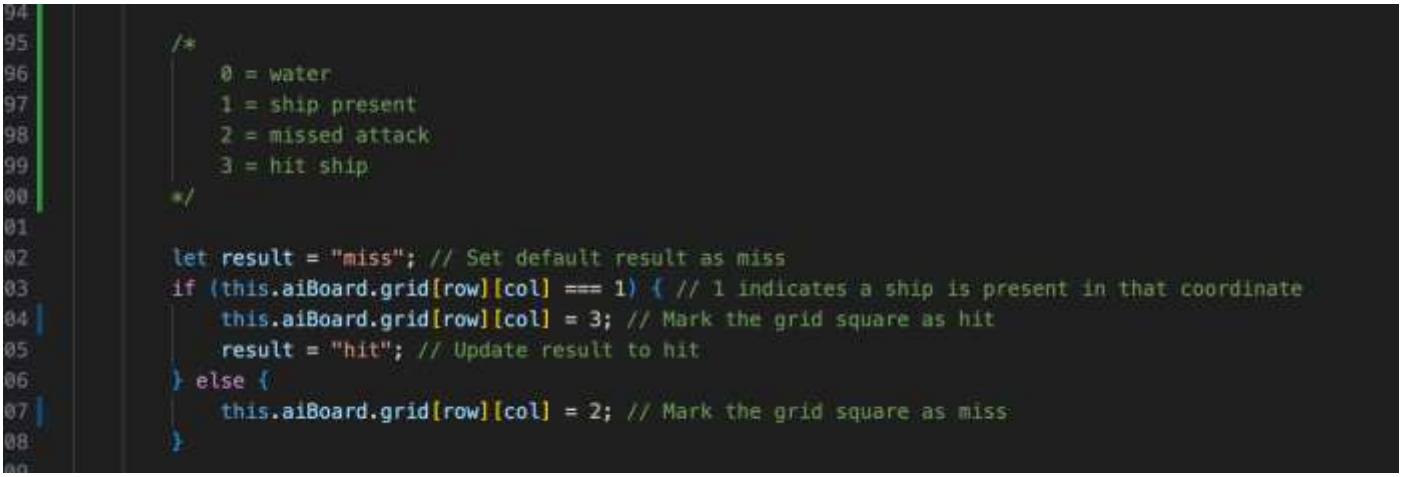
```
107 async function playerMove(row, col) {
108     if (gameOver) return;
109     const res = await fetch("/api/game/player-move", {
```

To prevent unexpected behaviour, this flag needs to be reset back to false when a game starts, otherwise if the user plays 2 games, the second game won't function correctly.

```
43  async function startGame() {
44      const res = await fetch("/api/game/start-game", {
45          method: "POST",
46          headers: { "Content-Type": "application/json" },
47          body: JSON.stringify({ difficulty: "easy" })
48      });
49
50      const data = await res.json();
51      gameId = data.gameId;
52      statusEl.textContent = "Game started";
53      gameOver = false;
54      await updateGameState();
55  }
```

The last of the 3 changes is changing the colour of the ships once they have been sunk to avoid confusing the player.

Before making changes for this, it is a logical time to change the cell data to use only 1 data type. This improves consistency and will avoid issues that may be caused later on.



```
/*
    0 = water
    1 = ship present
    2 = missed attack
    3 = hit ship
*/

let result = "miss"; // Set default result as miss
if (this.aiBoard.grid[row][col] === 1) { // 1 indicates a ship is present in that coordinate
    this.aiBoard.grid[row][col] = 3; // Mark the grid square as hit
    result = "hit"; // Update result to hit
} else {
    this.aiBoard.grid[row][col] = 2; // Mark the grid square as miss
}
```

I created a key for me to refer back to for maintainability so I remember what each value represents. I then replaced any mentions of “hit” or “miss” within the codebase with their new representations, as seen in the above screenshot.

Now it's a more suitable point to implement the sunk logic, as I can simply add a new state. The most logical number to use would be 4, as that's the next number up after a hit ship.

To plan out the changes needed:

1. Create a helper function to iterate through all cells, detecting any ships with all cells hit (status of 3), and set their status to sunk (4).
2. Call this function in both player move logic and ai move logic
3. Update the frontend JavaScript to set a different class to the sunk cells
4. Add a new style to the stylesheet to display a different colour for the new class

To start off these changes, I created the helper method for the Game class:




```
checkAndUpdateSunkShips(board) {
    for (const ship of board.ships) {
        let hitCount = 0;
        for (let i = 0; i < ship.size; i++) {
            let r = ship.row;
            let c = ship.col;
            if (ship.orientation === "H"){
                c += i;
            } else {
                r += i;
            }

            if (board.grid[r][c] === 3) { // Check if that part of the ship is hit
                hitCount += 1;
            }
        }
        if (hitCount === ship.size) { // All parts of the ship are hit
            // Mark the ship as sunk on the grid
            for (let i = 0; i < ship.size; i++) {
                let r = ship.row;
                let c = ship.col;
                if (ship.orientation === "H") c += i;
                else r += i;
                board.grid[r][c] = 4; // 4 indicates sunk ship
            }
        }
    }
}
```

The logic can be split up into these sections:

1. The method accepts a board parameter. It iterates through every ship on the inputted board, initialising a new hitCount counter.

2. A for loop is used to iterate for the ship size (I had to research how to create the for loop using StackOverflow). Variables r and c are initialised containing the starting co-ordinate for the ship. Depending on the ship orientation, one of these will be incremented to access the next cell of the ship.

3. If the grid cell is 3 (representing a hit), then the hit counter is incremented.

4. Once every cell of the ship has been iterated through, the algorithm checks if the hit counter is equal to the ship size. This would mean that every cell of the ship is hit. If this is true, the algorithm iterates through every cell of the ship with the same iteration logic and replaces each cell's status with 4, representing sunk.

I then implemented this logic into the player and opponent algorithms. For example:

```
    }

    this.checkAndUpdateSunkShips(this.aiBoard); // Check if any opponent ships are sunk after the move
```

This will ensure the ships are updated correctly after every move.

At this stage, all the backend changes are finished, but the frontend will still not yet reflect these changes.

To make the new ship state reflect on the frontend grid, changes need to be made to the frontend. As the majority of the solution's logic is based on the backend, only very minor tweaks will be needed on the frontend.

```
function renderBoard(table, grid, isEnemy) {
    table.innerHTML = "";

    for (let r = 0; r < gridSize; r++) {
        const row = document.createElement("tr"); // tr = table row

        for (let c = 0; c < gridSize; c++) {
            const cell = document.createElement("td"); // td = table data cell
            const value = grid[r][c];

            if (value === 3) cell.className = "hit";
            else if (value === 4) cell.className = "sunk";
            else if (value === 2) cell.className = "miss";
            else if (value === 1 && !isEnemy) cell.className = "ship";
            else cell.className = "water";
```

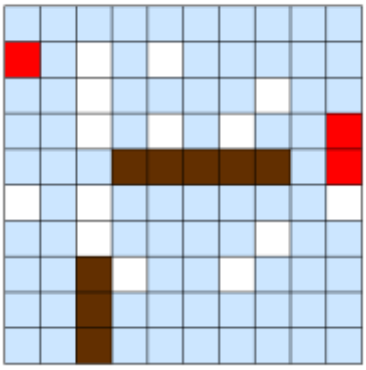
In script.js, I added one line to handle the cell status of 4, setting the class to "sunk".

```
.sunk {
  background-color: rgb(98, 47, 0);
}
```

I then just needed to add the sunk class to style.css.

After making these changes, the sunk ships now show in a darker colour, improving user experience and usability, further building upon success criteria 9.

Enemy Board



Stakeholder Feedback

- Does the additional colour make the game more usable?
  "I can see easily when the ship is sunk so it's much less confusing because before I wasted attacks after the ship was sunk because I never knew"

Improving the GUI

To work towards the planned UI design, I need to make some changes to the layout. This section will focus on success criteria 1 by making the UI easy to navigate.

I created a side navigational bar with this HTML:

```
<div id="sidebar">
    <button id="playBtn">Play Game</button>
    <button id="statsBtn">Statistics</button>
</div>
```

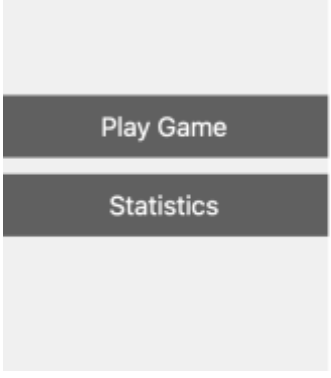
I then added CSS styles to customise the design of it.

```
#sidebar {
  position: fixed;
  left: 0;
  top: 0;
  width: 200px;
  height: 100%;
  background-color: #f0f0f0;
  padding-top: 60px;
  box-sizing: border-box;
}

#sidebar button {
  display: block;
  width: 100%;
  margin-bottom: 10px;
  padding: 10px;
  background-color: #606060;
  color: white;
  border: none;
  border-radius: 0;
  cursor: pointer;
  font-size: 1em;
}

#sidebar button:hover {
  background-color: #454545;
}
```

After making these changes, there is now a simplistic looking side navigation bar.



I also made other changes to the CSS:

```
.boards {
  display: flex;
  gap: 40px;
  justify-content: center;
}
```

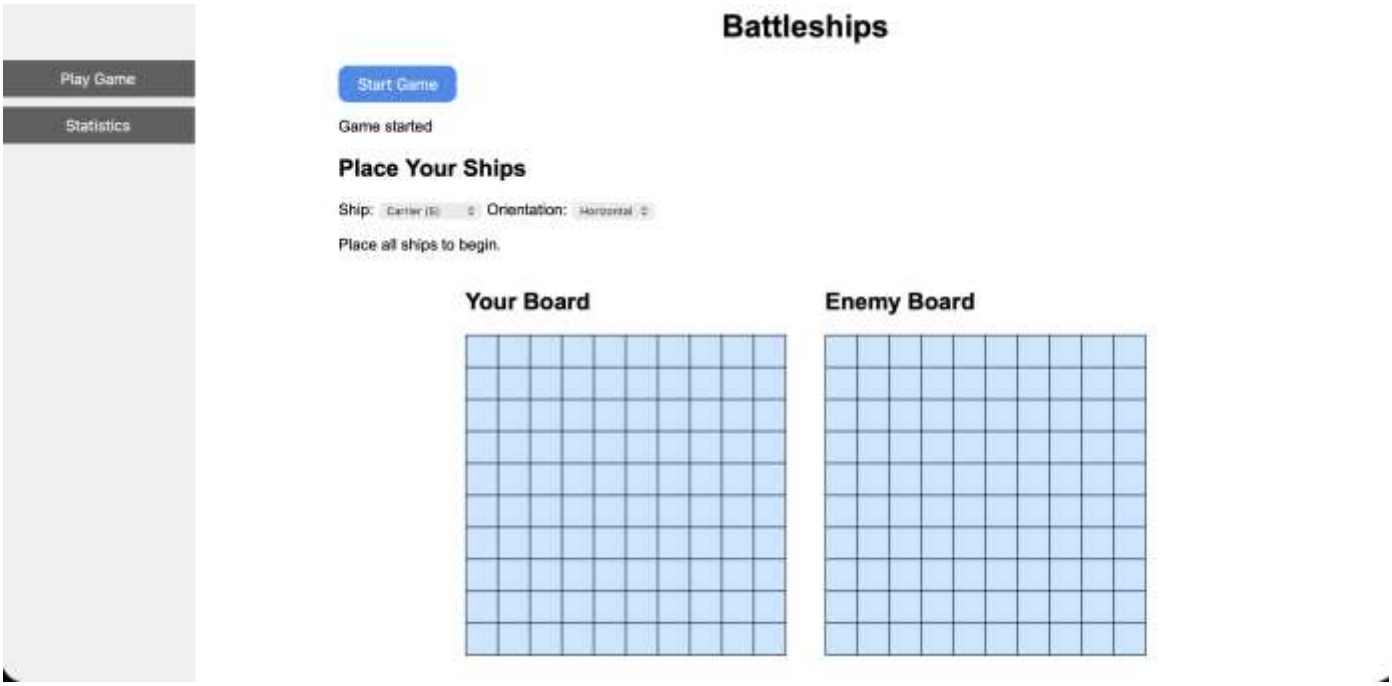
```
#main-content {
  margin-left: 220px;
  padding: 20px;
}

h1 {
  text-align: center;
  margin-top: 0;
}

button {
    font-size: 1em;
    padding: 10px 20px;
    background-color: #5280e7;
    color: white;
    border-radius: 10px;
    border: none;
    cursor: pointer;
}

.top-controls {
    margin-left: 100px;
}
```

After making these changes, the GUI now looks a lot closer to the planned wireframe.



At this stage, the layout now closely meets the planned design, and it meets a lot of the planned usability criteria, such as the layout being simple and easy to use, due to few unnecessary elements and large buttons. The larger and different coloured buttons also help to meet success criteria 16.

Stakeholder Feedback

- Do you like the new layout and do you think it's easier to navigate?
  "It looks a lot newer and the colours of the buttons like the start game button make it easier to see what to do"

Implementing the frontend statistics page

Game statistics are already being managed by the backend, so no big frontend logic needs to be implemented beyond loading the statistics from the backend. Frontend changes are however necessary in order to meet success criteria 4.

To improve user experience, I decided to use a single-page design rather than making the navigational bar direct the user to another page. This way the loading is almost instant.

I first added this HTML content which will be populated with data:

```
<div id="stats-page" style="display: none;">
    <h1>Statistics</h1>
    <div id="stats-display">
        <p id="games-played-counter"></p>
        <p id="player-wins-counter"></p>
        <p id="ai-wins-counter"></p>
        <p id="win-rate"></p>
        <p id="avg-game-time"></p>
    </div>
</div>
```

To make the stats page functional, I added this JavaScript code:

```
document.getElementById("startGameBtn").addEventListener("click", startGame);
document.getElementById("playBtn").addEventListener("click", showPlay);
document.getElementById("statsBtn").addEventListener("click", showStats);

// Show the main game page (shown by default)
function showPlay() {
    document.getElementById("play-page").style.display = "block";
    document.getElementById("stats-page").style.display = "none";
}

// Show the stats page
function showStats() {
    document.getElementById("play-page").style.display = "none";
    document.getElementById("stats-page").style.display = "block";
    loadStats(); // Dont really need to await it because nothing requires the results
}

async function loadStats() {
    const res = await fetch("/api/game/stats");
    const stats = await res.json();

    const gamesPlayedCounter = document.getElementById("games-played-counter");
    const playerWinsCounter = document.getElementById("player-wins-counter");
    const aiWinsCounter = document.getElementById("ai-wins-counter");
    const winRateDisplay = document.getElementById("win-rate");
    const avgGameTimeDisplay = document.getElementById("avg-game-time");

    const winRate = stats.gamesPlayed > 0 ? ((stats.wins / stats.gamesPlayed) * 100).toFixed(2) : 0;
    gamesPlayedCounter.textContent = `Games Played: ${stats.gamesPlayed}`;
    playerWinsCounter.textContent = `Player Wins: ${stats.wins || 0}`;
    aiWinsCounter.textContent = `AI Wins: ${stats.losses || 0}`;
    winRateDisplay.textContent = `Win Rate: ${winRate}%`;
    avgGameTimeDisplay.textContent = `Average Game Time: ${stats.averageTime.toFixed(2)} seconds`; // Limits to 2 decimal places
}
```

1. Click event listeners are added to the 2 nav buttons. The event listeners trigger functions that hide one page and display another. The showStats function specifically calls the loadStats function to load the statistics.

2. The loadStats function first uses fetch() to make a GET request to the /stats endpoint. The response is then parsed as JSON.

3. The relevant HTML elements are retrieved and stored and constants

4. The win rate is calculated, and each HTML element is populated with the data. Using "|| 0" for some stats acts as a fallback, just in case for some reason the value isn't defined.



Progress Review

At this point, the frontend is mostly done, with some additional low-priority changes that could be made either further on in development or at a later date, due to them not significantly impacting the user experience or functionality. Currently success criteria 1, 2, 4, 5, 7, 8, 9, 11, 13, 15, and 16 have been met, while some other success criteria have either been partially met or not met at all.

The frontend algorithms don't contain much validation, and this is intentional because the backend already consists of extensive validation for all endpoints so adding additional validation in the frontend would not make any difference. While it is still best practice to implement additional validation as it would reduce load on the backend server, due to time constraints this is low priority. However, if this solution was being pushed to production, this would be a good thing to implement.

Most importantly, the solution currently works to a satisfactory level as a prototype of the final solution.

Stakeholder Feedback

- Do you find the statistics page useful and easy to understand?
  "It's kind of useful because it shows me the data as numbers but I would prefer things like charts to make it more visual"

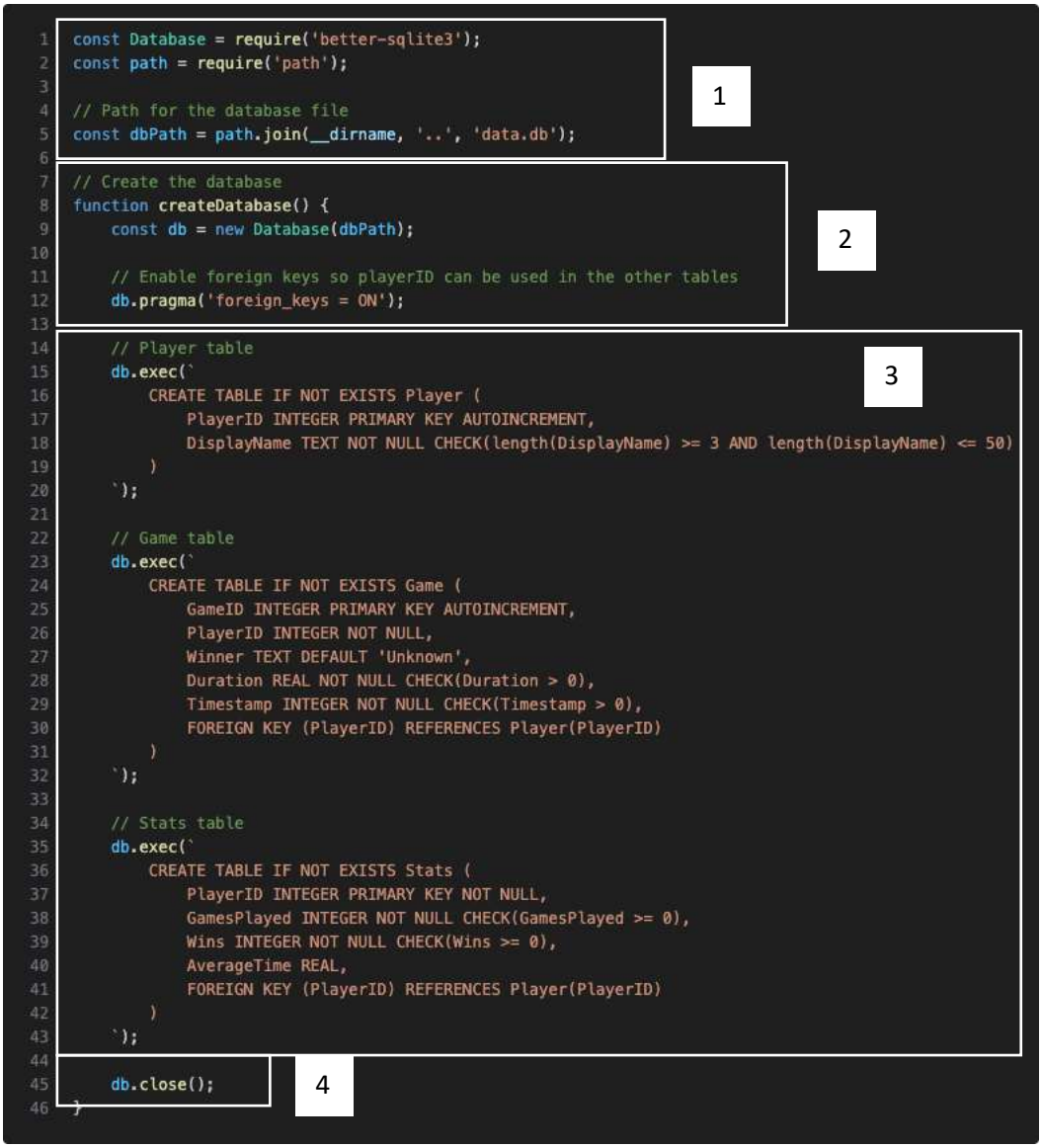
From the feedback, I have identified an additional desire of the stakeholders for the statistics page. This however is unlikely to be implemented due to the time scale and the low priority for this addition. This may be added in future if this solution continues to be maintained.

Implementing Database & Player Logic

At this point, all game statistics are stored in a JSON file with no separation between users. While this works for development, it's not ideal for a production environment. By implementing a proper database system, the data can be stored in a way that is less likely to get corrupted and is more robust. While implementing a database system, I will design it so that player data is separated, as planned in the design of this project. This will build upon success criteria 4, as the data will be more unique to the user.

Using a full SQL database would require a lot of setup with lots of external dependencies. For a solution like this, a SQLite database may be a better approach, as it's more robust than a .json file, but less setup than a full SQL database. Moving to a full SQL database would be a good idea if this project was used in a big production environment, but for now an SQLite database will be suitable.

To start off, I first created a module file for managing the database. This file first contains a createDatabase() function used for initially creating the SQLite database.




```
const Database = require('better-sqlite3');
const path = require('path');

// Path for the database file
const dbPath = path.join(__dirname, '..', 'data.db');

// Create the database
function createDatabase() {
    const db = new Database(dbPath);

    // Enable foreign keys so playerID can be used in the other tables
    db.pragma('foreign_keys = ON');

    // Player table
    db.exec(`
        CREATE TABLE IF NOT EXISTS Player (
            PlayerID INTEGER PRIMARY KEY AUTOINCREMENT,
            DisplayName TEXT NOT NULL CHECK(length(DisplayName) >= 3 AND length(DisplayName) <= 50)
        )
    `);

    // Game table
    db.exec(`
        CREATE TABLE IF NOT EXISTS Game (
            GameID INTEGER PRIMARY KEY AUTOINCREMENT,
            PlayerID INTEGER NOT NULL,
            Winner TEXT DEFAULT 'Unknown',
            Duration REAL NOT NULL CHECK(Duration > 0),
            Timestamp INTEGER NOT NULL CHECK(Timestamp > 0),
            FOREIGN KEY (PlayerID) REFERENCES Player(PlayerID)
        )
    `);

    // Stats table
    db.exec(`
        CREATE TABLE IF NOT EXISTS Stats (
            PlayerID INTEGER PRIMARY KEY NOT NULL,
            GamesPlayed INTEGER NOT NULL CHECK(GamesPlayed >= 0),
            Wins INTEGER NOT NULL CHECK(Wins >= 0),
            AverageTime REAL,
            FOREIGN KEY (PlayerID) REFERENCES Player(PlayerID)
        )
    `);

    db.close();
}
```

1. First, better-sqlite3 and path are imported. I am choosing to use better-sqlite3 because the methods are simpler than standard sqlite3. The path for the database file is also defined.

2. In the createDatabase function, a new database is initialised. A query is run to enable foreign keys. This is needed because foreign keys will be used to connect the tables together.

3. 3 queries are run to create tables. Each table contains the fields and validation as planned in the database table plan that I created before development.

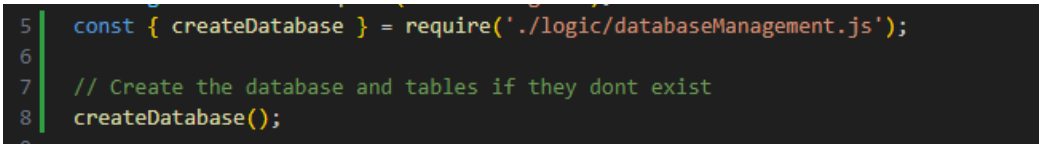
4. The database connection is then closed. It would most likely be automatically closed when the algorithm ends, but this is best practise to close it.



5. The getDatabase function is then defined. This function initialises a database instance, enables foreign_keys, and returns the database connection.

6. The 2 functions are then exported as modules.

In index.js, I then added this code:



This means that the solution can be deployed on any server device, and it will automatically create the database tables, preventing SQL errors from occurring on new host devices.

At this point, the functions needed for setting up and connecting to the database are ready.

To create the database, I just needed to run the createDatabase function once. I could then delete this function, but keeping it in the codebase improves maintainability in case a future maintainer needed to create the database again.

The next thing to do is implement the database logic elsewhere in the solution.

Logically, the first file to edit should be stats.js.

The changes made can be seen here:




```
const { getDatabase } = require('./databaseManagement');

// Create a new player
function createPlayer(displayName) {
    const db = getDatabase();
    try {
        const stmt = db.prepare('INSERT INTO Player (DisplayName) VALUES (?)');
        const info = stmt.run(displayName);
        const playerID = info.lastInsertRowid;

        // Initialize stats for the new player
        const statsStmt = db.prepare('INSERT INTO Stats (PlayerID, GamesPlayed, Wins, AverageTime) VALUES (?, 0, 0, 0)');
        statsStmt.run(playerID);

        db.close();
        return playerID;
    } catch  err) {
        db.close();
        throw err;
    }
}
```

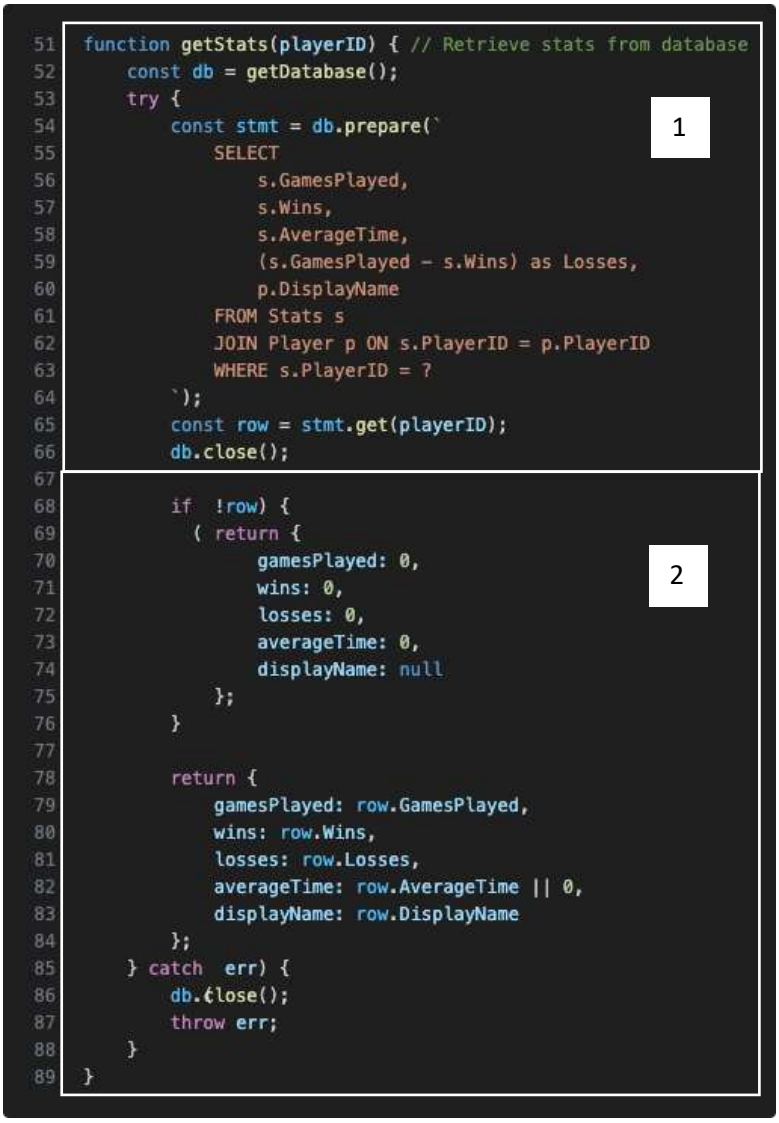
1. The getDatabase function is imported from the databaseManagement file. This means code doesn't need to be repeated unnecessarily.

2. The createPlayer function is defined. This function accepts a parameter of a display name. This will be used because statistics will be split up by player.

3. The algorithm then prepares a SQL statement for inserting a new player into the Player table. The query is then run and the player ID is retrieved by checking the row ID of the last entry. This reveals the player ID because the IDs simply use autoincrement.

4. Another query is prepared and ran. This query inserts a new record into the stats table, containing empty values which can be incremented after games are played. The connection is closed and the playerID is returned.

5. If an error occurs within the databse logic, the connection will close and the error will be thrown. In a production environment, the error should be handled more gracefully, but this will be fine for now as it allows for easier identification of logic or runtime errors.



```
function getStats(playerID) { // Retrieve stats from database
    const db = getDatabase();
    try {
        const stmt = db.prepare(`
            SELECT
                s.GamesPlayed,
                s.Wins,
                s.AverageTime,
                (s.GamesPlayed - s.Wins) as Losses,
                p.DisplayName
            FROM Stats s
            JOIN Player p ON s.PlayerID = p.PlayerID
            WHERE s.PlayerID = ?
        `);
        const row = stmt.get(playerID);
        db.close();

        if  !row) {
          ( return {
                gamesPlayed: 0,
                wins: 0,
                losses: 0,
                averageTime: 0,
                displayName: null
            };
        }

        return {
            gamesPlayed: row.GamesPlayed,
            wins: row.Wins,
            losses: row.Losses,
            averageTime: row.AverageTime || 0,
            displayName: row.DisplayName
        };
    } catch  err) {
        db.close();
        throw err;
    }
}
```


1. The getStats function gets a database connection. It then prepares a SQL query to select the relevant data from the stats table and the display name of the player from the player table. It uses a join to connect the 2 tables up with the foreign key. The query is ran and the database connection is closed.

2. The data is then checked. If the user doesn't exist in the database, it will return empty placeholder data. Otherwise, it will return the relevant data for the user. This allows for better error handling, as if empty data isn't returned for missing users, problems will occur with null data.

```
function updateStats(game) { // Update the statistics after a game concludes
    console.log("Updating stats...");

    if  !game.playerID) {
      ( console.error("No playerID found in game object");
        return;
    }

    const db = getDatabase();
    const timeTaken =  new Date() - game.startedAt) / 1000; // Time in seconds
    const timestamp =(Math.floor(new Date().getTime() / 1000); // Unix timestamp in seconds

    try {
        // Use transaction for atomicity
        const insertGame = db.prepare('INSERT INTO Game (PlayerID, Winner, Duration, Timestamp) VALUES (?, ?, ?, ?)');
        const getStatsStmt = db.prepare('SELECT GamesPlayed, Wins, AverageTime FROM Stats WHERE PlayerID = ?');
        const updateStatsStmt = db.prepare('UPDATE Stats SET GamesPlayed = ?, Wins = ?, AverageTime = ? WHERE PlayerID = ?');

        // Start transaction
        const transaction = db.transaction(() => {
            // Insert game record
            const gameInfo = insertGame.run(game.playerID, game.winner, timeTaken, timestamp);
            console.log(`Game record inserted with ID: ${gameInfo.lastInsertRowid}`);

            // Get current stats
            const row = getStatsStmt.get(game.playerID);

            const newGamesPlayed = row.GamesPlayed + 1;
            const newWins = game.winner === "Player" ? row.Wins + 1 : row.Wins;
            const totalTime =  row.AverageTime * row.GamesPlayed) + timeTaken;
            const newAverageTime = totalTime / newGamesPlayed;

            // Update stats
            updateStatsStmt.run(newGamesPlayed, newWins, newAverageTime, game.playerID);

            console.log("Stats updated successfully");
        });

        transaction();
        db.close();
    } catch  err) {
        db.close();
        console.error('Error updating stats:', err);
        throw err;
    }
}

module.exports = { getStats, updateStats, createPlayer, updatePlayerName, getPlayer };
```

1
2
3
4

1. The updateStats function first checks if a player ID was provided, as it is needed for saving the stats. If it's not provided for some reason, it will stop there. This is necessary validation to avoid errors later on.

2. A database connection is obtained and the time taken for the game, as well as the current timestamp. The SQL statements are prepared.

3. A DB transaction is started. This will prevent inconsistent data if one query succeeds but one fails. The queries are run and variables are created.

4. The transaction is run, then the connection is closed. Errors are caught and logged to the console for easier debugging.

Player management functions are also needed.

```
// Update player display name
function updatePlayerName(playerID, displayName) {
    const db = getDatabase();
    try {
        const stmt = db.prepare('UPDATE Player SET DisplayName = ? WHERE PlayerID = ?');
        const info = stmt.run(displayName, playerID);
        db.close();
        return info.changes;
    } catch  err) {
        db.close();
        throw err;
    }
}
```

1

```
// Get player by ID
function getPlayer(playerID) {
    const db = getDatabase();
    try {
        const stmt = db.prepare('SELECT * FROM Player WHERE PlayerID = ?');
        const row = stmt.get(playerID);
        db.close();
        return row;
    } catch  err) {
        db.close();
        throw err;
    }
}
```

2

1. The updatePlayerName function runs a SQL UPDATE query to update the display name for a given player

2. The getPlayer function runs a SQL SELECT query to retrieve the player data

Next, I need to create endpoints for interacting with the database/player logic.

The new endpoints are seen below:

```
app.get("/stats", (req, res) => { // Endpoint to retrieve game statistics
    try {
        const { playerID } = req.query;
        if  !playerID) {
          ( return res.status(400).json({ error: "Player ID is missing" });
        }
        const stats = getStats(parseInt(playerID)); // Retrieve stats using the function in stats.js
        res.json(stats);
    } catch  error) {
        con$ole.error('Error fetching stats:', error);
        res.status(500).json({ error: 'Internal server error' });
    }
});
```

1

```
// Player management endpoints
app.post("/create-player", (req, res) => {
    try {
        const { displayName } = req.body;
        if  !displayName) {
          ( return res.status(400).json({ error: "Display name is required" });
        }
        const playerID = createPlayer(displayName);
        res.json({ playerID, displayName });
    } catch  error) {
        con$ole.error('Error creating player:', error);
        res.status(400).json({ error: 'Invalid display name.' });
    }
});
```

2

```
app.put("/update-player-name", (req, res) => {
    try {
        const { playerID, displayName } = req.body;
        if  !playerID || !displayName) {
          ( return res.status(400).json({ error: "Player ID and display name are required" });
        }
        updatePlayerName(parseInt(playerID), displayName);
        res.json({ success: true, displayName });
    } catch  error) {
        con$ole.error('Error updating player name:', error);
        res.status(400).json({ error: 'Invalid display name.' });
    }
});
```

3

```
app.get("/player", (req, res) => {
    try {
        const { playerID } = req.query;
        if  !playerID) {
          ( return res.status(400).json({ error: "Player ID is missing" });
        }
        const player = getPlayer(parseInt(playerID));
        if  !player) {
          ( return res.status(404).json({ error: "Player not found" });
        }
        res.json(player);
    } catch  error) {
        con$ole.error('Error fetching player:', error);
        res.status(500).json({ error: 'Internal server error' });
    }
});
```

4

1. The existing /stats endpoint has been updated to retrieve the data from the SQLite database instead of the JSON file.
2. A new /create-player endpoint has been created which interacts with the createPlayer() function.
3. A new /update-player-name endpoint has been created which interacts with the updatePlayerName() function
4. A new /player endpoint has been created to retrieve the player with the getPlayer() function.

The next thing to do is to create the frontend HTML and JS for this.

I first created the HTML content for the player related elements.

```
        <div class="player-info">
            <label>
                Player Name:
                <input type="text" id="playerNameInput" placeholder="Enter your name" minlength="3" maxlength="50">
            </label>
            <button id="updateNameBtn">Update Name</button>
            <span id="playerNameStatus"></span>
        </div>
```

I chose to add the minlength and maxlength validation for the following reasons:

- Minlength validation to reduce the likelihood of users accidently updating their name with random text
- Maxlength to prevent potential errors from dramatically large data being inputted

Next, I created the JavaScript for interacting with the new endpoints.

```
document.getElementById("updateNameBtn").addEventListener("click", updatePlayerName);

// Initialise player when the page loads
window.addEventListener("DOMContentLoaded", async () => {
    await initialisePlayer();
});

// Initialise or load player
async function initialisePlayer() {
    const storedPlayerID = localStorage.getItem("playerID");
    if  storedPlayerID) { // Load an existing player
      ( try {
            const res = await fetch(`/api/game/player?playerID=${storedPlayerID}`);
            if  res.ok) {
              ( const player = await res.json();
                currentPlayerID = player.PlayerID;
                document.getElementById("playerNameInput").value = player.DisplayName;
                document.getElementById("playerNameStatus").textContent = `${player.DisplayName} loaded`;
                return;
            }
        } catch  error) {
            con$ole.error("Error loading player:", error);
        }
    }

    // If the user hasnt played before, create a new guest player
    await createNewPlayer("Player" + Math.floor(Math.random() * 1000)); // Use a 3 digit random number to reduce chances of clashes
}
```

1

2

1. When the page first loads, the initialisePlayer function is run.

2. The initalisePlayer function retrieves the playerID from the user's local storage. If it exists (user has played before), it retrieves the user data from the /player endpoint, and updates the HTML content with the data and ends there. If the user hasn't played before, it instead runs the createNewPlayer() function.

```
// Create a new player
async function createNewPlayer(displayName) {
    try {
        const res = await fetch("/api/game/create-player", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({displayName})
        });

        const data = await res.json();
        if  res.ok) {
          ( currentPlayerID = data.playerID;
            localStorage.setItem("playerID", currentPlayerID);
            document.getElementById("playerNameInput").value = data.displayName;
            document.getElementById("playerNameStatus").textContent = `Player created: ${data.displayName}`;
        } else {
            document.getElementById("playerNameStatus").textContent = data.error || "Error creating player";
        }
    } catch  error) {
        con$ole.error("Error creating player:", error);
        document.getElementById("playerNameStatus").textContent = "Error creating player";
    }
}
```

1

```
// Update player name
async function updatePlayerName() {
    const newName = document.getElementById("playerNameInput").value.trim();

    if  !newName || newName.length < 3 || newName.length > 50) {
      ( document.getElementById("playerNameStatus").textContent = "Name invalid";
        return;
    }

    try {
        const res = await fetch("/api/game/update-player-name", {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ playerID: currentPlayerID, displayName: newName })
        });

        const data = await res.json();
        if  res.ok) {
          ( document.getElementById("playerNameStatus").textContent = `Name updated to: ${data.displayName}`;
        } else {
            document.getElementById("playerNameStatus").textContent = data.error || "Error updating name";
        }
    } catch  error) {
        con$ole.error("Error updating player name:", error);
        document.getElementById("playerNameStatus").textContent = "Error updating name";
    }
}
```

2

1. The createNewPlayer function makes a POST request to the /create-player endpoint to create a new player. It then populates the HTML elements with the returned player data.

2. The updatePlayerName function will retrieve the new player name from the playerNameInput element. It will validate its length to reduce unnecessary API calls. It then makes a PUT request to the /update-player-name endpoint to update the player's name. It then populates the HTML elements with the data, or shows an error to the user if it failed.

Beyond these changes, I had to add small changes to other parts of code.

For example, adding a playerID attribute to the Game class:



```
class Game {
    constructor(difficulty = "easy", playerID = null) {
        this.gridSize = 10;
        this.difficulty = difficulty;
        this.playerBoard = new Board(); // Player's board
        this.aiBoard = new Board(); // AI's board
        this.isPlayerTurn = true;
        this.gameOver = false;
        this.winner = null;
        this.startedAt = new Date();
        this.playerID = playerID; // Store player ID for keeping stats separate

        this.placeAIShips();
    }
```

Then also implementing playerID validation and functionality into other functions. For example:



```
app.post('/start-game', (req, res) => {
    try {
        console.log(req.body.difficulty)
        const difficulty = req.body.difficulty || 'easy'; // Use easy as a default difficulty incase none is specified
        const playerID = req.body.playerID; // Get playerID from request

        if (!playerID) {
            return res.status(400).json({ error: "Player ID is required" });
        }

        console.log(difficulty)
        console.log(`Is valid: ${validateDifficultyString(difficulty)}`)
        if (!validateDifficultyString(difficulty)) {
            return res.status(400).json({ error: "Invalid difficulty level" });
        }
        const gameId = Date.now().toString(); // Unique ID based on current timestamp. Not suitable long-term, but will wor

        games[gameId] = new Game(difficulty, playerID);
        return res.json({ gameId, message: 'Game started' });
    } catch (error) {
        console.error('Error starting game:', error);
        return res.status(500).json({ error: 'Internal server error' });
    }
});
```

In the above code, I added validation to check the request body for the playerID, then pass the playerID into the new Game instance.

Now that this part of the solution is complete, I should test it to ensure it is fully functional.

Test Number	What's being tested	Expected Outcome	Actual Outcome	Passes Test
PLR-1	The player is able to set a name	Name updates correctly	Name updates correctly	Yes
PLR-2	The player is able to update an existing name	Name updates correctly	Name updates correctly	Yes
PLR-3	User data persistance	The player can close the page, return, and the user data persists	Data persists	Yes
PLR-4	The statistics page correctly populates data	Correct data is populated with no errors	Correct data is populated with no errors	Yes

The name changing logic can be seen demonstrated in this video:

Testing video name: "change name test.mov"

Stakeholder Feedback

- Do you find the name feature useful?
  "I don't really care much about setting and changing a name but it's useful because it means my statistics are split up from other people"

From the feedback, I have identified that the feature is useful because it makes the data meaningful by splitting it up by player.

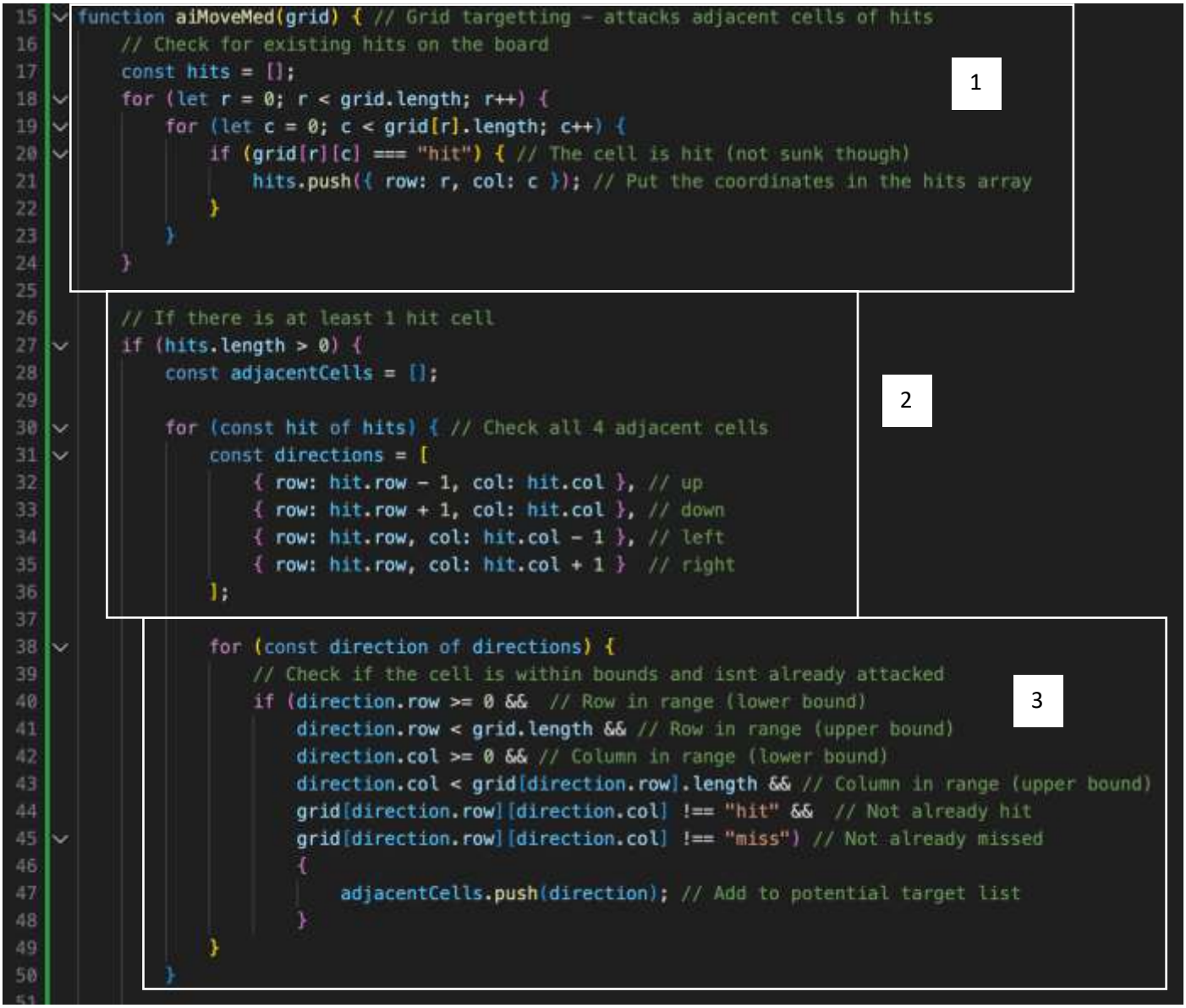
Implementing a more challenging opponent

Currently, the solution works as expected, but the opponent uses random targeting. Due to the modularity of the solution, additional difficulty options could be easily implemented into the solution. However, to meet success criteria 3 and 6, I should implement at least 1 more difficulty option. Due to time constraints, I will prioritise a rule based attacking method rather than using machine learning. This may be implemented at a later stage, or as something a future maintainer may choose to implement, as it can easily be integrated into the solution logic.

In order to create such logic, I will need to research how rule-based strategies work and how I should create it.

After researching how other targeting systems work from StackOverflow, I can now create a similar algorithm adapted to this use case.

To do this, I can simply create a new module function in ai.js and then change some other code to allow multiple difficulty modes.




```
function aiMoveMed(grid) { // Grid targetting - attacks adjacent cells of hits
    // Check for existing hits on the board
    const hits = [];
    for (let r = 0; r < grid.length; r++) {
        for (let c = 0; c < grid[r].length; c++) {
            if (grid[r][c] === "hit") { // The cell is hit (not sunk though)
                hits.push({ row: r, col: c }); // Put the coordinates in the hits array
            }
        }
    }

    // If there is at least 1 hit cell
    if (hits.length > 0) {
        const adjacentCells = [];

        for (const hit of hits) { // Check all 4 adjacent cells
            const directions = [
                { row: hit.row - 1, col: hit.col }, // up
                { row: hit.row + 1, col: hit.col }, // down
                { row: hit.row, col: hit.col - 1 }, // left
                { row: hit.row, col: hit.col + 1 }  // right
            ];

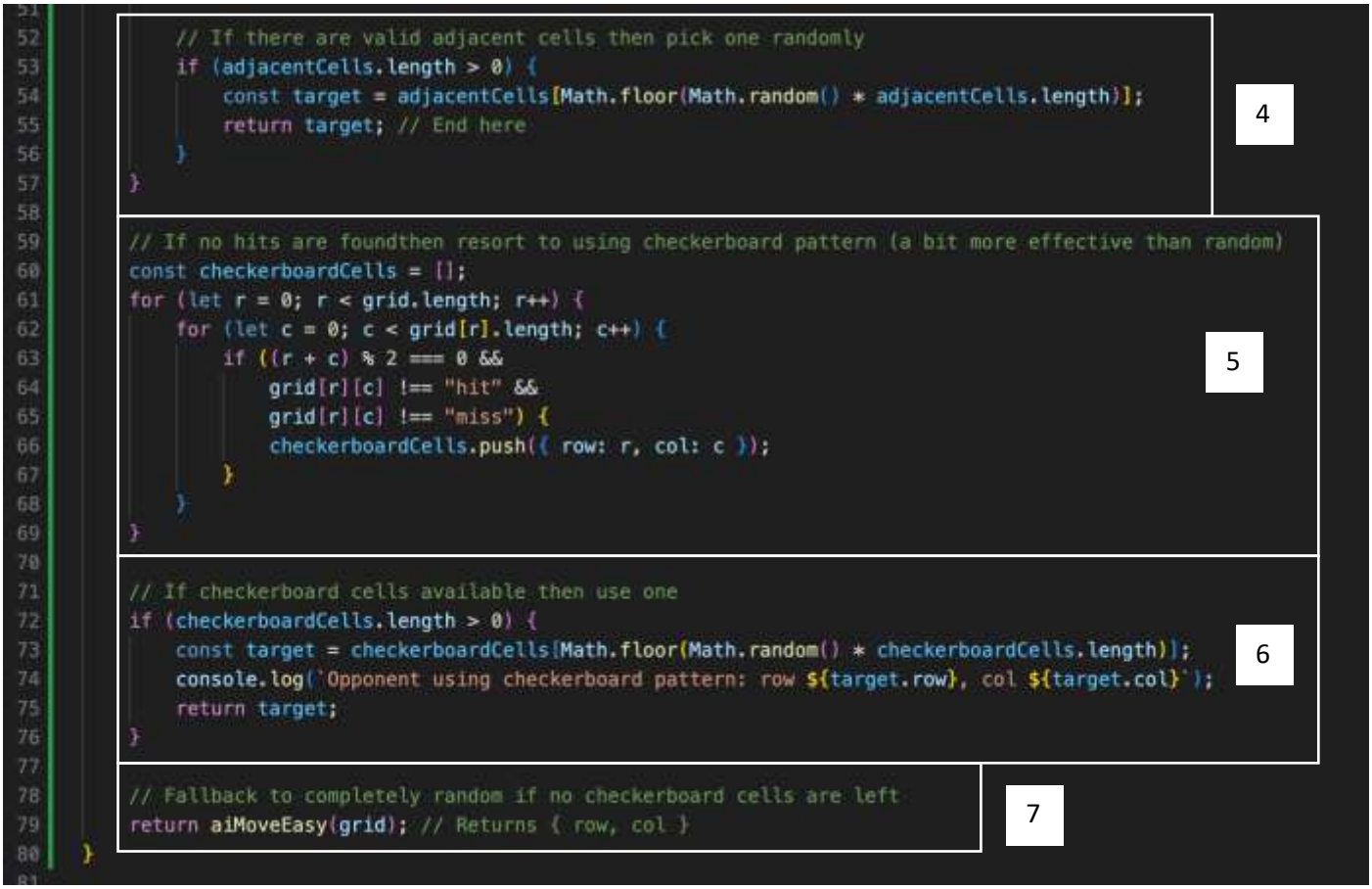
            for (const direction of directions) {
                // Check if the cell is within bounds and isnt already attacked
                if (direction.row >= 0 &&  // Row in range (lower bound)
                    direction.row < grid.length && // Row in range (upper bound)
                    direction.col >= 0 && // Column in range (lower bound)
                    direction.col < grid[direction.row].length && // Column in range (upper bound)
                    grid[direction.row][direction.col] !== "hit" &&  // Not already hit
                    grid[direction.row][direction.col] !== "miss") // Not already missed
                    {
                        adjacentCells.push(direction); // Add to potential target list
                    }
            }
        }
```

This function can be split up into logical sections:

1. The function first starts by checking all existing hits on the player's board and adds them to a hits array to easily access the hit coordinates later. I had to research how to create this loop on StackOverflow as I was unfamiliar with recursive loops

2. If there is at least 1 hit co-ordinate then it goes into the adjacent checks. This starts by iterating through all the hits and collecting the information about all 4 squares around them

3. Each one of the adjacent cells for each hit is checked to see if it's a potential target to attack. It checks if the cell is within the grid bounds and hasn't already been attacked. If it passes these checks then it adds it to a new array to access later.



4. After all the hit cells and adjacent cells are checked, the algorithm chooses a random one from the adjacentCells array to attack. It first validates that there is at least 1 to prevent logic errors.

5. If there are either no hit co-ordinates, or there are no valid adjacent cells, it falls back to using a checkerboard system. I found this idea and method from StackOverflow from a topic on a similar system. This checkerboard system will choose a random cell in a checkerboard pattern rather than entirely random, which is slightly more effective than entirely random. This will also prevent errors if there are no valid adjacent hits to choose.

6. If there is at least 1 valid cell to attack (should be until very near the end of the game), it will choose a random candidate cell to attack. For now, it has a console log to help with debugging.

7. If this checkerboard system doesn't have any valid candidate cells then it will fallback to using the random targeting mode. It can reuse the existing function to save on unnecessary lines and improve maintainability.

The next thing to do is connect this logic up to be used elsewhere. Only a few small changes were necessary. The first of which is to call a different aiMove algorithm depending on the difficulty attribute of the game.

```
113     // Function to process AI moves
114     aiMove() {
115         if (this.gameOver) return { result: "Game already over" }; // Make sure AI doesn't move if game is over
116
117         const difficulty = this.difficulty;
118
119         let move;
120         if(difficulty === "easy") {
121             move = aiMoveEasy(this.playerBoard.grid); // Returns a row and column
122         } else if (difficulty === "medium") {
123             move = aiMoveMed(this.playerBoard.grid);
124         }
125         // Move shouldnt ever be undefined so a fallback isnt necessary
```

1

1. In the AIMove function, instead of calling the aiMoveEasy function regardless of chosen difficulty, it instead runs the relevant algorithm depending on the game's difficulty attribute. No fallback is used because the validation in the parent function already rejects any invalid difficulties.

Finally, the frontend needs to be changed to allow the user to choose a difficulty, allowing it to meet success criteria 3 and 6.

First, I added a difficulty selector in index.html

```
31
32              <button id="startGameBtn">Start Game</button>
33              <label>
34                  Difficulty:
35                  <select id="difficultySelect">
36                      <option value="easy">Easy</option>
37                      <option value="medium">Medium</option>
38                  </select>
39              </label>
```

The last thing to do is retrieve that chosen value and pass that into the API request.

```
async function startGame() {
    if (!currentPlayerID) {
        statusEl.textContent = "Error: No player is set";
        return;
    }

    const difficulty = document.getElementById("difficultySelect").value.toLowerCase();

    const res = await fetch("/api/game/start-game", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ difficulty: difficulty, playerID: currentPlayerID })
    });

    const data = await res.json();
    gameId = data.gameId;
    statusEl.textContent = "Game started";
    gameOver = false;
    await updateGameState();
}
```

1
2

1. The HTML value is queried using its ID. The value is retrieved, and converted to lowercase because the backend expects lowercase difficulty values.
2. The POST request is then modified to send the difficulty in the JSON body of the request.

Test Number	What's being tested	Input	Expected Outcome	Actual Outcome	Passes Test
MD-1	Difficulty selector on frontend	Choosing moderate/easy difficulty	User can select a difficulty without problems	User can select a difficulty without problems	Yes
MD-2	The correct opponent algorithm is used	Choosing moderate difficulty	Moderate difficulty opponent is used	Moderate difficulty opponent is used (proved with console output)	Yes
MD-3	Core game functionality	Choosing moderate difficulty	The game works as normal and ends when player/AI wins	The game works as normal and ends when player/AI wins	Yes
MD-4	Overall algorithm correctness	Choosing moderate difficulty	Opponent never attacks invalid grids	Opponent sometimes tries to attack cells containing sunk ships	No

Gameplay demonstration video name: "med difficulty testing.mov"

While I hadn't noticed any problems with the algorithm until now, test 4 reveals a critical problem with the algorithm. After debugging the problem, I realised the root cause of the problem. The algorithm checks for cells that have either been misses (state 2) or hits (state 3), but not for sunk ships (state 4). This was causing problems with the algorithm trying to attack an already attacked co-ordinate.

Fixing this was a simple update of adding checks for the sunk state, as seen in the below example:

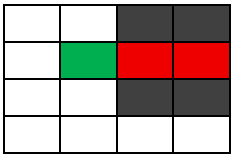
```
64    // If no hits are foundthen resort to using checkerboard pattern (a bit more effective than random)
65    const checkerboardCells = [];
66    for (let r = 0; r < grid.length; r++) {
67        for (let c = 0; c < grid[r].length; c++) {
68            if ((r + c) % 2 === 0 &&
69                grid[r][c] !== 3 &&
70                grid[r][c] !== 4 &&
71                grid[r][c] !== 2) {
72                checkerboardCells.push({ row: r, col: c });
73            }
74        }
75    }
76
```

After making this change, from the testing, I can conclude that the moderate difficulty opponent works correctly and doesn't cause issues with other parts of the game.

However, there is one small change I plan to make to the moderate difficulty opponent to make it behave more human-like, helping to meet success criteria 6 of using a proper strategy, and meeting the requirements from the interview.

This change is making the algorithm not attack cells that are logically impossible to be part of the ship. This is when the ship's orientation reveals which cells it can and can't be situated in.

This can be demonstrated in a simple table to help me think about it logically by seeing the problem in a visual form.



Thinking logically, if the red cells indicate ones that have been attacked at and hit a ship, once more than 1 consecutive cell has been hit, various cells are ruled out (the grey cells in the visualisation). This is because the orientation of the ship is known, so attacking cells vertically is not how a human would behave, as a human is more intelligent, which I am trying to achieve with this opponent: semi-humanlike-looking intelligence. In the above visualisation, the only cell that can logically contain the ship is the green cell.

To make the moderate difficulty opponent behave more like this, I will need to make some changes to make the algorithm keep track of the orientation.

After making these changes, this is the changed code within the aiMoveMed algorithm:

```
for (const hit of hits) {
    // Detect ship orientation by checking for adjacent hits
    let orientation = null; // Unknown as of now

    // Check for hit above or below (vertical orientation)
    const hasVerticalHit = hits.some(h => h.col === hit.col && Math.abs(h.row - hit.row) === 1);
    // That checks if any of the hits in the hits array share the same column and are one row different

    // Check for hit left or right (horizontal orientation)
    const hasHorizontalHit = hits.some(h => h.row === hit.row && Math.abs(h.col - hit.col) === 1);
    // That checks if any of the hits in the hits array share the same row and are one column different

    if (hasVerticalHit) {
        orientation = 'V';
    } else if (hasHorizontalHit) {
        orientation = 'H';
    } // Otherwise the orientation is still unknown

    const directions = [
        { row: hit.row - 1, col: hit.col, orientationType: 'V' }, // up
        { row: hit.row + 1, col: hit.col, orientationType: 'V' }, // down
        { row: hit.row, col: hit.col - 1, orientationType: 'H' }, // left
        { row: hit.row, col: hit.col + 1, orientationType: 'H' }  // right
    ];

    for (const direction of directions) {
        if (orientation && direction.orientationType !== orientation) {
            continue; // Skip this direction if orientation is known and doesnt match orientationType
        }
```

This code can be broken down into these sections:

1. First a new orientation variable is defined with the value null, indicating the default state of an unknown ship orientation.

2. The hits array (containing all hit cells) is checked to see if any of them are on the same column as the current hit and 1 row up or down. This would indicate that the ship has vertical orientation

3. The same logic is used for checking for horizontal orientation.

4. If the vertical condition is met, the orientation variable is set to “V”, and “H” for the horizontal condition. If neither of the conditions are met, it will keep the orientation variable as null, indicating it’s unknown.

5. The directions array has been updated so that each object within it now contains an orientationType to identify what orientation each direction would imply.

6. In the directions for loop, the orientation is validated first. If the orientation of the ship is known, and the current direction’s implied orientation is not the same orientation then that direction is skipped to avoid illogical attacks.

After making these changes, the algorithm now behaves a lot more intelligently, helping to more closely meet the success criteria for the solution.

Progress Review

At this point, the moderate difficulty works as expected and feels a lot more intelligent than the easy opponent.

While this moderate opponent is not an actually intelligent machine learning model, it still behaves in an intelligent-looking way and did not consume too much development time. In the future, either I or a future maintainer could easily integrate a ML model difficulty mode into the solution as the modularity allows new difficulties to be introduced with only minimal changes to the code.

Stakeholder Feedback

- Do you like the new moderate difficulty opponent?
  "Yes it's a lot more difficult to play against than the easy opponent because it actually feels like it has a strategy"

- Would you make any changes to the moderate difficulty opponent?
  "No it works well as a moderate difficulty. An even harder opponent would be cool though"

From the feedback, I have identified that the moderate difficulty opponent implementation is suitable, and a hard difficulty opponent would be desired, although this is unlikely to be implemented due to time constraints. I could have focused on a hard difficulty opponent before the moderate difficulty opponent, but it may have been too time consuming, potentially leading to neither being developed.

Evaluation

Testing for evaluation

Game start

Test Number	Requirement	Input	Expected Outcome	Actual Outcome	Passes Test
PD-1	The player can click a button to start the game	N/A	Game starts and both grids appear	Game starts and both grids appear	Yes
PD-2	The player can choose a difficulty which correctly updates the game difficulty	N/A	The difficulty is set to the chosen difficulty	The difficulty is set to the chosen difficulty	Yes

Testing video name: “eval start game testing.mov”

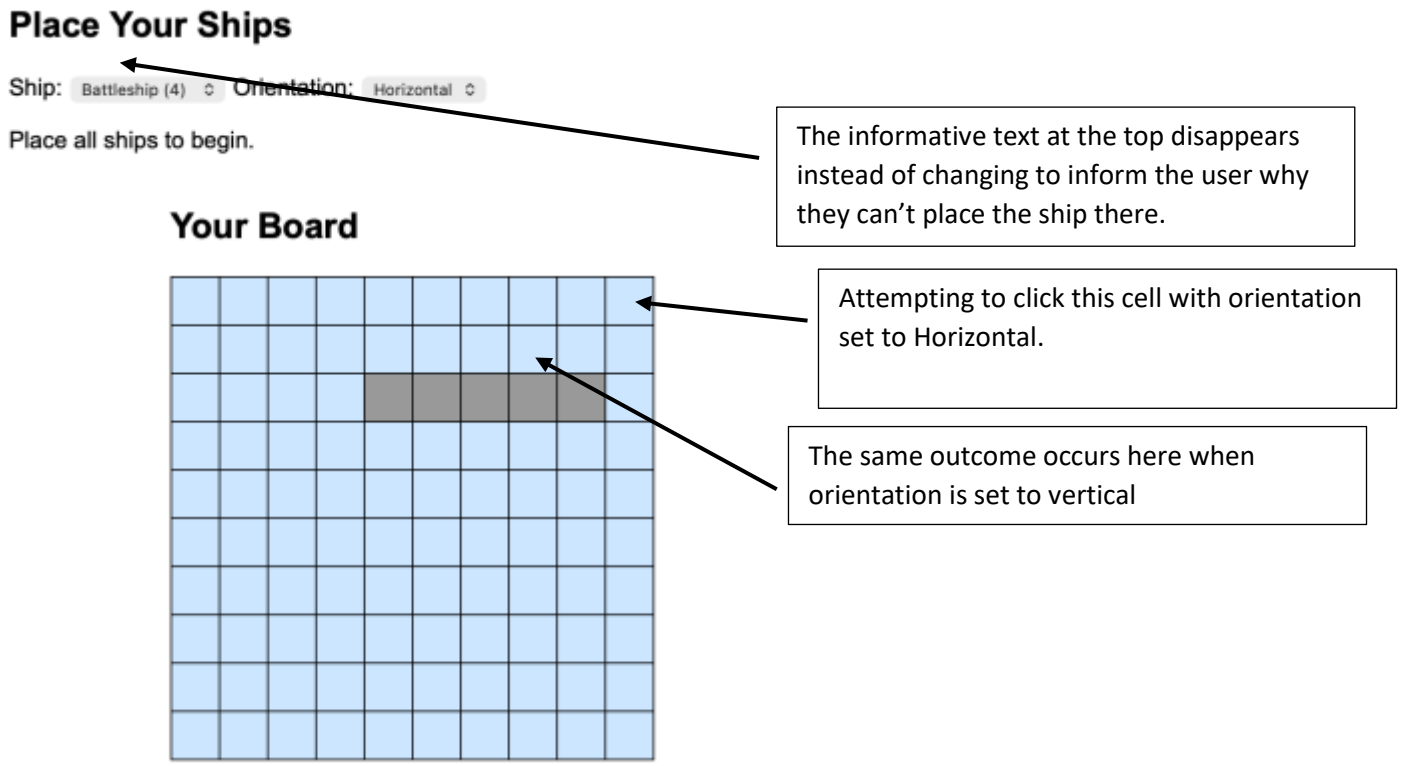
The video above demonstrates that the game starts by simply clicking the start game button, and the difficulty can be changed using the dropdown option. Once the start game button is clicked, the grids appear and the user can start placing ships. This works on both desktop and mobile devices. The tests reveal that it works every time, demonstrating that it is robust enough for public use. This allows it to better meet success criteria 7 and 11.

Stakeholder Feedback

- Is it obvious how to start the game and is there anything you would improve about this part of the solution?
  “It’s easy to start the game and set the difficulty, but I would want an easier way to change the difficulty than clicking onto a dropdown menu”

Ship Placement phase

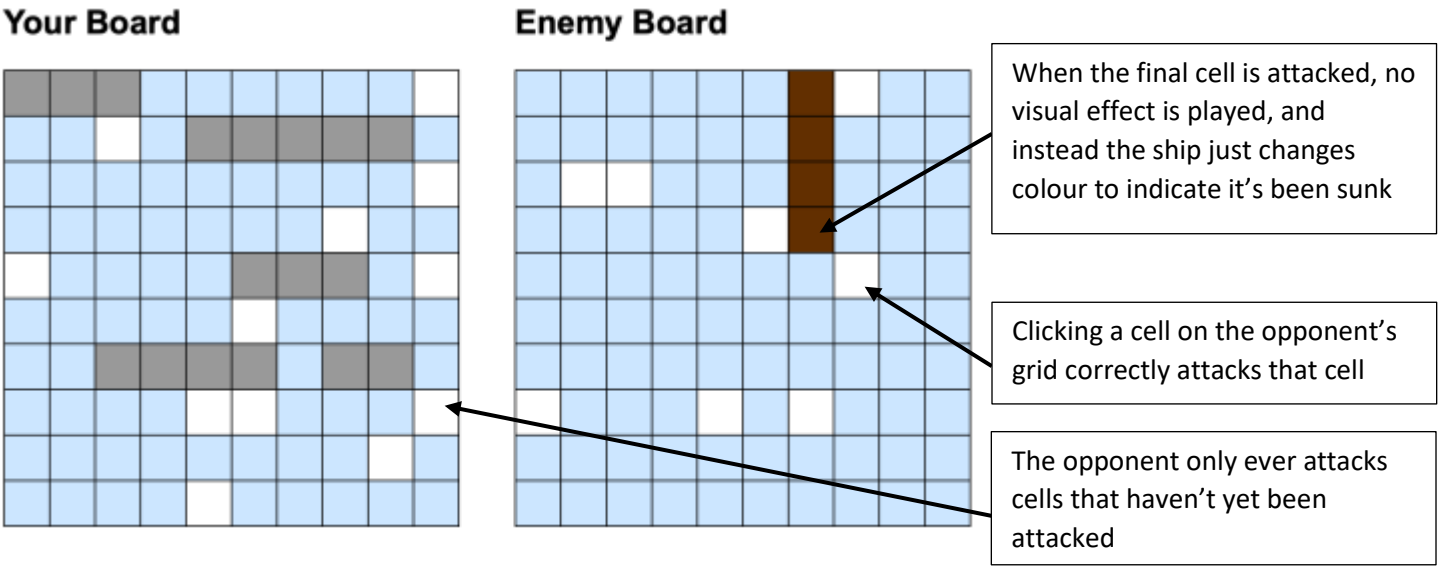
Test Number	Requirement	Input	Expected Outcome	Actual Outcome	Passes Test
PD-3	The player cannot place a ship that overlaps another ship	Invalid ship placement (overlapping)	Ship does not get placed, and the user is informed why	Ship does not get placed, but the user is not informed why	Partially
PD-4	The player can place all their ships (in valid placements) on the grid	User selects the ships and the starting cell, and chooses the orientation of the ship	The ships appear correctly in the desired coordinates without any invalid placement	The ships appear correctly in the desired coordinates without any invalid placement	Yes
PD-5	The player cannot place ships outside of the grid boundaries	Invalid ship placement (out of bounds)	Ship does not get placed, and the user is informed why	Ship does not get placed, but the user is not informed why	Partially
PD-6	The player cannot place the same ship multiple times	Attempting to place a ship in 2 different co-ordinates	The ship is placed only once	The ship is placed only once	Yes



The validation correctly prevents the player placing a ship in an invalid cell, but it however is slightly unclear what the reason is. This could be later improved to better meet the success criteria of clear instructions for playing the game (success criteria 10). The testing determines that the solution is robust, as it handles invalid inputs without causing unexpected behaviour. These tests allow it to meet success criteria 11, 13, and 15.

Attacking phase

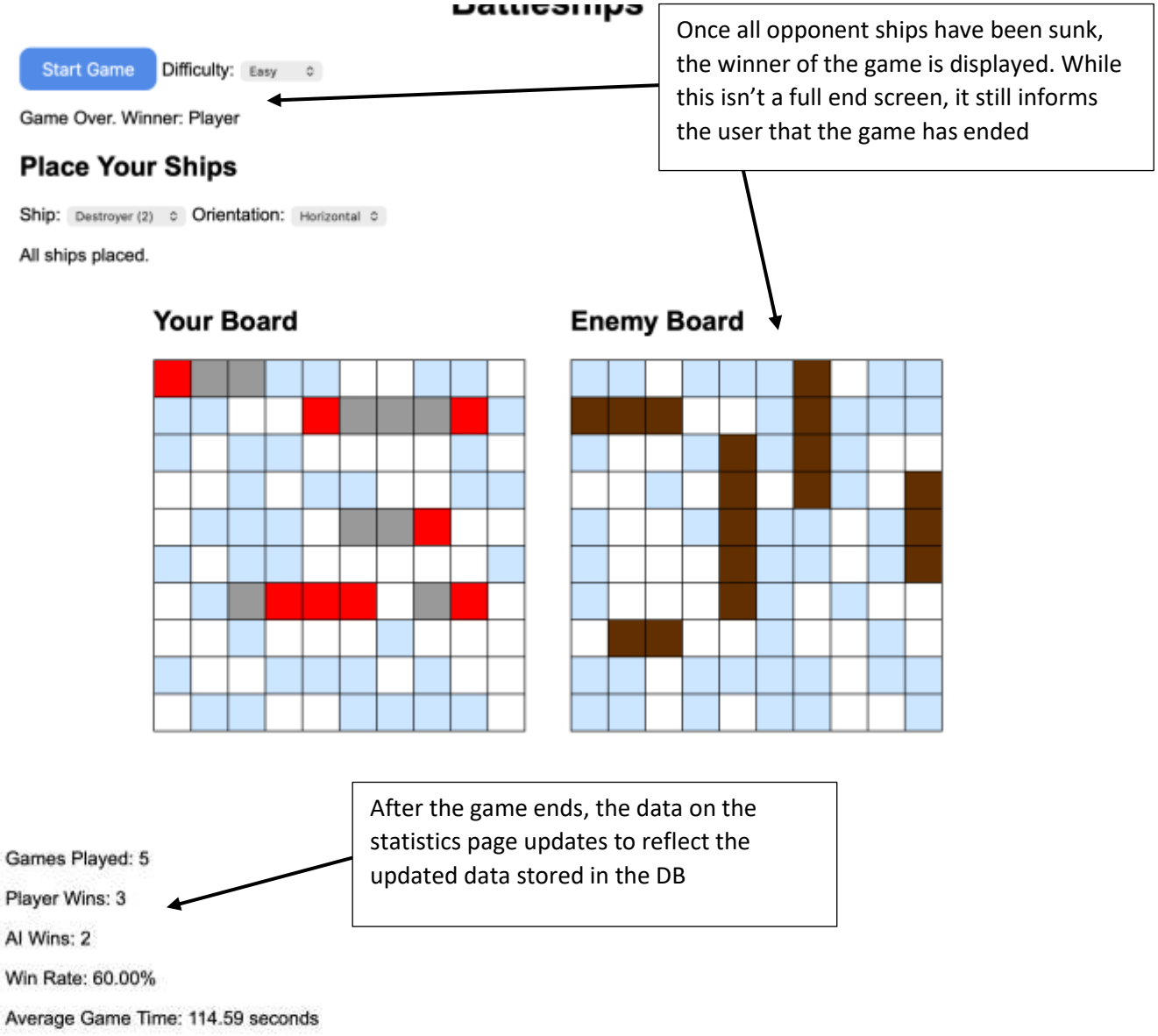
Test Number	Requirement	Input	Expected Outcome	Actual Outcome	Passes Test
PD-7	The player can choose a coordinate to attack	Selecting a grid cell	Cell status changes to hit or miss, and feedback is provided visually	Cell status changes to hit or miss, and feedback is provided visually	Yes
PD-8	The AI opponent can make a valid attack	N/A	AI makes an attack on a coordinate that has not yet been attacked	AI makes an attack on a coordinate that has not yet been attacked	Yes
PD-9	The AI opponent always attacks after the player's turn (except once game ends)	N/A	AI always makes an attack	AI always makes an attack	Yes
PD-10	Hits and misses get tracked correctly	Player or AI attacks a grid cell	Visual representation of hits and misses, and does not discard any statuses	Visual representation of hits and misses, and does not discard any statuses	Yes
PD-11	Player cannot attack invalid grid cells	Player clicks a grid cell that has already been attacked	Grid cell is not attacked again, and instead visual feedback is provided to the user. Game does not error.	Grid cell is not attacked again, but no visual feedback is provided to the user. Game does not error.	Partially
PD-12	Player cannot take two turns consecutively	User clicks a grid cell twice in quick succession	Only the first click is used for the turn. The following click(s) are discarded	Only the first click is used for the turn. The following click(s) are discarded	Yes
PD-13	Visual effects when a ship is sunk	Player attacks the final cell of a ship	A visual effect is displayed	Not implemented	No



These tests reveal that the attacking phase of the solution is fully functional and robust. User inputs work correctly and invalid inputs are handled correctly. Usability however could be improved due to the lack of visual effects (revealed in test PD-13) and the lack of feedback to the user in test PD-11. These tests allow it to meet success criteria 11 and 13.

Game ended phase

Test Number	Requirement	Input	Expected Outcome	Actual Outcome	Passes Test
PD-14	The game detects when all the ships of the player or AI are sunk	Player or AI attacks and sinks the final ship of the opponent	Game ends and end-of-game results are shown to the user.	Game ends and winner is shown to the user.	Yes
PD-15	Statistics update after game ends	N/A	Winner and other statistics are stored in the DB	Winner and other statistics are stored in the DB	Yes



Once all opponent ships have been sunk, the winner of the game is displayed. While this isn't a full end screen, it still informs the user that the game has ended

After the game ends, the data on the statistics page updates to reflect the updated data stored in the DB

These tests reveal that the game functions correctly when the game ends, without any unexpected behaviour or edge cases.

Other tests

Test Number	Requirement	Input	Expected Outcome	Actual Outcome	Passes Test
PD-16	Navigation menu functions correctly	User clicks a navigation option	Correct page is displayed	Correct page is displayed	Yes
PD-17	Game does not run forever	Player closes the game tab in browser	Game times out after a specific period of time with no user requests	Game is stored in memory indefinitely	No
PD-18	Player can set and update their name	Player enters a name and clicks “Update Name”	Name is updated correctly	Name is updated correctly	Yes
PD-19	Games start and end with no crashing or unexpected behaviour	N/A	No crashes or unexpected behaviour	No crashes or unexpected behaviour	Yes

Navigation demonstration video name: “navigation demo.mov”

The above video demonstrates that the navigation menu functions correctly. This helps to improve usability as non-functional navigational menu items would confuse users.

The player’s name modification tests produce the same outcomes as tests PLR-1, PLR-2, PLR-3, and PLR-4 carried out during development.

Test PD-17 reveals a memory leak, which could be addressed if the solution continued to be maintained. Currently though it isn’t a priority.

Test PD-19 reveals that the game performs well and is robust as it has no unexpected behaviour.

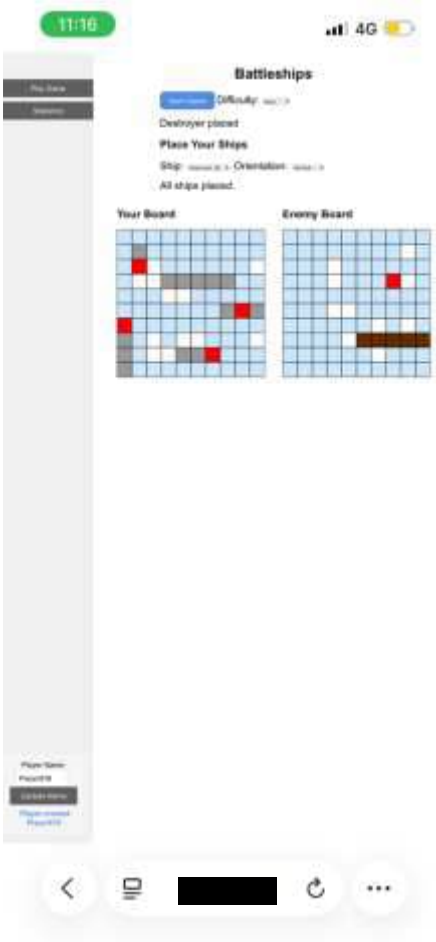
Stakeholder Feedback

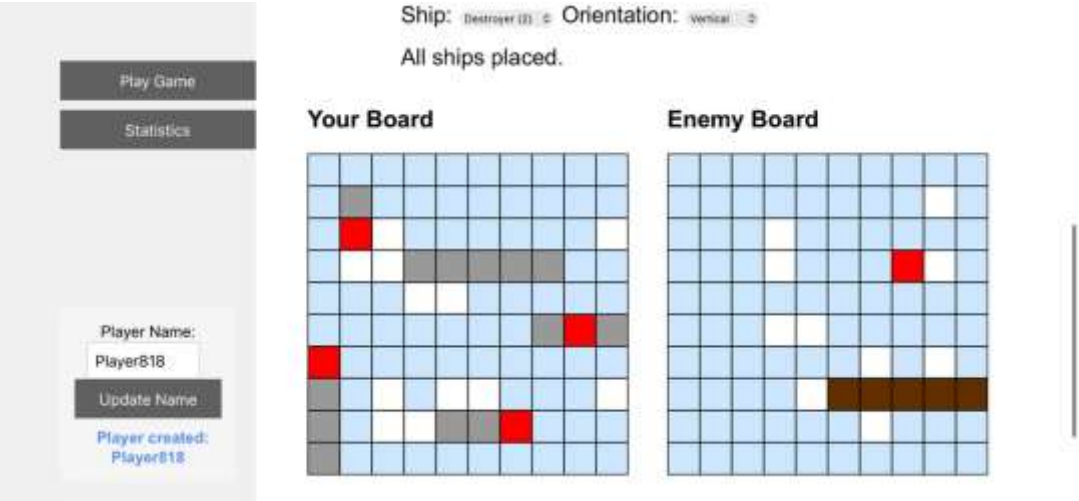
- Did you experience any poor performance or bugs when testing?
  “Not really. It lagged sometimes but I didn’t see any bugs”

Robustness tests

Test Number	Requirement	Input	Expected Outcome	Actual Outcome	Passes Test
PD-20	Works on multiple platforms	Testing on a mobile device and desktop device	Performance and usability is the same on both	Performance and usability is the same on both	Yes
PD-21	Game does not run forever	Player closes the game tab in browser	Game times out after a specific period of time with no user requests	Game is stored in memory indefinitely	No
PD-22	Player can set and update their name	Player enters a name and clicks “Update Name”	Name is updated correctly	Name is updated correctly	Yes
PD-23	Games start and end with no crashing or unexpected behaviour	N/A	No crashes or unexpected behaviour	No crashes or unexpected behaviour	Yes

Mobile layout:





Stakeholder Feedback

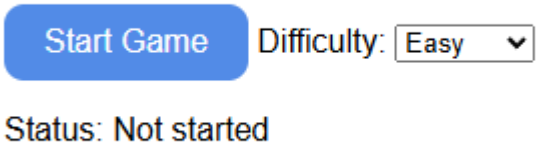
- Did you experience any poor performance or bugs when testing?
  "Not really. It lagged sometimes but I didn't see any bugs"

- Did it slow down after playing a few games?
  "No it was mostly fast the whole time apart from lagging sometimes randomly"

From the feedback, I have identified that the solution seems to be robust as the stakeholder has not experienced any bugs or poor performance when playing the game like a normal player.

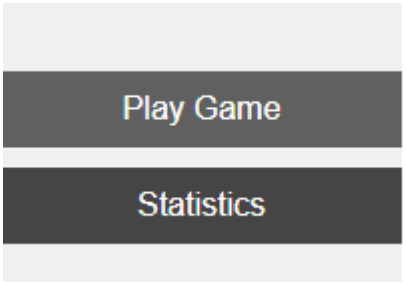
Usability Features

Navigation



By clicking the “Start Game” button, the player can easily start a new game.

The difficulty dropdown provides a clear way for the player to decide on a game difficulty. A dropdown is used instead of other input types because it makes it more accessible to some users.



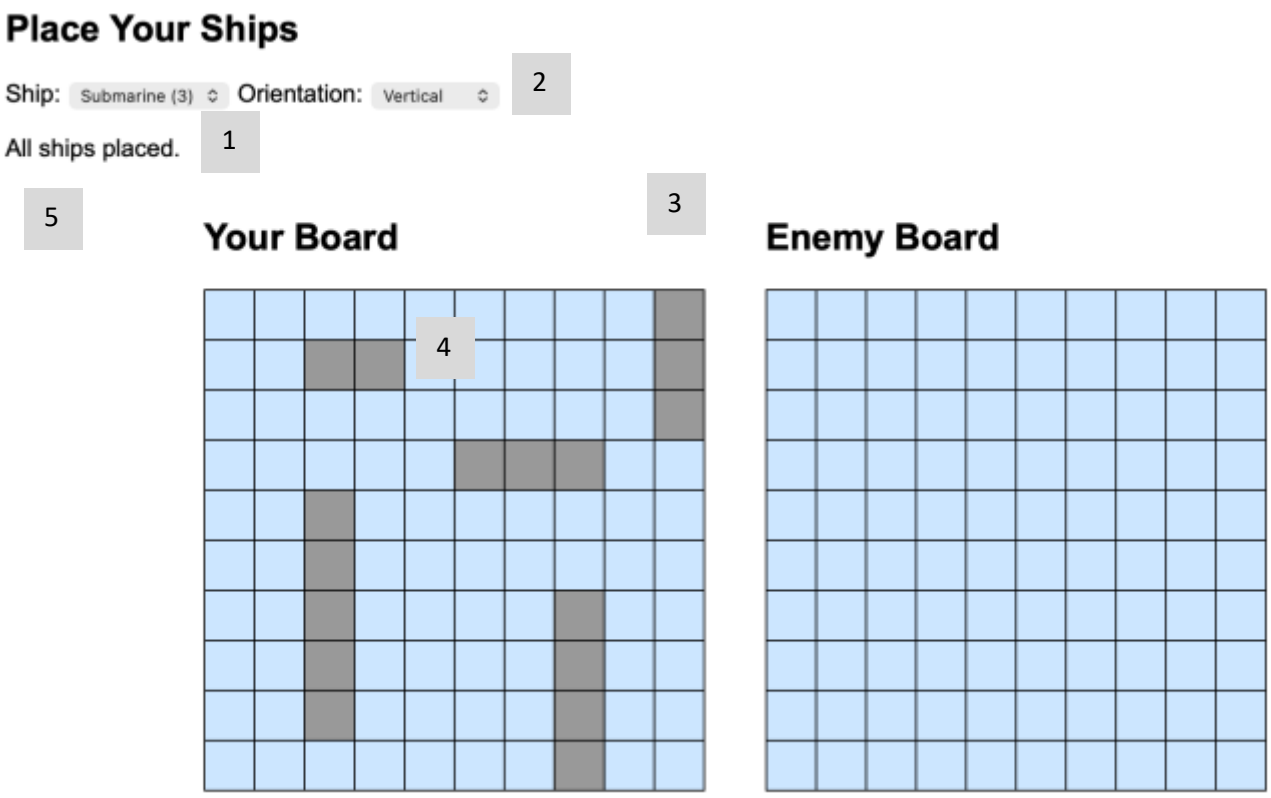
The side navigation bar has clearly labelled pages, allowing the user to easily navigate between the game page and the statistics page, as required by success criteria 1 and 16.

- Is the navigation intuitive enough to use without too much thought?
  “Yes the buttons are easy to understand without anyone explaining them to me and I can see everything on the page clearly”
- Is there anything you’d improve upon for usability in terms of the navigation?
  “Yes I would want keyboard keys to start the game or change difficulty but clicking is fine”
- Did you experience any problems with navigation when testing the solution?
  “No it worked fine the whole time without bugs”

From the feedback, I have identified that the navigation is suitable and meets the success criteria for usability. While changes could be made, such as implementing keyboard controls, this is low priority and could be implemented if the solution continues to be maintained.

The final layout mostly meets the planned wireframe from the design of this solution. It however does not contain the additional instructions at the bottom of the page, but the grid, header, and navigational side bar were implemented as planned in the planned locations.

Screen Elements



1. A ship can be selected using a simple dropdown menu. Each option shows in brackets the size of the ship. A simple clear option selector is used because more complex selection may be less accessible to some users.

2. The orientation of the ship can be chosen using a dropdown containing Vertical and Horizontal

3. Both boards are visible during the ship placement phase. The boards are labelled clearly to help the user identify where they should place their ships.

4. Placed ship cells are shown in grey for clarity instead of using text, as colours are more accessible to people who maybe can't read text

5. Status text shows important information about the game state which the player may not be able to infer from elsewhere in the game

- Is there anything you like or don't like about the ship selector?
  "It's clear which ship is which but it doesn't show me which ships I have already placed which makes it a bit confusing if I place them in a random order"

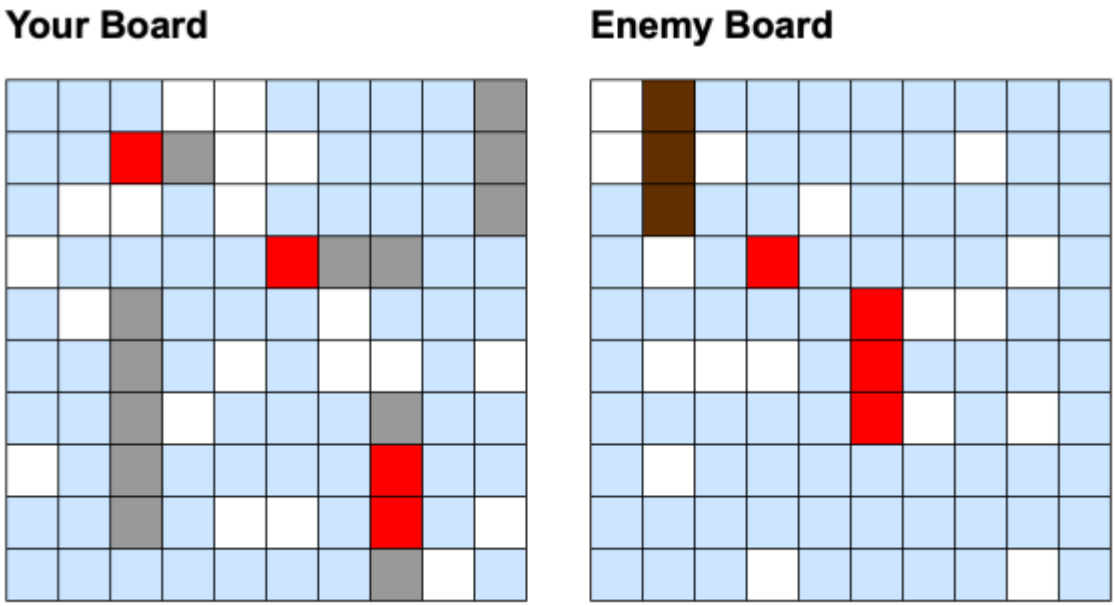
- Is there anything you like or don't like about the orientation selection?
  "It's not confusing but having 2 buttons or a keyboard shortcut would be easier than having to use a dropdown every time to change orientation."

- Is it clear where to place the ships?
  "Yes because it says your board so I know which board to place the ships on. There aren't any hints on the rules so when it doesn't let me place a ship it doesn't say why"

- Did you experience anything unexpected with ship placement while testing?
  "No it worked fine the whole time"

From the feedback, I have identified that the screen elements are suitable and not confusing, meeting the success criteria of a simple layout.

The position of the grids meets the planned layout from the design of this solution, where the grids are laid out side by side.



In the attacking phase, grid cells change colour based on their state. For example, white for miss, red for hit, brown for sunk. This provides a clear way for the user to identify what each grid cell contains, without the need for text. This allows users who don't speak English to understand if they have hit or missed.

- Is it clear when to attack the opponent, and which board to attack?
  "It says enemy board so I know where to attack, but it doesn't tell me to start attacking. It just tells me that I have placed all ships"

- Is it clear which co-ordinates have been missed, hit, or sunk?
  "Yes they change colour to show me what was in the coordinate"

How well does the solution meet the success criteria?

User Interface / User Experience

Criteria Number	Requirement	Explanation	Reference	Criteria Met
1	Easily navigable UI	This ensures stakeholders can play the game without too much confusion, or needing instructions	A feature in existing solutions such as C. Liam Brown's Battleship; Interview on 18/06/2025	Yes
4	Win/loss history page (optional)	Allows players to see how they have improved across multiple games	Interview on 18/06/2025; Project plan	Yes
7	Mobile and desktop compatibility	The game should function correctly on both desktop and mobile without major issues, and should fit on the screen without overlapping elements. However, mobile usability depends on time constraints.	Interview on 18/06/2025; Project plan	Yes
8	Simple layout	The layout should be clean and simple with not too many unnecessary visual elements.	Interview on 18/06/2025	Yes
10	Clear instructions for playing the game	Allows the player to understand what they need to do without prior knowledge of the game	Project plan; Other solutions	Yes
16	Accessible UI	This makes the UI easier to understand and navigate for all users of different abilities	Project plan; Other solutions	Yes
17	Sound effects for different actions	This provides an audible indication of what the player's action has done.	Project plan	No

The above requirements were mostly met with some limitations.

- **Criteria 1:** The UI is easily navigable as there are a small number of important UI elements without too much clutter or unnecessary content. Test PD-15 demonstrates that the navigation menu functions correctly, and PD-1 and PD-2 demonstrate that other parts of the UI are functional.
- **Criteria 4:** A statistics page was implemented which contains clearly labelled data such as win and loss counts. Test PD-14, as well as tests GS-1 to GS-7 reveal that this page is fully functional and the data is managed correctly without problems.
- **Criteria 7:** While the page is usable on mobile, meeting the success criteria of compatibility, it's best suited for desktop. On mobile, the page does not fill the page correctly and requires zooming out. This was demonstrated with test PD-20. This is a less important UI improvement that could be implemented in future if the solution continues to be maintained. The success criteria could be further met by making changes to the UI in order to fix the scaling problems, making it more accessible to mobile users.

- **Criteria 10:** Some clear instructions are visible, however there could be more to implement as some instructions are unclear. Furthermore, responsive instructions could have been implemented, as currently if the user does something wrong, there is a lack of text informing them what to do, such as in tests PD-3 and PD-5, where the status text disappears when the user does something wrong, instead of informing them of their mistake. The success criteria could be more closely met by making small changes to the frontend to update the status text to inform players of what to do.
- **Criteria 16:** The UI is easy to navigate and clear to understand. The buttons are easy to click, making the solution accessible to all users. This was tested all throughout development, with feedback being provided by a potential stakeholder to ensure actual potential users find it accessible. This was tested with tests PD-16 and PD-20.
- **Criteria 17:** No sound effects were implemented due to the low priority of this feature. From the research and interviews, this was not an important feature so other features were prioritised in the limited development time available. This could however easily be implemented with a few small frontend changes to play an mp3 file when an action happens.

Gameplay

Criteria Number	Requirement	Explanation	Reference	Criteria Met
3	Different difficulty levels	Allows players of all levels of experience to play without a disadvantage, keeping it engaging. This isn't a critical feature but from the interview, it appears to be a desired feature.	Interview on 18/06/2025	Yes
5	Easy opponent	An easy opponent should be present for inexperienced players	Project plan; Other solutions	Yes
6	Strategic AI opponent	The AI should use a proper strategy to play, rather than randomly targeting grid squares.	Interview on 18/06/2025; Project plan	Yes
9	Clear visual responses to actions	Players should be able to see the result of their actions, with a visual change and maybe sound effects too.	Interview on 18/06/2025	Yes
12	Game result screen after game ends	This shows the player that the game is over and shows them who won, potentially with additional statistics such as total time, etc.	Project plan	Partially
15	Stable and consistent ship placement system	This makes the game less confusing as the rules don't change unexpectedly	Project plan; Other solutions	Yes

The above requirements were all met to varying extents.

- **Criteria 3:** Easy and moderate difficulty levels were implemented to keep the game engaging. While no hard difficulty mode was implemented, this can easily be implemented if the solution continues to be maintained. Easy difficulty was fully implemented and tested with tests AIM-1 to AIM-7, demonstrating that it is fully working. The moderate difficulty level was later fully implemented and tested with tests MD-1 to MD-4. Both difficulty levels were tested after development with tests PD-8 and PD-9.
- **Criteria 5:** The easy difficulty mode contained a random attacking opponent. This was fully implemented and tested with tests AIM-1 to AIM-7. This mode is useful because it can be used by inexperienced players, meeting the requirements of the project.
- **Criteria 6:** The moderate difficulty mode contained a strategic AI opponent. While it did not use machine learning, it did have a strategy of targeting grid cells adjacent to existing hits. This helps to keep the game more engaging for players. This was fully implemented and tested with tests MD-1 to MD-5, as well as after development with tests PD-8 and PD-9. This mode can be used by more experienced players.
- **Criteria 9:** Attacking opponent grid cells shows a visual response to the action in the form of changing the colour of the cell to indicate what was in the cell. This is clear for all users. This was tested with the frontend tests (FE-1 to FE-7), and post-development test PD-10.

- **Criteria 12:** When the game ends, there is no dedicated results screen, however it does display the winner in large text at the top. This was tested in test FE-7 and post-development test PD-14. It could however be more visible to improve accessibility though. No additional statistics (such as elapsed time) are shown, although they are tracked so could easily be implemented by modifying the frontend code to retrieve the data from the backend and display it as HTML elements.
- **Criteria 15:** From repeated testing during and after development, the ship placement system appears to behave as expected, with no unusual behaviour. This was tested with tests PS-1 to PS-8, FE-1 to FE-4, and PD-3 to PD-6.

Navigation

Criteria Number	Requirement	Explanation	Reference	Criteria Met
13	Using the mouse to select grid squares	This is an easy and intuitive way of interacting with the game	Project plan; Hardware requirements	Yes

The navigation requirement was fully met.

- **Criteria 13:** Grid cells are selected using the mouse cursor. This makes it easy to interact with the game. This was tested in test FE-5, and post development tests PD-4 and PD-7, and has been demonstrated in this gameplay demonstration video: “game state change test.mov”.

Misc

Criteria Number	Requirement	Explanation	Reference	Criteria Met
2	Client and server communicate through HTTP requests	This is an effective way for the client to interact with the backend server in a way that is compatible with as many devices as possible	Project plan	Yes
11	Stable performance	The game should be able to run smoothly without glitching or causing frequent errors. It should also load within 5 seconds.	Project plan; Other solutions	Yes
14	Time limit for games stored in memory	This ensures there are no memory leaks, ensuring fast and stable performance	Project plan	No

Most of these requirements were fully met, while others were not.

- **Criteria 2:** The client and server successfully communicate through HTTP requests, allowing the full functionality of the game. This was tested with tests FE-1 to FE-7, and post-development test PD-20, and is demonstrated in the final gameplay testing video.
- **Criteria 11:** During final testing after development, I experienced no bugs or glitches. Loading times were extremely fast due to the small number of assets required. This has been tested throughout development with tests such as FE-1 to FE-7, as well as all other frontend and backend tests. This was then tested again after development in test PD-23, and is demonstrated in the full gameplay testing video seen further on.
- **Criteria 14:** Game data is stored in memory indefinitely while the server is running, seen by post-development test PD-21, causing a small memory leak which could become a big problem if released for public use. This means this requirement was not met, however it could easily be met by adding a periodic cleanup of games in memory, discarding any that haven't had any new moves or ships placed within a certain time frame (e.g. 2 hours).

Final Gameplay Testing Video

The following video contains full gameplay, demonstrating the performance, robustness, and stability of the solution, demonstrated from the frontend.

Video Name: “full testing video.mp4”

Maintenance

Current and future maintenance

I have not implemented any features into the solution specifically for maintenance due to time constraints. If the solution however was to be distributed to the public, it would be necessary to implement debugging/maintenance features. Furthermore, while the solution has been extensively tested for bugs, further testing would be required to make it suitable for public use.

While the solution is in a working state, more features should be added, such as the features outlined in the success criteria which were not met, in order to make it suitable for public use, as in its current state it's doesn't have as many features as I would hope. A bug reporting feature would also be useful for future maintenance as it would help with finding bugs as users can report bugs when they experience it.

Currently, the solution would be easy to maintain to a certain extent. This is partially due to the code being modular, with clearly defined sections to split up the logic. There are many comments throughout the codebase to explain complex logic, aiding future development and maintenance, as further changes should be relatively easy after reading through the codebase.

In terms of performance, the code could be made to run marginally more efficiently by cutting down on unnecessary code and fixing memory leaks, such as the games currently being held in memory indefinitely.

Limitations and resolutions to them

Below are additional elements of the solution I would hope to add if I continued development of this solution.

Requirement	Justification
Smarter AI opponent	This would make the game more engaging by having a more human-like opponent

In order to achieve this requirement, I would need to need to train a machine learning model on the existing battleships implementation against an existing difficulty mode, and then implement the model into the game as an additional difficulty level. This would help to meet the success criteria of a strategic opponent, but the current implementation works fine for this scale of project. This could however be easily implemented without significant changes being made to the solution.

Requirement	Justification
Advanced graphical effects and animations	This would improve the overall visual experience for the user, making it more entertaining

In order to achieve this requirement, I would create sprite sheets for each animation and effect, and update the grid to display these effects in an animated form. The sound effects would also need to be synchronised with the animations. This is a feature which I would prioritise for public release. This would help to meet success criteria 17 of sound effects for different actions. This could be implemented without too much modification to the solution as the event handlers are already present.

Requirement	Justification
PvP mode	This would make the game engaging as players could play against each other

To achieve this requirement, I would need to expand the game system to handle requests from multiple clients to the same game. This would require a large amount of further development and reprogramming, and would be quite complex.

Requirement	Justification
Full mobile support	This would make the game more accessible to mobile users

In order to achieve this requirement, I would modify the style to make the UI scale to any mobile device screen, as well as ensuring all UI elements are scaled correctly with no clipping issues or overlapping UI elements. This would be quite simple to implement as it wouldn't require much modification. This would likely be a fairly high priority if I continued development of this solution, as it would allow more users to easily play the game.

Requirement	Justification
Keyboard controls	This would make the game easier to navigate and play by pressing keyboard keys instead of clicking buttons

In order to achieve this requirement, I would bind certain keys to functions on the frontend, mapping these functions to certain actions. This would further meet the success criteria of an accessible UI, but as it was low priority, it was not implemented. As it was requested by a stakeholder, this should be a priority if the solution continues to be maintained and developed, aiding in accessibility of the solution.

Full Code Listing

Root directory

Index.js

```
const express = require('express');
const app = express();
const port = 80;
const gameRoutes = require('./routes/game');
const { createDatabase } = require('./logic/databaseManagement.js');

// Create the database and tables if they dont exist
createDatabase();

// Static html and js files will be served from the public directory, as that's common
practice
app.use(express.static('public'));

// Middleware to parse JSON request bodies
app.use(express.json());

// Use game routes from routes/game.js
app.use('/api/game', gameRoutes);

// Endpoint for testing connectivity
app.get('/api/ping', (req, res) => {
    res.send('Pong');
});


app.listen(port, () => {
    console.log(`Server running on port ${port}`);
});
```

/logic

ai.js

```
function aiMoveEasy(grid) { // Random targetting
    let row = Math.floor(Math.random() * grid.length); // Choose a random row
    let col = Math.floor(Math.random() * grid.length); // Choose a random column
    // While the chosen grid square has already been attacked, repeat the random selection
    console.log(`AI selecting row ${row}, col ${col}`);
    while (grid[row][col] === 3 || grid[row][col] === 2) {
        row = Math.floor(Math.random() * grid.length);
        col = Math.floor(Math.random() * grid.length);
    }
    return { row, col }; // Return both the row and column in separate variables
}

function aiMoveMed(grid) { // Grid targetting - attacks adjacent cells of hits
    // Check for existing hits on the board
    const hits = [];
    for (let r = 0; r < grid.length; r++) {
        for (let c = 0; c < grid[r].length; c++) {
            if (grid[r][c] === 3) { // The cell is hit (not sunk though)
                hits.push({ row: r, col: c }); // Put the coordinates in the hits array
            }
        }
    }

    // If there is at least 1 hit cell
    if (hits.length > 0) {
        const adjacentCells = [];

        for (const hit of hits) {
            // Detect ship orientation by checking for adjacent hits
            let orientation = null; // Unknown as of now

            // Check for hit above or below (vertical orientation)
            const hasVerticalHit = hits.some(h => h.col === hit.col && Math.abs(h.row -
hit.row) === 1);
            // That checks if any of the hits in the hits array share the same column and
are one row different

            // Check for hit left or right (horizontal orientation)
            const hasHorizontalHit = hits.some(h => h.row === hit.row && Math.abs(h.col -
hit.col) === 1);
            // That checks if any of the hits in the hits array share the same row and are
one column different
```

```
            if (hasVerticalHit) {
                orientation = 'V';
            } else if (hasHorizontalHit) {
                orientation = 'H';
            } // Otherwise the orientation is still unknown

            const directions = [
                { row: hit.row - 1, col: hit.col, orientationType: 'V' }, // up
                { row: hit.row + 1, col: hit.col, orientationType: 'V' }, // down
                { row: hit.row, col: hit.col - 1, orientationType: 'H' }, // left
                { row: hit.row, col: hit.col + 1, orientationType: 'H' }  // right
            ];

            for (const direction of directions) {
                if (orientation && direction.orientationType !== orientation) {
                    continue; // Skip this direction if orientation is known and doesnt
match orientationType
                }

                // Check if the cell is within bounds and isnt already attacked
                if (direction.row >= 0 &&  // Row in range (lower bound)
                    direction.row < grid.length && // Row in range (upper bound)
                    direction.col >= 0 && // Column in range (lower bound)
                    direction.col < grid[direction.row].length && // Column in range
(upper bound)
                    grid[direction.row][direction.col] !== 3 &&  // Not already hit
                    grid[direction.row][direction.col] !== 4 && // Not already sunk
                    grid[direction.row][direction.col] !== 2) // Not already missed
                {
                    // Validation passed
                    adjacentCells.push({ row: direction.row, col: direction.col }); // Add
to potential target list
                }
            }
        }

        // If there are valid adjacent cells then pick one randomly
        if (adjacentCells.length > 0) {
            const target = adjacentCells[Math.floor(Math.random() *
adjacentCells.length)];
            return target; // End here
        }
    }

    // If no hits are foundthen resort to using checkerboard pattern (a bit more effective
than random)
    const checkerboardCells = [];
    for (let r = 0; r < grid.length; r++) {
```

```
        for (let c = 0; c < grid[r].length; c++) {
            if ((r + c) % 2 === 0 &&
                grid[r][c] !== 3 &&
                grid[r][c] !== 4 &&
                grid[r][c] !== 2) {
                checkerboardCells.push({ row: r, col: c });
            }
        }
    }

    // If checkerboard cells available then use one
    if (checkerboardCells.length > 0) {
        const target = checkerboardCells[Math.floor(Math.random() *
checkerboardCells.length)];
        console.log( Opponent using checkerboard pattern: row ${target.row}, col
${target.col} );
        return target;
    }

    // Fallback to completely random if no checkerboard cells are left
    return aiMoveEasy(grid); // Returns { row, col }
}

function aiMoveHard(grid) {
    // I will add this function at a later stage of development.
}

module.exports = { aiMoveEasy, aiMoveMed, aiMoveHard };
```

databaseManagement.js

```
const Database = require('better-sqlite3');
const path = require('path');

// Path for the database file
const dbPath = path.join(__dirname, '..', 'data.db');

// Create the database
function createDatabase() {
    const db = new Database(dbPath);

    // Enable foreign keys so playerID can be used in the other tables
    db.pragma('foreign_keys = ON');

    // Player table
    db.exec(
        CREATE TABLE IF NOT EXISTS Player (
            PlayerID INTEGER PRIMARY KEY AUTOINCREMENT,
            DisplayName TEXT NOT NULL CHECK(length(DisplayName) >= 3 AND
length(DisplayName) <= 50)
        )
    `);

    // Game table
    db.exec(
        CREATE TABLE IF NOT EXISTS Game (
            GameID INTEGER PRIMARY KEY AUTOINCREMENT,
            PlayerID INTEGER NOT NULL,
            Winner TEXT DEFAULT 'Unknown',
            Duration REAL NOT NULL CHECK(Duration > 0),
            Timestamp INTEGER NOT NULL CHECK(Timestamp > 0),
            FOREIGN KEY (PlayerID) REFERENCES Player(PlayerID)
        )
    `);

    // Stats table
    db.exec(
        CREATE TABLE IF NOT EXISTS Stats (
            PlayerID INTEGER PRIMARY KEY NOT NULL,
            GamesPlayed INTEGER NOT NULL CHECK(GamesPlayed >= 0),
            Wins INTEGER NOT NULL CHECK(Wins >= 0),
            AverageTime REAL,
            FOREIGN KEY (PlayerID) REFERENCES Player(PlayerID)
        )
     );

    db.close();
```

```
}

// Get the database connection
function getDatabase() {
    const db = new Database(dbPath);
     db.pragma('foreign_keys = ON'); // Enable foreign keys on the database
    return db; // Return the db connection
}

module.exports = { createDatabase, getDatabase };
```

game.js

```
const { generateShips, validateShipPlacement } = require('./ships');
const { aiMoveEasy, aiMoveMed } = require('./ai');
const { updateStats } = require('./stats');


class Board {
    constructor() {
        this.grid = this.createEmptyGrid();
        this.ships = [];
    }

    // Create a 10x10 grid for the game board
    createEmptyGrid() {
        return Array.from({ length: 10 }, () => Array(10).fill(0)); // No point setting a
height attribute because grid is square
    }
}

class Game {
    constructor(difficulty = "easy", playerID = null) {
        this.gridSize = 10;
        this.difficulty = difficulty;
        this.playerBoard = new Board(); // Player's board
        this.aiBoard = new Board(); // AI's board
        this.isPlayerTurn = true;
        this.gameOver = false;
        this.winner = null;
        this.startedAt = new Date();
        this.playerID = playerID; // Store player ID for keeping stats separate

        this.placeAIShips();
    }
```

```
    // Randomly place AI ships
    placeAIShips() {
        const shipsToPlace = generateShips(); // Creates a list of Ship objects
        shipsToPlace.forEach(ship => {
        let valid = false;
        while (!valid) {
            const orientation = Math.random() < 0.5 ? 'H' : 'V'; // Choose a random
orientation
            const row = Math.floor(Math.random() * this.gridSize);
            const col = Math.floor(Math.random() * this.gridSize);
            valid = validateShipPlacement(ship.size, row, col, orientation,
this.aiBoard.grid);
            if (valid) { // Ship placement is valid (not out of bounds and not
overlapping)
                this.placeShipOnGrid(ship.size, row, col, orientation, this.aiBoard.grid);
// Place each ship on the AI's board
                this.aiBoard.ships.push({ ...ship, row, col, orientation, hits: 0 });
                break;
            }
        }
        });
    }

    // Function for placing ships on the grid
    placeShipOnGrid(size, startRow, startCol, orientation, grid) {
        for (let i = 0; i < size; i++) { // For each grid cell the ship occupies
            if (orientation === "H") grid[startRow][startCol + i] = 1; // Set to 1 to
indicate there is a ship
            else grid[startRow + i][startCol] = 1;
        }
    }

    // Function for player placing a ship
    placeShip(shipType, startRow, startCol, orientation) {
        const ships = generateShips();
        const ship = ships.find(s => s.name.toLowerCase() === shipType?.toLowerCase()); //
Use lowercase incase of case differences
        if (!ship) return { success: false, message: "Invalid ship type" };

        if(orientation !== "H" && orientation !== "V") {
            return { success: false, message: "Invalid orientation" };
        }
        if(startRow < 0 || startRow >= this.gridSize || startCol < 0 || startCol >=
this.gridSize) {
            return { success: false, message: "Invalid co-ordinate" };
        }
```

```
        const valid = validateShipPlacement(ship.size, startRow, startCol, orientation,
this.playerBoard.grid); // Validate the ship placement
        if (!valid) return { success: false, message: "Invalid ship placement" }; //
Return both a boolean flag and a message

        this.placeShipOnGrid(ship.size, startRow, startCol, orientation,
this.playerBoard.grid);
        this.playerBoard.ships.push({ ...ship, row: startRow, col: startCol, orientation,
hits: 0 });
        return { success: true, message: "" }; // A message isnt really relevant if its
successful
    }

    checkAndUpdateSunkShips(board) {
        for (const ship of board.ships) {
            let hitCount = 0;
            for (let i = 0; i < ship.size; i++) {
                let r = ship.row;
                let c = ship.col;
                if (ship.orientation === "H"){
                    c += i;
                } else {
                    r += i;
                }

                if (board.grid[r][c] === 3) { // Check if that part of the ship is hit
                    hitCount += 1;
                }
            }
            if (hitCount === ship.size) { // All parts of the ship are hit
                // Mark the ship as sunk on the grid
                for (let i = 0; i < ship.size; i++) {
                    let r = ship.row;
                    let c = ship.col;
                    if (ship.orientation === "H") c += i;
                    else r += i;
                    board.grid[r][c] = 4; // 4 indicates sunk ship
                }
            }
        }
    }

    // Function to process player move
    processPlayerMove(row, col) {
        if (!this.isPlayerTurn) return { error: "Not player's turn" }; // Prevent player
from making two moves in a row
        console.log( Player attacking row ${row}, col ${col} );
```

```
        // Check if row or column is out of range
        if (row < 0 || row >= this.gridSize || col < 0 || col >= this.gridSize) {
            return { error: "Invalid coordinates" };
        }

        if (this.aiBoard.grid[row][col] === 3 || this.aiBoard.grid[row][col] === 2) { // 
Prevent attacking the same grid square twice
            return { error: "Cell already attacked" }; // Return a error message to the 
client
        }

        /*
            0 = water
            1 = ship present
            2 = missed attack
            3 = hit ship
            4 = sunk ship
        */

        let result = "miss"; // Set default result as miss
        if (this.aiBoard.grid[row][col] === 1) { // 1 indicates a ship is present in that 
coordinate
            this.aiBoard.grid[row][col] = 3; // Mark the grid square as hit
            result = "hit"; // Update result to hit
        } else {
            this.aiBoard.grid[row][col] = 2; // Mark the grid square as miss
        }

        this.checkAndUpdateSunkShips(this.aiBoard); // Check if any opponent ships are 
sunk after the move

        this.isPlayerTurn = false; // Set the player turn to false, indicating it's the 
AI's turn
        if (this.checkGameOver(this.aiBoard.grid)) { // Check if all AI ships are sunk
            this.gameOver = true;
            this.winner = "Player";
            updateStats(this); // Update stats if game is over
        }

        return { result, gameOver: this.gameOver }; // Return the result of the move and 
also whether or not the game is over
    }

    // Function to process AI moves
    aiMove() {
        if (this.gameOver) return { result: "Game already over" }; // Make sure AI doesn't 
move if game is over
```

```
        const difficulty = this.difficulty;

        let move;
        if(difficulty === "easy") {
            move = aiMoveEasy(this.playerBoard.grid); // Returns a row and column
        } else if (difficulty === "medium") {
            move = aiMoveMed(this.playerBoard.grid);
        }
        // Move shouldnt ever be undefined so a fallback isnt necessary

        const { row, col } = move;
        let result = "miss";
        if (this.playerBoard.grid[row][col] === 1) { // Indicates a ship is present
            this.playerBoard.grid[row][col] = 3; // Mark the grid square as hit
            result = "hit";
        } else {
            this.playerBoard.grid[row][col] = 2;
        }

        this.checkAndUpdateSunkShips(this.playerBoard); // Check if any player ships are
sunk after the move

        this.isPlayerTurn = true;
        if (this.checkGameOver(this.playerBoard.grid)) { // Check if all player ships are
sunk
            this.gameOver = true;
            this.winner = "AI";
            updateStats(this); // Update stats if game is over
        }

        return { row, col, result, gameOver: this.gameOver }; // Return the AI's move to
the client
    }

    // Check if all ships are sunk
    checkGameOver(board) {
        for (let row = 0; row < this.gridSize; row++) {
            for (let col = 0; col < this.gridSize; col++) {
                if (board[row][col] === 1){
                    return false; // 1 indicates theres a ship that hasnt been hit yet
                }
            }
        }
        return true;
    }
}

module.exports = { Board, Game };
```

ships.js

```
function validateShipPlacement(size, startRow, startCol, orientation, grid) {
  const gridSize = grid.length;

  if (orientation === "H" && startCol + size > gridSize) return false;
  if (orientation === "V" && startRow + size > gridSize) return false;

  // Check the area including the ship and its adjacent cells to prevent touching
  let minRow, maxRow, minCol, maxCol;
  if (orientation === "H") {
    minRow = Math.max(0, startRow - 1);
    maxRow = Math.min(gridSize - 1, startRow + 1);
    minCol = Math.max(0, startCol - 1);
    maxCol = Math.min(gridSize - 1, startCol + size);
  } else { // Vertical
    minRow = Math.max(0, startRow - 1);
    maxRow = Math.min(gridSize - 1, startRow + size);
    minCol = Math.max(0, startCol - 1);
    maxCol = Math.min(gridSize - 1, startCol + 1);
  }

  for (let r = minRow; r <= maxRow; r++) {
    for (let c = minCol; c <= maxCol; c++) {
      if (grid[r][c] !== 0) return false;
    }
  }

  // Ship placement valid
  return true;
}



class Ship {
  constructor(name, size, status = 0) {
    this.name = name;
    this.size = size;
    this.status = status;
  }
}

function generateShips() {
  return [
    new Ship("Carrier", 5),
    new Ship("Battleship", 4),
    new Ship("Cruiser", 3),
    new Ship("Submarine", 3),
    new Ship("Destroyer", 2)
```

```
  ];
}

module.exports = { validateShipPlacement, Ship, generateShips };
```

stats.js

```
const { getDatabase } = require('./databaseManagement');

// Create a new player
function createPlayer(displayName) {
    const db = getDatabase();
    try {
        const stmt = db.prepare('INSERT INTO Player (DisplayName) VALUES (?)');
        const info = stmt.run(displayName);
        const playerID = info.lastInsertRowid;

        // Initialize stats for the new player
        const statsStmt = db.prepare('INSERT INTO Stats (PlayerID, GamesPlayed, Wins,
AverageTime) VALUES (?, 0, 0, 0)');
        statsStmt.run(playerID);

        db.close();
        return playerID;
    } catch (err) {
        db.close();
        throw err;
    }
}

// Update player display name
function updatePlayerName(playerID, displayName) {
    const db = getDatabase();
    try {
        const stmt = db.prepare('UPDATE Player SET DisplayName = ? WHERE PlayerID = ?');
        const info = stmt.run(displayName, playerID);
        db.close();
        return info.changes;
    } catch (err) {
        db.close();
        throw err;
```

```
    }
}

// Get player by ID
function getPlayer(playerID) {
    const db = getDatabase();
    try {
        const stmt = db.prepare('SELECT * FROM Player WHERE PlayerID = ?');
        const row = stmt.get(playerID);
        db.close();
        return row;
    } catch (err) {
        db.close();
        throw err;
    }
}

function getStats(playerID) { // Retrieve stats from database
    const db = getDatabase();
    try {
        const stmt = db.prepare(`
            SELECT
                s.GamesPlayed,
                s.Wins,
                s.AverageTime,
                (s.GamesPlayed - s.Wins) as Losses,
                p.DisplayName
            FROM Stats s
            JOIN Player p ON s.PlayerID = p.PlayerID
            WHERE s.PlayerID = ?
         );
        const row = stmt.get(playerID);
        db.close();

        if (!row) {
            return {
                gamesPlayed: 0,
                wins: 0,
                losses: 0,
                averageTime: 0,
                displayName: null
            };
        }

        return {
            gamesPlayed: row.GamesPlayed,
            wins: row.Wins,
            losses: row.Losses,
```

```
            averageTime: row.AverageTime || 0,
            displayName: row.DisplayName
        };
    } catch (err) {
        db.close();
        throw err;
    }
}

function updateStats(game) { // Update the statistics after a game concludes
    console.log("Updating stats...");

    if (!game.playerID) {
        console.error("No playerID found in game object");
        return;
    }

    const db = getDatabase();
    const timeTaken = (new Date() - game.startedAt) / 1000; // Time in seconds
    const timestamp = Math.floor(new Date().getTime() / 1000); // Unix timestamp in
seconds

    try {
        // Use transaction for atomicity
        const insertGame = db.prepare('INSERT INTO Game (PlayerID, Winner, Duration,
Timestamp) VALUES (?, ?, ?, ?)');
        const getStatsStmt = db.prepare('SELECT GamesPlayed, Wins, AverageTime FROM Stats
WHERE PlayerID = ?');
        const updateStatsStmt = db.prepare('UPDATE Stats SET GamesPlayed = ?, Wins = ?,
AverageTime = ? WHERE PlayerID = ?');

        // Start transaction
        const transaction = db.transaction(() => {
            // Insert game record
            const gameInfo = insertGame.run(game.playerID, game.winner, timeTaken,
timestamp);
            console.log(`Game record inserted with ID: ${gameInfo.lastInsertRowid}`);

            // Get current stats
            const row = getStatsStmt.get(game.playerID);

            const newGamesPlayed = row.GamesPlayed + 1;
            const newWins = game.winner === "Player" ? row.Wins + 1 : row.Wins;
            const totalTime = (row.AverageTime * row.GamesPlayed) + timeTaken;
            const newAverageTime = totalTime / newGamesPlayed;

            // Update stats
            updateStatsStmt.run(newGamesPlayed, newWins, newAverageTime, game.playerID);
```

```
            console.log("Stats updated successfully");
        });

        transaction();
        db.close();
    } catch (err) {
        db.close();
        console.error('Error updating stats:', err);
        throw err;
    }
}


module.exports = { getStats, updateStats, createPlayer, updatePlayerName, getPlayer };
```

/routes

game.js

```
const express = require('express');

const { getStats, createPlayer, updatePlayerName, getPlayer } =
require('../logic/stats.js');

const {Game} = require('../logic/game.js');
const app = express.Router();
let games = {}; // Store active games in memory

function validateDifficultyString(difficulty){
    const validDifficulties = ['easy', 'medium']; // Currently only easy and medium is
implemented
    return validDifficulties.includes(difficulty);
}

app.post('/start-game', (req, res) => {
    try {
        const difficulty = req.body.difficulty || 'easy'; // Use easy as a default
difficulty incase none is specified
        const playerID = req.body.playerID; // Get playerID from request

        if (!playerID) {
            return res.status(400).json({ error: "Player ID is required" });
```

```
        }

        if (!validateDifficultyString(difficulty)) {
            return res.status(400).json({ error: "Invalid difficulty level" });
        }
        const gameId = Date.now().toString(); // Unique ID based on current timestamp. Not
suitable long-term, but will work for now

        games[gameId] = new Game(difficulty, playerID);
        return res.json({ gameId, message: 'Game started' });
    } catch (error) {
        console.error('Error starting game:', error);
        return res.status(500).json({ error: 'Internal server error' });
    }
});

app.post('/place-ship', (req, res) => { // Endpoint used when the player places their
ships
    const { gameId, shipType, startRow, startCol, orientation } = req.body; // Retrieve
these values from the request body
    const game = games[gameId];
    if (!game) return res.status(404).json({ error: "Game not found" }); // Return 404 if
the game ID is invalid

    const result = game.placeShip(shipType, startRow, startCol, orientation); // Call the
placeShip method
    if (!result.success) {
        return res.status(400).json({ error: result.message }); // Return 400 if something
is wrong
    }
    res.json(result);
});

app.post("/player-move", (req, res) => { // Endpoint used when the player makes a move
    const { gameId, row, col } = req.body;
    const game = games[gameId];
    if (!game) return res.status(404).json({ error: "Game not found" });

    const result = game.processPlayerMove(row, col); // Process the player's move
    if (result.error) {
        return res.status(400).json({ error: result.error }); // Return 400 if there was
an error
    }
    if (!game.gameOver) {
        // If the game isn't over, immediately make the AI's move and return the result to
the user
        const aiResult = game.aiMove();
        return res.json({ playerResult: result, aiResult });
```

```
    }
    res.json({ playerResult: result, aiResult: null }); // This happens if the game is
over. Result contains the move result and also the gameOver status
});

app.post("/ai-move", (req, res) => { // Endpoint used to get the AI's move - mainly for
testing purposes
    const { gameId } = req.body;
    const game = games[gameId];
    if (!game) return res.status(404).json({ error: "Game not found" });

    const result = game.aiMove(); // Process the AI's move
    if (result.error) {
        return res.status(400).json({ error: result.error }); // Return 400 if there was
an error
    }
    res.json({ aiResult: result }); // Return the AI's move result
});

app.get("/check-game-over", (req, res) => { // Endpoint to check if the game is over
    const { gameId } = req.query;
    const game = games[gameId];
    if (!game) return res.status(404).json({ error: "Game not found" });

    if(game.gameOver) { // If the game is already over, return the winner
        return res.json({ gameOver: true, winner: game.winner });
    }

    if (game.checkGameOver(game.aiBoard.grid)) { // Check if all AI ships are sunk
        this.gameOver = true;
        this.winner = "Player";
        updateStats(game); // Update stats if game is over
    } else if (game.checkGameOver(game.playerBoard.grid)) { // Check if all player ships
are sunk
        this.gameOver = true;
        this.winner = "AI";
        updateStats(game); // Update stats if game is over
    }

    res.json({ gameOver: game.gameOver, winner: game.winner });
});

app.get("/get-game-state", (req, res) => { // Endpoint to retrieve the current game state
    /* This endpoint will return:
        masked AI grid
        full player grid
        correct turn
        correct gameOver flag
```

```
    */
    const { gameId } = req.query;
    const game = games[gameId];
    if (!game) return res.status(404).json({ error: "Game not found" });

    // Mask the AI grid to hide remaining ships
    const maskedAIBoard = game.aiBoard.grid.map(row =>
        row.map(cell => (cell === 1 ? 0 : cell)) // Replaces existing ships with empty
spaces (0)
    );

    res.json({
        playerBoard: game.playerBoard.grid,
        aiBoard: maskedAIBoard,
        isPlayerTurn: game.isPlayerTurn,
        gameOver: game.gameOver,
        winner: game.winner
    });
});


app.get("/stats", (req, res) => { // Endpoint to retrieve game statistics
    try {
        const { playerID } = req.query;
        if (!playerID) {
            return res.status(400).json({ error: "Player ID is missing" });
        }
        const stats = getStats(parseInt(playerID)); // Retrieve stats using the function
in stats.js
        res.json(stats);
    } catch (error) {
        console.error('Error fetching stats:', error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

// Player management endpoints
app.post("/create-player", (req, res) => {
    try {
        const { displayName } = req.body;
        if (!displayName) {
            return res.status(400).json({ error: "Display name is required" });
        }
        const playerID = createPlayer(displayName);
        res.json({ playerID, displayName });
    } catch (error) {
        console.error('Error creating player:', error);
        res.status(400).json({ error: 'Invalid display name.' });
```

```
    }
});

app.put("/update-player-name", (req, res) => {
    try {
        const { playerID, displayName } = req.body;
        if (!playerID || !displayName) {
            return res.status(400).json({ error: "Player ID and display name are required"
});
        }
        updatePlayerName(parseInt(playerID), displayName);
        res.json({ success: true, displayName });
    } catch (error) {
        console.error('Error updating player name:', error);
        res.status(400).json({ error: 'Invalid display name.' });
    }
});

app.get("/player", (req, res) => {
    try {
        const { playerID } = req.query;
        if (!playerID) {
            return res.status(400).json({ error: "Player ID is missing" });
        }
        const player = getPlayer(parseInt(playerID));
        if (!player) {
            return res.status(404).json({ error: "Player not found" });
        }
        res.json(player);
    } catch (error) {
        console.error('Error fetching player:', error);
        res.status(500).json({ error: 'Internal server error' });
    }
});

module.exports = app;
```

/public

Index.html

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <title>Battleships</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <div id="sidebar">
        <button id="playBtn">Play Game</button>
        <button id="statsBtn">Statistics</button>

        <div class="player-info">
            <label>
                Player Name:
                <input type="text" id="playerNameInput" placeholder="Enter your name"
minlength="3" maxlength="50">
            </label>
            <button id="updateNameBtn">Update Name</button>
            <span id="playerNameStatus"></span>
        </div>
    </div>

    <div id="main-content">
        <div id="play-page">
            <h1>Battleships</h1>

            <div class="top-controls">

                <button id="startGameBtn">Start Game</button>
                <label>
                    Difficulty:
                    <select id="difficultySelect">
                        <option value="easy">Easy</option>
                        <option value="medium">Medium</option>
                    </select>
                </label>

                <p id="status">Status: Not started</p>
```

```
            <div id="placementControls">
                <h2>Place Your Ships</h2>

                <label>
                    Ship:
                    <select id="shipSelect">
                        <option value="Carrier">Carrier (5)</option>
                        <option value="Battleship">Battleship (4)</option>
                        <option value="Cruiser">Cruiser (3)</option>
                        <option value="Submarine">Submarine (3)</option>
                        <option value="Destroyer">Destroyer (2)</option>
                    </select>
                </label>

                <label>
                    Orientation:
                    <select id="orientationSelect">
                        <option value="horizontal">Horizontal</option>
                        <option value="vertical">Vertical</option>
                    </select>
                </label>

                <p id="placementStatus">Place all ships to begin.</p>
            </div>

        </div>

        <div class="boards">
            <div>
                <h2>Your Board</h2>
                <table id="playerBoard"></table>
            </div>

            <div>
                <h2>Enemy Board</h2>
                <table id="aiBoard"></table>
            </div>
        </div>
    </div>

    <div id="stats-page" style="display: none;">
        <h1>Statistics</h1>
        <div id="stats-display">
            <p id="games-played-counter"></p>
            <p id="player-wins-counter"></p>
            <p id="ai-wins-counter"></p>
            <p id="win-rate"></p>
            <p id="avg-game-time"></p>
```

```
            </div>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

script.js

```
let gameId = null;
const gridSize = 10;
let currentPlayerID = null;

const playerBoardEl = document.getElementById("playerBoard");
const aiBoardEl = document.getElementById("aiBoard");
const statusEl = document.getElementById("status"); // Status element for messages about
game state etc

document.getElementById("startGameBtn").addEventListener("click", startGame);
document.getElementById("playBtn").addEventListener("click", showPlay);
document.getElementById("statsBtn").addEventListener("click", showStats);
document.getElementById("updateNameBtn").addEventListener("click", updatePlayerName);

// Initialise player when the page loads
window.addEventListener("DOMContentLoaded", async () => {
    await initialisePlayer();
});

// Initialise or load player
async function initialisePlayer() {
    const storedPlayerID = localStorage.getItem("playerID");
    if (storedPlayerID) { // Load an existing player
        try {
            const res = await fetch(`/api/game/player?playerID=${storedPlayerID}`);
            if (res.ok) {
                const player = await res.json();
                currentPlayerID = player.PlayerID;
                document.getElementById("playerNameInput").value = player.DisplayName;
                document.getElementById("playerNameStatus").textContent =
 ${player.DisplayName} loaded`;
                return;
            }
        } catch (error) {
            console.error("Error loading player:", error);
```

```
        }
    }

    // If the user hasnt played before, create a new guest player
    await createNewPlayer("Player" + Math.floor(Math.random() * 1000)); // Use a 3 digit
random number to reduce chances of clashes
}

// Create a new player
async function createNewPlayer(displayName) {
    try {
        const res = await fetch("/api/game/create-player", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({displayName})
        });

        const data = await res.json();
        if (res.ok) {
            currentPlayerID = data.playerID;
            localStorage.setItem("playerID", currentPlayerID);
            document.getElementById("playerNameInput").value = data.displayName;
            document.getElementById("playerNameStatus").textContent =  Player created:
${data.displayName}`;
        } else {
            document.getElementById("playerNameStatus").textContent = data.error || "Error
creating player";
        }
    } catch (error) {
        console.error("Error creating player:", error);
        document.getElementById("playerNameStatus").textContent = "Error creating player";
    }
}

// Update player name
async function updatePlayerName() {
    const newName = document.getElementById("playerNameInput").value.trim();

    if (!newName || newName.length < 3 || newName.length > 50) {
        document.getElementById("playerNameStatus").textContent = "Name invalid";
        return;
    }

    try {
        const res = await fetch("/api/game/update-player-name", {
            method: "PUT",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ playerID: currentPlayerID, displayName: newName })
```

```
        });

        const data = await res.json();
        if (res.ok) {
            document.getElementById("playerNameStatus").textContent = `Name updated to:
${data.displayName}`;
        } else {
            document.getElementById("playerNameStatus").textContent = data.error || "Error
updating name";
        }
    } catch (error) {
        console.error("Error updating player name:", error);
        document.getElementById("playerNameStatus").textContent = "Error updating name";
    }
}

// Show the main game page (shown by default)
function showPlay() {
    document.getElementById("play-page").style.display = "block";
    document.getElementById("stats-page").style.display = "none";
}

// Show the stats page
function showStats() {
    document.getElementById("play-page").style.display = "none";
    document.getElementById("stats-page").style.display = "block";
    loadStats(); // Dont really need to await it because nothing requires the results
}

async function loadStats() {
    if (!currentPlayerID) {
        document.getElementById("playerNameStatus").textContent = "No player loaded";
        return;
    }

    const res = await fetch(`/api/game/stats?playerID=${currentPlayerID}`);
    const stats = await res.json();

    const gamesPlayedCounter = document.getElementById("games-played-counter");
    const playerWinsCounter = document.getElementById("player-wins-counter");
    const aiWinsCounter = document.getElementById("ai-wins-counter");
    const winRateDisplay = document.getElementById("win-rate");
    const avgGameTimeDisplay = document.getElementById("avg-game-time");

    const winRate = stats.gamesPlayed > 0 ? ((stats.wins / stats.gamesPlayed) *
100).toFixed(2) : 0;
    gamesPlayedCounter.textContent =  Games Played: ${stats.gamesPlayed} ;
    playerWinsCounter.textContent = `Player Wins: ${stats.wins || 0}`;
```

```
    aiWinsCounter.textContent = `AI Wins: ${stats.losses || 0}`;
    winRateDisplay.textContent = `Win Rate: ${winRate}%`;
    avgGameTimeDisplay.textContent = `Average Game Time: ${stats.averageTime.toFixed(2)}
seconds ; // Limits to 2 decimal places
}

async function startGame() {
    if (!currentPlayerID) {
        statusEl.textContent = "Error: No player is set";
        return;
    }

    const difficulty = document.getElementById("difficultySelect").value.toLowerCase();

    const res = await fetch("/api/game/start-game", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ difficulty: difficulty, playerID: currentPlayerID })
    });

    const data = await res.json();
    gameId = data.gameId;
    statusEl.textContent = "Game started";
    gameOver = false;
    await updateGameState();
}

let gameOver = false;

async function updateGameState() {
    const res = await fetch( /api/game/get-game-state?gameId=${gameId} );
    const data = await res.json();

    renderBoard(playerBoardEl, data.playerBoard, false);
    renderBoard(aiBoardEl, data.aiBoard, true);

    if (data.gameOver) {
        statusEl.textContent = `Game Over. Winner: ${data.winner}`;
        gameOver = true;
    }
}

function renderBoard(table, grid, isEnemy) {
    table.innerHTML = "";

    for (let r = 0; r < gridSize; r++) {
        const row = document.createElement("tr"); // tr = table row
```

```
        for (let c = 0; c < gridSize; c++) {
            const cell = document.createElement("td"); // td = table data cell
            const value = grid[r][c];

            if (value === 3) cell.className = "hit";
            else if (value === 4) cell.className = "sunk";
            else if (value === 2) cell.className = "miss";
            else if (value === 1 && !isEnemy) cell.className = "ship";
            else cell.className = "water";

            // Ship placement
            if (!isEnemy && placementStage) {
                // Attach an event listener to each cell for placing ships
                cell.addEventListener("click", () => placeShip(r, c));
            }

            // Attacking AI (only after placement)
            if (isEnemy && !placementStage && value === 0) {
                // Attach an event listener to each cell for attacking
                cell.addEventListener("click", () => playerMove(r, c));
            }

            row.appendChild(cell);
        }

        table.appendChild(row);
    }
}


async function playerMove(row, col) {
    if (gameOver) return;
    const res = await fetch("/api/game/player-move", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ gameId, row, col })
    });

    const data = await res.json();

    if (data.error) {
        statusEl.textContent = data.error;
        return;
    }

    await updateGameState();
}
```

```
let placementStage = true;

const ships = {
    Carrier: 5,
    Battleship: 4,
    Cruiser: 3,
    Submarine: 3,
    Destroyer: 2
};

const placedShips = new Set();


async function placeShip(row, col) {
    const shipName = document.getElementById("shipSelect").value;

    if (placedShips.has(shipName)) {
        statusEl.textContent = "Ship already placed";
        return;
    }

    let orientationVal = document.getElementById("orientationSelect").value;
    const orientation = orientationVal === "horizontal" ? "H" : "V";

    const res = await fetch("/api/game/place-ship", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
            gameId,
            shipType: shipName,
            startRow: row,
            startCol: col,
            orientation
        })
    });

    const data = await res.json();

    if (!data.success) {
        statusEl.textContent = data.message;
        return;
    }

    placedShips.add(shipName);
    statusEl.textContent = `${shipName} placed`;
```

```
    if (placedShips.size === Object.keys(ships).length) {
        placementStage = false;
        document.getElementById("placementStatus").textContent = "All ships placed.";
    }

    await updateGameState();
}
```

Style.css

```
body {
  font-family: Arial, sans-serif;
}

table {
  border-collapse: collapse;
}

td {
  width: 30px;
  height: 30px;
  border: 1px solid black;
  text-align: center;
  cursor: pointer;
}

.water {
  background-color: #cce6ff;
}

.ship {
  background-color: #999;
}

.hit {
  background-color: red;
}

.sunk {
  background-color: rgb(98, 47, 0);
}

.miss {
  background-color: white;
}
```

```
.boards {
  display: flex;
  gap: 40px;
  justify-content: center;
}

#sidebar {
  position: fixed;
  left: 0;
  top: 0;
  width: 200px;
  height: 100%;
  background-color: #f0f0f0;
  padding-top: 60px;
  box-sizing: border-box;
}

#sidebar button {
  display: block;
  width: 100%;
  margin-bottom: 10px;
  padding: 10px;
  background-color: #606060;
  color: white;
  border: none;
  border-radius: 0;
  cursor: pointer;
  font-size: 1em;
}

#sidebar button:hover {
  background-color: #454545;
}

#main-content {
  margin-left: 220px;
  padding: 20px;
}

h1 {
  text-align: center;
  margin-top: 0;
}

button {
    font-size: 1em;
    padding: 10px 20px;
```

```
    background-color: #528be7;
    color: white;
    border-radius: 10px;
    border: none;
    cursor: pointer;
}

.top-controls {
    margin-left: 100px;
}

.player-info {
    text-align: center;
    margin: 20px 0;
    padding: 15px;
    background-color: #f5f5f5;
    border-radius: 8px;
    position: absolute;
    left: 0;
    bottom: 0;
    width: 150px;
}

.player-info input {
    padding: 8px;
    font-size: 1em;
    border: 1px solid #ccc;
    border-radius: 4px;
    margin-right: 10px;
    width: 10vw;
}

.player-info button {
    padding: 8px 15px;
    font-size: 0.9em;
}

#playerNameStatus {
    margin-left: 10px;
    color: #528be7;
    font-weight: bold;
}
```